

TQS®

Manual de programação Python

TQS



As informações contidas neste documento, incluindo links, telas e funcionamento de comandos estão sujeitos a alterações sem aviso prévio.

Nenhuma parte deste documento, ou qualquer outro documento/texto que acompanhe este software pode ser reproduzida ou transmitida de qualquer forma ou por qualquer meio (eletrônico, fotocópia, gravação, etc.) ou para qualquer finalidade sem a permissão expressa, por escrito, da TQS Informática Ltda.

O software TQS, seus subsistemas e programas e seus manuais são de autoria da TQS Informática Ltda. e são protegidos pela legislação de direitos autorais do Brasil, tratados internacionais e demais leis aplicáveis.

Todas as outras marcas comerciais pertencem a seus respectivos proprietários.

TQS®

Manual de programação Python Sumário

1. Introdução.....	1
1.1. Pacote e Módulos	1
1.2. Instalação e uso	2
1.3. Organização dos módulos	4
1.4. Algumas convenções usadas	4
2. Módulo TQSUtl.....	5
2.1. Constantes declaradas	5
2.2. Funções	5
2.3. Programa TSTUtl.py para teste do TQSUtl	6
3. Módulo TQSGeo	7
3.1. Programa TSTGeo.py de teste de TQSGeo.....	9
4. Módulo TQSDwg	10
4.1. Modo de funcionamento	10
4.2. Operações de arquivos com TQSDwg.file	10
4.3. Desenho com TQSDwg.Draw	12
4.3.1. Propriedades de TQSDwg.Draw	12
4.3.2. Métodos de TQSDwg.Draw	12
4.4. Cotagens com TQSDwg.dim.....	14
4.4.1. Atributos de cotagem	14
4.4.2. Constantes de cotagem	15
4.4.3. Métodos de cotagem.....	16
4.5. Estados de desenho com TQSDwg.settings	17
4.5.1. Constantes que definem o número do sistema	17
4.5.2. Propriedades de sistema.....	18
4.5.3. Outras propriedades de estado.....	19
4.6. Limites de desenho com TQSDwg.limits	21
4.6.1. Métodos de TQSDwg.limits	21
4.7. Leitura de desenho com TQSDwg.iterator	21
4.7.1. Tipos de elemento de desenho	22
4.7.2. Propriedades de um objeto gráfico lido	23
4.7.3. Métodos de TQSDwg.iterator	25
4.8. Tabela de níveis com TQSDwg.levelstable.....	26
4.9. Tabela de blocos com TQSDwg.blockstable.....	27
4.10. Referências externas e o TQSDwg.xreference.....	27
4.11. Edição de elementos com TQSDwg.edit	28
4.12. Leitura da tabela de plotagem associada com TQSDwg.plotting.....	29
4.13. Programa TSTDwg.py de teste do TQSDwg	30

4.13.1. Função TestarDesenho()	30
4.13.2. Função TestarEdit()	30
4.13.3. Função TestarCotagem()	30
4.13.4. Função TestarLeitura()	30
4.13.5. Função TestarGlobais()	31
4.13.6. Função TestarLerNiveis()	31
4.13.7. Função TestarLerBlocos()	31
4.13.8. Função TestarXRef()	31
4.13.9. Função TestarLerTabPlt()	31
5. Ferros inteligentes e o objeto TQSDwg.SmartRebar	33
5.1. Nomenclatura de ferros	33
5.1.1. Ferros retos	33
5.1.2. Ferros genéricos	34
5.1.3. Estribos de vigas	35
5.1.4. Estribos genéricos (de pilares)	35
5.1.5. Inserção de linhas de ferros	35
5.1.6. Raio de curvatura	36
5.1.7. Ganchos	36
5.1.8. Faixas de distribuição	37
5.1.9. Faixas de distribuição com comprimento variável	37
5.1.10. Faixas de distribuição de múltiplas posições	38
5.1.11. Identificação de posição	38
5.2. Atributos assumidos e arquivo de critérios	38
5.3. Definição de ferros	39
5.3.1. Dados gerais	39
5.3.2. Linha de ferro	41
5.3.3. Ferros retos	41
5.3.4. Ferros genéricos	43
5.3.5. Estribos de vigas	44
5.3.6. Grampos de vigas	46
5.3.7. Estribos genéricos (de pilares)	46
5.3.8. Posição do texto de identificação	47
5.3.9. Linha de chamada para identificação longe do ferro	49
5.3.10. Reposicionando textos de identificação existentes	49
5.3.11. Raios de curvatura e comprimentos de dobra	50
5.3.12. Leitura de linha de ferro	51
5.3.13. Alteração da linha de ferro	52
5.3.14. Outras operações geométricas com ferros	53
5.4. Faixas de distribuição	54
5.4.1. Leitura de faixa de distribuição	55
5.4.2. Faixas de distribuição variável	56
5.5. Faixa de distribuição de múltiplas posições	58
5.6. Identificação de posições	58
5.7. Cotagem de ponta de ferro	60

5.8. Linhas adicionais de ferro	60
5.9. Duplicação de um ferro	60
5.10. Controle de posições e o objeto TQSDwg. GlobalRebar	60
5.11. Iteração com ferros inteligentes.....	61
5.12. Programa TSTFer.py de teste de geração de ferros inteligentes no TQSDwg..	62
6. Módulo TQSJan	64
6.1. Métodos e propriedades de TQSJan	64
6.2. Métodos que interagem com o usuário	65
6.3. Eventos de TQSJan	66
6.4. Baixando o módulo wxPython.....	67
6.5. Programa TSTJan.py para teste de TQSJan	67
6.5.1. O que faz o programa TSTJan.py.....	68
7. Módulo TQSEag.....	70
7.1. Como uma aplicação do EAG é escolhida	70
7.2. Como uma extensão em Python é carregada.....	72
7.3. Menu PYMEN comum a todos os editores	72
7.4. Chamada do programa Python pelo editor.....	72
7.4.1. Reutilização de menus de editores.....	73
7.4.2. Rotinas Python chamado pelo .pymen	74
7.5. Pré-entrada de dados e o TQSEag.entry	75
7.6. Execução de comandos com TQSEag.exec.....	76
7.7. Mensagens na janela do editor com TQSEag.msg	76
7.8. Leitura de pontos e objetos com TQS.locate.....	77
7.9. Estados do editor com TQSEag.state	80
7.10. Menus de demonstração EAGFOR.PYMEN e programa EAGFor.py.....	84
7.10.1. Comando "Geração de desenho"	84
7.10.2. Comando "Acionamento de comandos do editor"	84
7.10.3. Comando "Entrada de pontos"	84
7.10.4. Comando "Seleção de elementos e movimentação".....	84
7.10.5. Comando "Estados do editor"	84
8. Módulo TQSBuild.....	85
8.1. Funções globais sem o objeto Building	86
8.2. Funções globais sem edifício aberto	86
8.3. Leitura e gravação com TQSBuilding.file	86
8.4. Dados gerais com TQSBuilding.project	87
8.5. Modelo de cálculo com TQSBuilding.model.....	89
8.5.1. Modelo de pré-moldados	89
8.6. Dados do pavimento - TQSBuilding.floorsplan.....	90
8.7. Materiais e o objeto TQSBuilding.materials	99
8.7.1. Materiais pré-moldados TQSBuilding.materials.precast.....	100
8.8. Cobrimentos TQSBuilding.covers.....	101
8.8.1. Cobrimentos ACI-318 TQSBuilding.covers.aci.....	102
8.8.2. Cobrimentos pré-moldados TQSBuilding.covers.precast	103

8.8.3. Cobrimentos por planta TQSBuilding.covers.floorplan	104
8.9. Cargas e o objeto TQSBuilding.loads	106
8.9.1. Cargas de vento TQSBuild.loads.windloads	108
8.9.2. Cargas em etapas construtivas pré-moldadas	112
8.10. Sincronização com o GerPrE	112
9. Módulo TQSGrid	114
9.1. Modo de funcionamento	114
9.2. Atributos de uma célula	115
9.3. Atributos das células com TQSGrid.attrib	116
9.4. Formatação e preenchimento com TQSGrid.Format	117
9.4.1. Propriedades de dimensionamento da tabela	117
9.4.2. Métodos de formatação e preenchimento	117
9.5. Salvamento do grid	117
9.6. Programa TSTGRID.py para teste do TQSGrid	118
10. Execução do TQS com o módulo TQSExec	119
10.1. Modo de funcionamento	119
10.2. Definição de tarefas	119
10.2.1. Pasta atual e a classe TaskFolder	119
10.2.2. Pasta raiz do edifício	120
10.2.3. Processamento global	120
10.2.4. Planta de cargas	121
10.2.5. Desenho de lajes	121
10.2.6. Desenho de lajes treliçadas	121
10.2.7. Desenho de lajes mistas nervuradas	122
10.2.8. Tabela de ferros com todos os desenhos de armação	122
10.2.9. Processamento de escadas	122
10.2.10. Desenho de formas	122
10.2.11. Desenhos de pré-moldados	123
10.2.12. Transferência de esforços para vigas pré-moldadas	123
10.2.13. Transferência de esforços para fundações pré-moldadas	123
10.2.14. Dimensionamento, detalhamento e desenho de pilares pré-moldados	123
10.2.15. Dimensionamento, detalhamento e desenho de consolos pré-moldados	123
10.2.16. Quantitativos de pré-moldados	123
10.2.17. Resumo Estrutural	124
10.3. Execução de tarefas – classe Job	124
10.3.1. Entrada de uma tarefa	124
10.3.2. Execução das tarefas	124
10.4. Programa de teste TSTEXEC.py para a classe TQSExec	124
11. Módulo TQSModel e o Modelo Estrutural	126
11.1. Modelo Estrutural	126
11.1.1. Dados do Edifício e Modelo Estrutural	126
11.1.2. Organização de dados do Modelo Estrutural	127
11.1.3. Criando elementos novos e dados atuais	127

11.2. Objeto TQSModel.Model	128
11.2.1. Leitura e gravação de modelos com o Model.file	128
11.2.2. Lista de tipos de objeto com Model.type	129
11.2.3. Acesso aos pavimentos via Model.floors	130
11.2.4. Dados globais Model.current	131
11.2.5. Modos de visualização Model.visData	132
11.3. Objeto do pavimento: Floor()	132
11.3.1. Dados atuais por pavimento Floor.current	132
11.3.2. Criação de objetos com Floor.create	133
11.3.3. Leitura do pavimento com Floor.iterator	136
11.3.4. Utilitários do pavimento: Floor.util	137
11.4. Identificação de elementos - SMOBJECTIDENT()	137
11.5. Identificação de pré-moldados PreCastGroup()	137
11.6. Objeto Polygon	138
11.7. Carga básica Load()	139
11.8. Superclasse SMOBJECT dos objetos do Modelador	140
11.9. Pilar - Column(SMOBJECT)	141
11.9.1. Dados atuais de pilares	141
11.9.2. Dados de pilares	144
11.9.3. Seção de pilar – ColumnSection()	147
11.9.4. Geometria de pilares – ColumnGeometry()	148
11.9.5. Lista de planta de pilares - ColumnFloorNames()	149
11.9.6. Informações de pilar por seção ou pavimento - ColumnFloorData()	149
11.9.7. Inserção interativa de pilares - ColumnInsertion()	150
11.9.8. Condições de contorno ColumnBoundaryCond()	150
11.9.9. Detalhamento de pilar – ColumnDetailing()	152
11.9.10. Molas de pilar no portico especial - ColumnCoil()	152
11.9.11. Dados de fundações - FoundationData()	154
11.9.12. Dados de pré-moldados - ColumnPrecastData()	157
11.9.13. Pré-moldados: tubo de água pluvial RainWaterPipe()	158
11.9.14. Pré-moldados: alças de içamento LiftingAnchors()	158
11.9.15. Pré-moldados: furos de levantamento LiftOpenning()	159
11.10. Viga - Beam(SMOBJECT)	160
11.10.1. Dados atuais de vigas	161
11.10.2. Dados de vigas	162
11.10.3. Nó de viga - BeamNode()	163
11.10.4. Geometria de viga - BeamGeometry()	164
11.10.5. Inércia de viga - BeamInertia()	165
11.10.6. Vinculações de viga - BeamBond()	166
11.10.7. Inserção interativa de viga - BeamInsertion()	167
11.10.8. Detalhamento de viga - BeamDetailing()	167
11.10.9. Articulação de vigas – BeamRestraint()	167
11.10.10. Segmento de arco de viga - ArcSegment()	168
11.10.11. Mísula e seção variável em viga - VariableSection()	169

11.10.12. Dados de viga pré-moldada - BeamPrecastData()	170
11.10.13. Seção metálica - BeamMetalSection()	170
11.11. Laje - Slab(SMObject)	170
11.11.1. Dados atuais de laje	170
11.11.2. Dados de laje	171
11.11.3. Dados de geometria da laje - SlabGeometry()	172
11.11.4. Discretização de laje por grelha - SlabGrid()	175
11.11.5. Detalhamento de lajes - SlabDetailing()	176
11.11.6. Dados de escada - StairCase()	176
11.11.7. Laje somente de volume - VolumeOnlySlab()	177
11.12. Fechamento de bordo SlabContour(SMObject)	178
11.12.1. Dados de fechamento de bordo	178
11.13. Capitel - DropPanel(SMObject)	178
11.13.1. Dados atuais de um capitel	178
11.13.2. Dados de um capitel	178
11.14. Forma de laje nervurada SlabMould(SMObject)	179
11.14.1. Dados atuais de forma de laje nervurada	179
11.14.2. Dados de forma de laje nervurada	179
11.15. Furo em laje SlabOpening(SMObject)	180
11.15.1. Dados de furo em laje	180
11.16. Furo em pilar - ColumnOpening(SMObject)	180
11.17. Estaca para laje radier SlabFoundationPile (SMObject)	180
11.18. Carga adicional em laje AdditionalLoad(SMObject)	180
11.18.1. Dados atuais de carga adicional	180
11.18.2. Dados de carga adicional	181
11.19. Carga distribuída por área AreaDistributedLoad(SMObject)	181
11.19.1. Dados atuais de carga adicional	181
11.19.2. Dados de carga adicional	181
11.20. Carga concentrada ConcentratedLoad(SMObject)	182
11.20.1. Dados atuais de carga concentrada	182
11.20.2. Dados de carga concentrada	182
11.21. Carga distribuída linearmente LinearyDistributedLoad(SMObject)	183
11.21.1. Dados atuais de carga distribuída linearmente	183
11.21.2. Dados de carga distribuída linearmente	183
11.22. Carga de empuxo SoilPressureLoad(SMObject)	183
11.22.1. Dados atuais de carga de empuxo	184
11.22.2. Dados de carga de empuxo	184
11.23. Temperatura e retração de vigas e lajes - TemperatureShrink()	184
11.24. Modos de visualização - VisModes()	185
11.24.1. Modos de visualização de pilares	186
11.24.2. Modos de visualização de vigas	187
11.24.3. Modos de visualização de lajes	188
11.24.4. Modos de visualização de BIM	188
11.24.5. Modos de visualização de cargas	189

11.24.6. Modos de visualização de fundações	190
11.24.7. Modos de visualização de pré-moldados.....	190
11.24.8. Outros modos de visualização	191
11.25. Atributos BIM de objeto - UserAttrib0.....	192
11.26. Atributos BIM globais - GlobalAttrib0.....	193
11.27. Parede 3D importada do BIM – Wall0.....	194
11.28. Testes do TQSModel.PY	195
11.28.1. Programa TSTMODEL1.PY	195
11.28.2. Programa TSTMODEL2.PY	196
11.28.3. Programa TSTMODEL3.PY	198
11.28.4. Programa TSTMODEL4.PY	199

Error! Use the Home tab to apply Título 1 to the text that you want to appear here. 1

1. Introdução

Os sistemas TQS tiveram origem na década de 1970 e se tornaram ao longo dos anos um complexo sistema de modelagem, análise, dimensionamento, detalhamento e desenho de estruturas de concreto armado, protendido, pré-moldado, alvenarias e paredes estruturais. Em geral, a introdução de novos elementos, sistemas estruturais e critérios de cálculo estiveram a cargo da TQS. A possibilidade de desenvolver ferramentas por parte do engenheiro usuário do sistema permite estendê-lo para atender novas necessidades, calcular novos elementos, e facilitar o detalhamento.

A TQS já possui hoje dois sistemas de programação de desenhos que podem ser usados pelo usuário. Este manual detalha um terceiro, criado a partir da versão 23, que usa a linguagem de programação Python® como ferramenta.

Nos anos 1990 a TQS introduziu o sistema DP de desenho paramétrico. O DP é uma linguagem interpretada orientada para construções geométricas, que permite gravar desenhos em geral, e também gerar objetos de ferros inteligentes

(<http://docs.tqs.com.br/Docs/Details?id=3854&language=pt-BR>)

Nos anos 2000 a TQS criou os componentes tipo OCX, TQSDWG, TQSJAN e TQSGEO, que permitem programar em qualquer linguagem que aceite componentes OCX. O componente TQSDWG permite a geração de desenhos, o TQSJAN mostra estes desenhos em uma janela Windows, e o TQSGEO complementa com uma biblioteca de geometria analítica 2D. A linguagem mais usada na época com estes componentes era o VB, mas também podiam ser usadas linguagens como C# e Delphi (<http://docs.tqs.com.br/Docs/Details?id=3536&language=pt-BR>)

A mais nova interface com TQS foi escrita na linguagem Python, que além das funções criadas nos componentes anteriores estende a programação para outros sistemas como edição gráfica e modelagem. Novos componentes estão sendo desenvolvidos para acessar mais funcionalidades do TQS.

Python é uma linguagem de programação de propósito geral de alto nível, interpretada de script, multiparadigma, imperativa, orientada a objetos, funcional, de tipagem dinâmica forte, lançada por Guido van Rossum em 1991 (Wikipedia <https://pt.wikipedia.org/wiki/Python>). Uma de suas principais características é permitir a fácil leitura do código e exigir poucas linhas se comparada ao mesmo programa em outras linguagens – é um dos motivos de ser tão difundida atualmente.

1.1. Pacote e Módulos

A distribuição é feita através de um pacote instalável TQS. Dentro deste pacote são definidos os seguintes módulos documentados neste manual:

- **TQSDwg:** para a criação e leitura de desenhos DWG TQS. Permite criar desenhos como qualquer programa TQS, e também ler e extrair informações de desenhos. Na programação foram incluídos os objetos de cotação associativa e ferros inteligentes.

- TQSTJan: para mostrar um desenho TQS em uma janela Windows. Um programa qualquer com uma janela Windows pode mostrar um desenho TQS controlado por TQSDwg através de uma janela ligada ao módulo TQSTJan.
- TQSEag: interface para programação dentro dos editores gráficos. Um ou mais comandos em menus podem ser introduzidos em um dos editores gráficos TQS para tratar um desenho. A programação dentro dos editores gráficos se faz com uma combinação do módulo TQSDwg (para tratar o desenho), TQSTJan (para tratar a janela) e TQSEag (iteração com o usuário e estados do editor).
- TQSGeo: biblioteca auxiliar de geometria analítica 2D para usar na programação de desenhos.
- TQSUtil: outros utilitários comuns aos componentes
- TQSGrid: Montagem de tabelas em desenho DWG, HTML e LST.
- TQSExec: Execução automática de tarefas com o gerenciador TQS.
- TQSBuild: Leitura e gravação de dados do edifício. São informações de pisos, cotas, materiais, cobrimentos, cargas e critérios. Permite fazer tudo o que o Editor de Dados do Edifício faz.
- TQSModel: Leitura e gravação de modelos estruturais. Tem acesso a todos os componentes do Modelador, e pode ser usado para criar modelos novos, alterar modelos existentes e extrair informações de modelos. Facilita o teste de alternativas de projeto.

A interface dentro dos módulos é feita através de funções, objetos e propriedades. Devido ao TQS ser um sistema traduzido para mais de uma língua, foi escolhida a língua inglesa para dar nome a todos os componentes da interface. O nome dos componentes assim é independente da língua do sistema.

1.2. Instalação e uso

A programação em Python exige a instalação de um interpretador Python compatível. Você precisa instalar uma versão do Python para Windows, **64 bits**, versão 3.7 ou superior. Um site sugerido para obter o interpretador é o python.org.

O pacote TQS é distribuído através de um arquivo tipo *Wheel* e instalado pelo utilitário PIP, que vem junto com a instalação do Python. Todos os arquivos relacionados à interface Python são distribuídos na pasta TQSW\EXEC\Python e copiados para esta pasta durante a instalação do TQS.

Todos os programas que rodam dentro dos editores gráficos precisam estar na pasta TQSW\EXEC\Python. Outros programas podem rodar em qualquer pasta, usando o módulo TQS.

Error! Use the Home tab to apply Título 1 to the text that you want to appear here. 3

Este manual trata da versão 2.1.0 da interface TQS-Python, distribuída a partir da versão V25 do TQS. A instalação do TQS se possível instala automaticamente o pacote. Se a instalação não for possível, você deve fazer isto, entrando na pasta TQSW\EXEC\Python e rodando o utilitário PIP, como abaixo:

```
pip install TQSPythonInterface-1.2.1-py310-none-any.whl
```

Se o PIP não estiver no PATH da instalação, é possível também acioná-lo através do Python:

```
py -m pip install ....
```

O nome do arquivo *Wheel* varia de versão para versão. Após a instalação, os módulos TQSDwg, TQSJan, TQSEag, TQSGeo, TQSUtil, TQSGrid, TQSExec, TQSBuild e TQSModel ficarão disponíveis para importação dentro do pacote TQS. Os arquivos distribuídos junto com o TQS na pasta TQSW\EXEC\Python tem a seguintes finalidades:

Arquivo	Finalidade
Programacao_TQS_Python.pdf	Este manual
Aquivo .whl	Instalação do pacote TQS para Python
EAGPpy.py	Ponte entre os editores gráficos e os comandos nos menus Python tipo .PYMEN
EAGFOR.py	Teste de programação do Editor Gráfico de Formas EAGFOR
EAGFOR.PYMEN	Menu de edição gráfica que usa o EAGFOR.py
TSTDWG.py	Programa de teste do módulo TQSDwg
TSTFER.py	Programa de teste do módulo TQSDwg com Ferros Inteligentes
TSTJAN.py	Programa de teste do módulo TQSJan
TSTGEO.py	Programa de teste do módulo TQSGeo
TSTUTIL.py	Programa de teste do módulo TQSUtil
TSTGrid.py	Teste do módulo TQSGrid
TSTExec.py	Teste do módulo TQSExec
TSTLISFER.PY	Funções de listagem de linhas de ferro
TSTITERATOR.PY	Teste de múltiplos iteradores de DWG
TSTBUILD.py	Teste do módulo TQSBuild
TSTMODEL1.PY	Teste de geração de modelo estrutural de um pavimento com o TQSModel
TSTMODEL2.PY	Teste de geração de modelo estrutural de 20 pavimentos com o TQSModel

O programa `TSTJAN.PY` que testa o módulo `TQSTJan` faz a montagem de uma janela Windows para teste. Para este teste foi usado o Módulo `wxPython`, que deve ser baixado compatível com a versão Python atual pelo comando:

```
pip install -U wxPython.
```

1.3. Organização dos módulos

Os módulos `TQSGeo` e `TQSUtl` definem apenas funções. Os demais definem objetos e funções e/ou propriedades com o propósito do módulo. Para evitar a poluição de nomes de rotinas e propriedades, as funcionalidades foram divididas em objetos menores (subobjetos). Por exemplo:

Um objeto `dwg` representa um desenho e é criado com o módulo `TQSDwg`:

```
from TQS import TQSDwg
dwg = TQSDwg.Dwg ()
```

Para desenhar uma linha, usamos uma função dentro do subobjeto `draw`:

```
dwg.draw.Line (500, 300, 0, 300)
```

Mas para salvar um desenho, temos o subobjeto `file`:

```
dwg.file.SaveAs ("TESTE")
```

Esta organização é mostrada dentro de cada capítulo.

1.4. Algumas convenções usadas

Trechos e sintaxe de programas, comandos acionados em linha de comando, nomes de pastas no computador – são mostrados com fonte Courier New.

Nomes de constantes, propriedades e funções são mostrados em **negrito**.

Constantes são adotadas em letra maiúscula, como em `TQSUtl.PI`.

Propriedades começam em letra minúscula, como em `dim.idmnil`. A maior parte das propriedades permite leitura e gravação, a menos quando indicado diferente.

Funções começam sempre em letra maiúscula, como em `TQSUtl.SupportFolder()`.

Error! Use the Home tab to apply Título 1 to the text that you want to appear here. 5

2. Módulo TQSUtil

Este módulo tem algumas funções usadas pelos demais módulos. A mais importante é a que permite emitir uma mensagem em uma janela TQS (janela JMSG.EXE). Como certos programas rodam em ambiente Windows, nem sempre a janela de console está disponível para impressão. A função `TQSUtil.writef` abre uma janela TQS se já não estiver aberta, e funciona de maneira semelhante a um `print()`. A função `TQSUtil.DEBW` auxilia a depuração de programas, mostrando o arquivo e linha em que foi chamada.

2.1. Constantes declaradas

`TQSUtil.PI`

Constante Pi = 3.14159...

`TQSUtil.RADGRAUS`

Conversão radiano para graus

`TQSUtil.GRAUSRAD`

Conversão graus para radianos

`TQSUtil.CHARSET`

Nome do jogo de caracteres usados nos programas ("latin-1")

`TQSUtil.MAXNCSTR`

Limite de caracteres dos strings retornados pelas DLLs C++

2.2. Funções

`TQSUtil.writef(*args)`

Imprime em janela TQS independente de console. Recebe um número variável de argumentos. Funcionamento equivalente ao `print()`.

`TQSUtil.DEBW()`

Usando `writef`, imprime o nome do arquivo e linha do programa onde foi colocada. Usada com propósito de depuração de programas.

`TQSUtil.ProgramsFolder()`

Retorna a pasta de programas TQS. Ex: "C:\TQSW\EXEC"

`TQSUtil.SupportFolder()`

Retorna a pasta de arquivos de suporte TQS. Ex: "C:\TQSW\SUPORTE"

`TQSUtil.UserFolder()`

Retorna a pasta de preferências de usuário TQS. Ex: "C:\TQSW\USUARIO"

`TQSUtl.ExecTqs` (command)

Executa um programa e seus parâmetros na pasta `TQSW\EXEC`

2.3. Programa `TSTUtil.py` para teste do `TQSUtl`

Este programa simplesmente mostra o uso da função `TQSUtl.writef` e imprime em uma janela TQS as pastas de programas, suporte e usuário TQS:

```
Teste de mensagem 3.141593
Pasta de programas [C:\TQSWV22\TQSW\EXEC]
Pasta de suporte    [C:\TQSWV22\TQSW\SUPORTE]
Pasta de usuario    [C:\TQSWV22\TQSW\USUARIO]
```


Error! Use the Home tab to apply Título 1 to the text that you want to appear here. 7

3. Módulo TQSGeo

Contém rotinas comuns de geometria analítica 2D comumente usadas na geração de desenhos.

TQSGeo.SetPrecision (delx)

Define a precisão para igualar pontos

TQSGeo.SetAnglePrecision (delang)

Define a precisão para igualar ângulos

TQSGeo.Equals (x1, y1, x2, y2)

Retorna (1) se dois pontos são iguais dentro da precisão atual.

TQSGeo.Distance (x1, y1, x2, y2)

Retorna a distância entre dois pontos

TQSGeo.LineCoefficients (x1, y1, x2, y2)

Retorna os coeficientes a,b,c da reta 1-2.

TQSGeo.DistancePointLine (x1l,y1l,x2l,y2l,xpoint,ypoint)

Retorna a distância entre uma reta e um ponto

TQSGeo.Angle2p (x1, y1, x2, y2)

Retorna o ângulo entre dois pontos entre 0..360 graus

TQSGeo.Angle2r (x1, y1, x2, y2, x3, y3, x4, y4)

Retorna o ângulo entre as retas 1-2 e 3-4 entre 0..180 graus

TQSGeo.NormalizeAngle (angle)

Retorna um ângulo normalizado entre 0 e 360 graus

TQSGeo.Area (vecx, vecy)

Retorna a área dos vetores vecx[] e vecy [] com sinal positivo no sentido horário.

TQSGeo.Perimeter (vecx, vecy)

Retorna o perímetro de vecx[] e vecy []

TQSGeo.GravityCenter (vecx, vecy)

Retorna x,y do centro de gravidade e a área (com sinal) de vecx[] e vecy[]

TQSGeo.PointInSegment (x, y, x1, y1, x2, y2)

Retorna (1) se o ponto x,y, contido na reta 1-2, está contido no segmento de reta 1-2

O ponto x,y é projetado em 1-2 antes do teste

TQSGeo.IntersectionByCoefs(a, b, c, d, e, f)

Retorna x, y, istat da intersecção da reta 1 (coeficientes a,b e c) e a reta 2 (coeficientes d, e, f). istat != 0 se não houve intersecção.

TQSGeo.Intersection2r(x1, y1, x2, y2, x3, y3, x4, y4)

Retorna x, y, istat da intersecção da reta 1 (x1, y1, x2, y2) e a reta 2 (x3, y3, x4, y4). istat != 0 se não houve intersecção.

TQSGeo.IntersectionSegment (x1, y1, x2, y2, x3, y3, x4, y4)

Retorna x, y, istat da intersecção da reta 1 (x1, y1, x2, y2) e a reta 2 (x3, y3, x4, y4). A intersecção tem que acontecer dentro dos segmentos 1-2 e 3-4. istat != 0 se não houve intersecção.

TQSGeo.Projection(x1, y1, x2, y2, xp, yp)

Projeta o ponto xp,yp na reta 1-2. Retorna x, y, o ponto projetado

TQSGeo.ParallelPoint(x1, y1, x2, y2, dist)

Retorna o ponto xp,yp sobre uma reta ortogonal a 1-2 passando por 2 e uma distância positiva à direita de 1-2.

TQSGeo.ParallelLine(x1, y1, x2, y2, dist)

Calcula o a reta paralela a a 1-2 a uma distância positiva à direita de 1-2. Retorna a reta xp1, yp1, xp2, yp2

TQSGeo.ParallelPolyline(vecx, vecy, dist)

Modifica os vetores vecx [] e vecy [] por uma paralela a uma distancia fornecida, positiva à direita das retas

TQSGeo.Limits(vecx, vecy)

Retorna os limites xmin, ymin, xmax, ymax dos vetores vecx [], vecy []

TQSGeo.VectorProduct(x1, y1, x2, y2, x3, y3, x4, y4)

Retorna o produto vetorial do vetor 1-2 por 3-4

TQSGeo.PointProduct(x1, y1, x2, y2, x3, y3, x4, y4)

Retorna o produto pontual do vetor 1-2 por 3-4

TQSGeo.Versor(x1, y1, x2, y2)

Retorna o versor vx,vy do vetor 1-2

TQSGeo.PointInArea (x, y, vecx, vecy)

Verifica se o ponto x,y esta dentro da poligonal vecx [], vecy[].Retorna (1) se estiver.

Error! Use the Home tab to apply Título 1 to the text that you want to appear here. 9

TQSGeo.MirrorInit (x1, y1, x2, y2)

Define a linha de espelhamento 1-2 que será usada pela função **Mirror**. Retorna o ângulo original e o final de espelhamento da reta 1-2 em graus.

TQSGeo.Mirror (xe, ye)

Retorna o ponto *xe, ye* espelhado pela linha definida em **MirrorInit**.

TQSGeo.Rotate (x, y, ang, orgx, orgy)

Retorna o ponto *x, y* rodado por *ang* graus em torno de *orgx, orgy*.

3.1. Programa TSTGeo.py de teste de TQSGeo

O programa TSTGeo.py testa as diversas funções do módulo **TQSGeo**, e imprime os resultados em uma janela TQS.

4. Módulo TQSDwg

Este módulo permite ler e gravar desenhos DWG TQS. Além dos elementos gráficos padrão manipulados, a biblioteca de desenho permite manipular os "objetos inteligentes de desenho". Estes objetos são implementados dentro do desenho através de DLLs externas com chamadas próprias. Dentre estes objetos, está prevista a manipulação de objetos de cotação associativa e de Ferros Inteligentes. Os ferros inteligentes, por terem um grande número de funções e propriedades, serão mostrados no capítulo seguinte.

Nas entradas e saídas do sistema TQS, as unidades de medidas de projeto são tratadas. Entretanto, por convenção, todos os valores definidos internamente nos programas tem alguma unidade fixa convencionada. Todas as chamadas ao módulo TQSDwg, referentes à geometria usam unidades em centímetros e graus.

4.1. Modo de funcionamento

Um objeto TQSDwg.Dwg representa um desenho na memória. Ao criar o objeto, o desenho está vazio:

```
from TQS import TQSDwg
dwg = TQSDwg.Dwg ()
```

A partir daí um desenho pode ser lido do disco para dentro deste objeto, elementos gráficos novos podem ser criados, desenhos misturados, blocos inseridos, o desenho ser salvo, etc. A maioria das operações é feita dentro dos subobjetos, que definem categorias de operação.

Múltiplos desenhos podem ser tratados simultaneamente, cada um contido em um objeto criado por TQSDwg.Dwg.

A criação de elementos gráficos se dá através da definição de atributos de desenho e chamadas de funções do objeto Dwg.

4.2. Operações de arquivos com TQSDwg.file

O objeto tipo TQSDwg contém um subobjeto file da classe TQSDwg.File usado para operações que afetam um desenho como um todo. Por exemplo, para abrir um desenho:

```
from TQS import TQSDwg
dwg = TQSDwg.Dwg ()
nomedwg = "TESTE.DWG"
if dwg.file.Open (nomedwg) != 0:
    print ("Não abri o desenho ", nomedwg)
```

Error! Use the Home tab to apply Título 1 to the text that you want to appear here.11

`file.New()`

Inicializa novo desenho para ser salvo com `SaveAs`. Fecha desenho atual sem salvar, se existir algum.

`file.Open(dwgname)`

Abre e carrega um desenho existente com nome fornecido.

Retorna: (0) se teve sucesso na operação ou (!=0) se não conseguir carregar.

`file.Close()`

Fecha um desenho aberto, não salva o conteúdo.

`file.Save()`

Salva o desenho com nome atual, não fecha o arquivo.

`file.SaveAs(dwgname)`

Salva o desenho atual com um nome fornecido, que pode ser diferente do atual. Se o nome do arquivo não tiver tipo, será salvo com tipo `.DWG`. Outros tipos que podem ser usados são `.DXF` e `.PDF`, para salvar nestes formatos.

No caso do formato PDF, o sistema de plotagem é usado, utilizando os parâmetros de plotagem e tabela de plotagem correspondente. Por isto, é possível que uma assinatura com o nome do cliente seja gerada do lado esquerdo do desenho, e um número de revisão seja somado ao nome do arquivo.

`file.Name()`

Retorna o nome do desenho atual.

`file.PurgeBlocks()`

Elimina todas as definições de blocos não utilizados no desenho.

`file.PurgeLevels()`

Elimina todos os níveis não utilizados no desenho.

`file.LoadColors()`

Carrega a tabela padrão de cores associada ao sistema/subsistema, que devem ser definidos antes (propriedades `systemId` e `subSystemId`). Veja adiante para que servem estas propriedades.

`file.IsModified()`

Retorna (1) se o desenho atual foi modificado desde que foi aberto.

`file.IsModifiedAfterCreation()`

Retorna (1) se o desenho foi modificado por edição gráfica alguma vez após a sua criação.

```
file.IsOutOfProject()
```

Retorna (1) se o desenho foi retirado do projeto.

```
file.IsVerified()
```

Retorna (1) se o desenho foi verificado pelo engenheiro.

4.3. Desenho com TQSDwg.Draw

O objeto do tipo TQSDwg tem o subobjeto draw da classe TQSDwg.Draw com o objetivo de criação de elementos gráficos. Por exemplo, para desenhar uma linha entre dois pontos, fazemos:

```
dwg.draw.Line (100., 100., 200., 200.)
```

As rotinas de desenho não permitem a construção direta de objetos gráficos; em vez disto são usadas funções e propriedades e os objetos gráficos são criados dentro de TQSDwg. Algumas propriedades de objetos gráficos são definidas antes como valores atuais, e usadas automaticamente sempre que um objeto é criado. Por exemplo, nível, estilo e cor. No exemplo, acima, uma linha foi criada nas coordenadas fornecidas, usando nível, estilo e cor atual.

4.3.1. Propriedades de TQSDwg.Draw

Devem ser definidas antes dos elementos gráficos, e se tornam atuais para todos os elementos criados depois.

```
draw.level
```

Nível atual de desenho, sempre na forma numérica

```
draw.style
```

Estilo atual de desenho ou (-1) para usar o estilo associado ao nível

```
draw.color
```

Cor atual de desenho (1..255) ou (-1) para usar a cor associada ao nível de desenho.

4.3.2. Métodos de TQSDwg.Draw

```
draw.Line(x1, y1, x2, y2)
```

Uma linha entre 2 pontos

```
draw.Rectangle (x1, y1, x2, y2)
```

Um retângulo entre 2 pontos – canto esquerdo inferior e direito superior

```
draw.PolyStart()
```

Error! Use the Home tab to apply Título 1 to the text that you want to appear here.13

Inicia a acumulação de pontos por `PolyEnterPoint`. Os pontos acumulados podem ser usados para desenhar linhas poligonais pelas rotinas `Polyline`, `PolylineFilled` e `PolylineCurve`.

`draw.PolyEnterPoint(x, y)`

Acumula um ponto de uma linha poligonal. Este ponto fará parte da poligonal usada pelas rotinas `Polyline`, `PolylineFilled` e `PolylineCurve`.

`draw.PolyEnterXY(xy)`

Acumula um vetor de uma polyline, no formato [[x1,y1], [x2,y2], ...]. Usa a rotina `PolyEnterPoint`. Os pontos acumulados podem ser usados em `Polyline`, `PolylineFilled` e `PolylineCurve`.

`draw.Polyline()`

Desenha linha poligonal com os pontos acumulados por `PolyEnterPoint`

`draw.PolylineFilled()`

Linha poligonal preenchida com os pontos acumulados por `PolyEnterPoint`.

`draw.PolylineCurve()`

Linha poligonal aproximada por spline cúbica com os pontos acumulados por `PolyEnterPoint`.

`draw.Arc(xc, yc, r, angi, angf)`

Arco no sentido anti-horário com centro `xc,yc` raio `r` e ângulos inicial `angi` e final `angf` em graus.

`draw.Circle(xc, yc, r)`

Círculo com centro em `xc,yc` e raio `r`.

`draw.Text(x, y, h, ang, text)`

Têxto com ponto de inserção em `x,y`, altura `h` e ângulo `ang` em graus. O string do texto é definido em `text`.

`draw.BlockOpen(blockname, xbase, ybase)`

Criação de um novo bloco interno. Todos os elementos gráficos criados a seguir entrarão dentro do bloco, até que o bloco seja fechado por `draw.BlockClose`. `blockname` define o nome do bloco e `xbase,ybase` as coordenadas que coincidirão com o ponto de inserção, quando o bloco for inserido no desenho.

`draw.BlockClose()`

Termina a criação de um bloco, aberto em `draw.BlockOpen`.

`draw.BlockInsert(blockname, x, y, escx, escy, ang)`

Inserção de um bloco interno pré-definido de nome `blockname`, com ponto de inserção `x,y`, escala X `escx`, escala Y `escy`, e ângulo `ang` em graus.

`draw.BlockLoadFromDwg (dwgname)`

Carrega um arquivo de desenho e transforma em bloco interno com o mesmo nome do desenho. Retorna (0) se completou a operação ou (1) se houve um erro.

`draw.DwgMix (dwgname, deltax, deltay)`

Carrega e mistura o desenho `dwgname` no desenho com o atual, aplica um deslocamento `deltax, deltay` no desenho carregado. Retorna (0) se completou a operação, ou (1) se houve um erro.

`draw.XrefInsert (dwgname, deltax, deltay, ang)`

Insere uma referência externa com nome `dwgname`, deslocamento `deltax, deltay` e ângulo `ang`. Retorna (0) se completou a operação ou (1) se houve um erro.

4.4. Cotagens com TQSDwg.dim

Cotagens são geradas como objetos de editor gráfico tipo "IPOCOT", tratadas por uma DLL diferente da de desenho. Mas a criação destes objetos foi inteiramente embutida dentro da TQSDwg.

Assim como nos demais elementos gráficos, a criação dos objetos é feita por funções, e certos atributos de cotagem são definidos como "*atuais*", e são implícitos na criação de cotagens novas.

O objeto do tipo TQSDwg tem o subobjeto `dim` da classe TQSDwg.Dim para a criação de cotagens. Por exemplo, para uma cotagem horizontal entre dois pontos e passando por um terceiro, temos uma chamada como esta:

```
dwg.dim.DimHorizontal (0, 0, 500, 0, 0, -50)
```

As cotagens são em cm, mas as alturas de texto e dimensões de símbolos são todos multiplicados pela escala atual de desenho. Em um desenho com escala 1:50 e um texto de cotagem de altura 0.2, a altura real do texto será de $0.2 \times 50 = 10$ cm.

4.4.1. Atributos de cotagem

São os atributos que valem para as cotagens feitas a seguir. Quando não definidos, são assumidos os valores do arquivo de critérios de cotagem.

`dim.dimtxt`

Altura do texto de cotagem.

`dim.dimexe`

Extensão da linha de chamada.

`dim.dimdle`

Error! Use the Home tab to apply Título 1 to the text that you want to appear here.15

Extensão da linha de cotagem.

dim.**dimexo**

Folga na linha de chamada.

dim.**dimtsz**

Tamanho do símbolo de cotagem.

dim.**dimlfc**

Multiplicador de comprimentos.

dim.**idmniv**

Nível geral de cotagem.

dim.**idmnic**

Nível da linha de cotagem.

dim.**idmnil**

Nível da linha de chamada.

dim.**idmnib**

Nível do símbolo de cotagem.

dim.**idmcim**

(1) Se texto abaixo da linha de cotagem

dim.**idmar5**

(1) Se medidas arredondadas de 5 em 5.

dim.**dimblk**

Nome do bloco de cotagem (TICK/DOT/ARROW são pré-definidos).

dim.**dimse1**

(1) Se suprime linha de chamada

dim.**idmcot**

(1) Se suprime linha de cotagem

dim.**dimtxu**

Texto manual de cotagem.

4.4.2. Constantes de cotagem

TQSDwg.**IDHOR**

Identifica cotagem horizontal.

TQSDwg.**IDVER**

Identifica cotagem vertical.

`TQSDwg.IDANG`

Identifica cotagem alinhada.

`TQSDwg.IDINC`

Identifica cotagem inclinada.

4.4.3. Métodos de cotagem

`dim.Dim3P(itipo, x1, y1, x2, y2, x3, y3)`

Cotagem do tipo `itipo=TQSDwg.IDHOR/IDVER/IDANG/IDINC` entre os pontos `x1,y1` e `x2,y2` e com linha de cotagem passando por `x3,y3`.

`dim.DimHorizontal(x1, y1, x2, y2, x3, y3)`

Cotagem horizontal entre os pontos `x1,y1` e `x2,y2` e com linha de cotagem passando por `x3,y3`.

`dim.DimVertical(x1, y1, x2, y2, x3, y3)`

Cotagem vertical entre os pontos `x1,y1` e `x2,y2` e com linha de cotagem passando por `x3,y3`.

`dim.DimAligned(x1, y1, x2, y2, x3, y3)`

Cotagem alinhada com os pontos `x1,y1` e `x2,y2` e com linha de cotagem passando por `x3,y3`.

`dim.DimInclined(x1, y1, x2, y2, x3, y3, ang)`

Cotagem inclinada pelo ângulo `ang` em graus, entre os pontos `x1,y1` e `x2,y2` e com linha de cotagem passando por `x3,y3`.

`dim.DimContinue(x, y)`

Continua a última cotagem com ponto adicional.

`dim.DimRadius(x1, y1, x2, y2)`

Cotagem de raio de arco ou círculo passando por `x1,y1` e `x2,y2`.

`dim.DimDiameter(x1, y1, x2, y2)`

Cotagem de diâmetro de raio ou círculo passando por `x1,y1` e `x2,y2`.

`dim.DimAngular(x1, y1, x2, y2, x3, y3, x4, y4, xcota, ycota)`

Cotagem de ângulo entre as retas `(x1,y1 => x2,y2)` e `(x3,y3 => x4,y4)` passando pelo ponto `xcota,ycota`.

`dim.DimNote()`

Error! Use the Home tab to apply Título 1 to the text that you want to appear here.¹⁷

Desenho da linha com flecha de nota. Use `PolyStart ()` e `PolyEndPoint (x, y)` para a definição dos pontos. Será desenhada uma linha poligonal com uma flecha na ponta.

4.5. Estados de desenho com `TQSDwg.settings`

O objeto do tipo `TQSDwg` tem o subobjeto `settings` da classe `TQSDwg.Settings` com o objetivo de controlar modos de funcionamento do desenho que afetam sua edição gráfica. Tratam-se de propriedades que podem ser lidas ou gravadas. Por exemplo, para definir a escala de desenho 1:50 fazemos:

```
dwg.settings.scale = 50.
```

4.5.1. Constantes que definem o número do sistema

Mostraremos adiante quando usar o número do sistema. Um sistema é uma das aplicações contidas dentro do TQS. As constantes são:

`TQSDwg.IEDOUT`
Genérico

`TQSDwg.IEDFOR`
Formas

`TQSDwg.IEDLAJ`
Lajes

`TQSDwg.IEDFUS`
Fundações - sapatas

`TQSDwg.IEDFUE`
Fundações - blocos

`TQSDwg.IEDFUT`
Fundações - tubulões

`TQSDwg.IEDVIG`
Vigas

`TQSDwg.IEDPIL`
Pilares

`TQSDwg.IEDTAB`
Armação genérica AGC & DP

TQSDwg.**IEDMAD**

Madeira

TQSDwg.**IEDCORB**

G-Bar

TQSDwg.**IEDALV**

Alvenaria Estrutural

TQSDwg.**IEDPOR**

Pórtico TQS

TQSDwg.**IEDGRE**

Grelha TQS

TQSDwg.**IEDPLW**

IGV

TQSDwg.**IEDESC**

Escadas

TQSDwg.**IEDISE**

Interação solo-estrutura

TQSDwg.**IEDPRE**

Pré-moldados

TQSDwg.**IEDRER**

Reservatórios e elementos especiais

TQSDwg.**IEDPAR**

Paredes de concreto

4.5.2. Propriedades de sistema

Todo desenho recebe as propriedades de sistema e subsistema:

settings.**systemId**

Número do sistema

settings.**subSystemId**

Número do subsistema

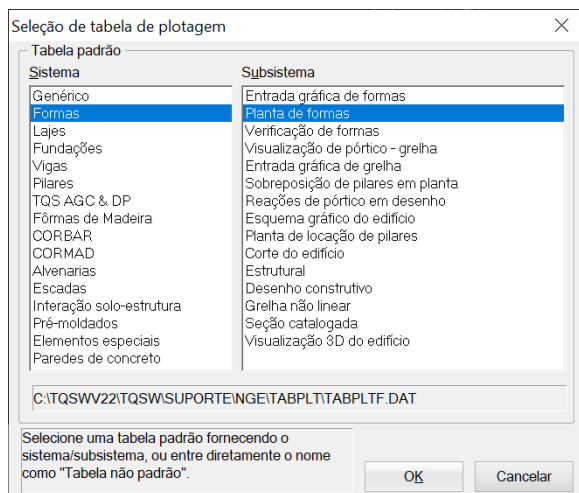
O arquivo TQSW\SUPORTE\NGE\GAPLIC.DAT relaciona o par (sistema, subsistema) com:

- Menu do editor gráfico associado

Error! Use the Home tab to apply Título 1 to the text that you want to appear here.19

- Nome da tabela de cores de níveis
- Desenho com armaduras ou não

Por exemplo, um desenho de planta de formas pertence ao sistema Formas (TQSDwg. **IEDFOR**) e subsistema 2, será associado com o menu de edição gráfica EAGFOR.MEN e tabela de cores DESFOR.COR. Ao dar um duplo-clique sobre um desenho de formas no gerenciador, o editor gráfico lê primeiro o desenho, determina o sistema e subsistema, e chama o menu EAGFOR.MEN associado. O número do subsistema pode ser obtido no gerenciador, menu de plotagem, "Edição de critérios, Tabelas, Tabelas de plotagem". Ao selecionar à esquerda o sistema, vemos os subsistemas à direita:



O número do subsistema é a ordem do item selecionado à direita, começando em 1.

4.5.3. Outras propriedades de estado

`settings.scale`

Retorna valor que divide uma unidade e resulta em centímetros de plotagem

`settings.xBase`

Retorna X do ponto que coincidirá com o cursor se este desenho for inserido dentro de outro como um bloco, referência ou mistura de desenhos.

`settings.yBase`

Retorna Y do ponto que coincidirá com o cursor se este desenho for inserido dentro de outro como um bloco, referência ou mistura de desenhos.

`settings.blockReadDelimiterMode`

(1) Se os delimitadores `DWGTYPE_BLOCK` e `DWGTYPE_BLOCKEND` de início e fim de bloco devem ser lidos

`settings.blockReadElementsMode`

(1) Para ler os elementos dentro do bloco inserido ou (0) apenas para os delimitadores `DWGTYPE_BLOCK` e `DWGTYPE_BLOCKEND`.

`settings.levelsReadMode`

(1) Para ler apenas elementos em níveis ligados ou (0) para todos os elementos

`settings.captureMode`

(0) Captura definida no gerenciador (1) desligado (2) todos ligados (3) ligados somente nos níveis com captura

`settings.gridMode`

(0) Captura (1) Grade (2) desligado

`settings.gridOriginX`

Origem X da grade

`settings.gridOriginY`

Origem Y da grade

`settings.gridSpacingX`

Espaçamento X da grade

`settings.gridSpacingY`

Espaçamento Y da grade

`settings.gridAngle`

Ângulo da grade em graus

`settings.handle_pedmv`

Handle de desenho para uso em outras bibliotecas. É o valor passado diretamente para as DLLs C++ TQS.

`settings.originalColorsMode`

(1) se o desenho deve ser plotado com cores originais e não da tabela de plotagem

`settings.comment`

Comentário associado ao desenho

`settings.revComment`

Comentários da revisão do desenho

`settings.blueprintMode`

Error! Use the Home tab to apply Título 1 to the text that you want to appear here.21

(1) Modo de visualização de plantas

4.6. Limites de desenho com TQSDwg.limits

O objeto do tipo TQSDwg tem o subobjeto `limits` da classe `TQSDwg.Limits` com o objetivo de ler e calcular os limites do desenho atual. Por exemplo, para obter os limites do desenho atual, fazemos:

```
xmin, ymin, xmax, ymax = dwg.limits.DwgLimits()
```

4.6.1. Métodos de TQSDwg.limits

`limits.FileLimits(dwgname)`

Retorna os limites de um desenho sem carregá-lo na memória.

Retorna: `xmin, ymin, xmax, ymax, istat`

`istat == (0)` se a operação foi bem sucedida ou `(1)` se não leu o arquivo

`limits.DwgLimits()`

Le os limites do desenho atual.

Retorna: `xmin, ymin, xmax, ymax`

`limits.UpdateLimits()`

Recalcula os limites do desenho atual.

`limits.WindowLimits()`

Lê a janela inicial do desenho nos editores gráficos.

Retorna: `xmin, ymin, xmax, ymax`.

`limits.DefineWindowLimits(xmin, ymin, xmax, ymax)`

Define a janela inicial de desenho nos editores gráficos.

4.7. Leitura de desenho com TQSDWg.iterator

O objeto do tipo TQSDwg tem o subobjeto `iterator` da classe `TQSDwg.Iterator` com o objetivo de ler e interpretar o conteúdo de um desenho. Por exemplo, uma iteração de leitura por todos os objetos seria como esta:

```
dwg.iterator.Begin()
while True:
    itipo = dwg.iterator.Next()
    if itipo == TQSDwg.DWGTYPE_EOF:
        break
    .....
```

A iteração pelo desenho não retorna objetos, mas valores simples como propriedades da classe `Iterator()`. As funções são usadas para controlar a iteração.

4.7.1. Tipos de elemento de desenho

Todo elemento gráfico tem um tipo, definido por uma das constantes:

`TQSDwg.DWGTYPE_EOF`

Fim de arquivo

`TQSDwg.DWGTYPE_LINE`

Linha

`TQSDwg.DWGTYPE_TEXT`

Texto

`TQSDwg.DWGTYPE_POLYLINE`

Linha poligonal, preenchida ou não.

`TQSDwg.DWGTYPE_BLOCK`

Inserção de bloco

`TQSDwg.DWGTYPE_BLOCKEND`

Fim de bloco

`TQSDwg.DWGTYPE_BLOCKBEGIN`

Início de bloco

`TQSDwg.DWGTYPE_CIRCLE`

Círculo

`TQSDwg.DWGTYPE_ARC`

Arco

`TQSDwg.DWGTYPE_CURVE`

Curva

`TQSDwg.DWGTYPE_OBJECT`

Objeto de desenho (como cotagem associativa ou Ferro Inteligente)

`TQSDwg.DWGTYPE_OBJECTEND`

Fim de leitura de um objeto de desenho

4.7.2. Propriedades de um objeto gráfico lido

As propriedades a seguir ficam disponíveis conforme o tipo de elemento gráfico (TQSDwg.DWGTYPE_xxx), após a chamada da função `iterator.Next()`. Estas propriedades são somente de leitura.

`iterator.iType`

Tipo TQSDwg.DWGTYPE_XXXX do elemento gráfico lido.

`iterator.elementName`

Nome do tipo do elemento gráfico lido.

`iterator.x1`

X1 de um elemento gráfico

`iterator.y1`

Y1 de um elemento gráfico

`iterator.x2`

X2 de um elemento gráfico

`iterator.y2`

Y2 de um elemento gráfico

`iterator.xySize`

Número de pontos de um elemento gráfico

`iterator.GetPolylinePt(ipt)`

Retorna x, y de um elemento gráfico com ipt entre 0 e `iterator.xySize-1`.

`iterator.xy`

Leitura da matriz de pontos inteira no formato xy [[0][1]]

`iterator.isFilled`

(1) se a linha poligonal atual é do tipo preenchida.

`iterator.xc`

X do centro de arco ou círculo.

`iterator.yc`

Y do centro de arco ou círculo.

`iterator.radius`

Raio de um arco ou círculo.

`iterator.startAngle`

Ângulo inicial de um arco em graus.

`iterator.endAngle`

Ângulo final de um arco em graus.

`iterator.textHeight`

Altura de texto

`iterator.textAngle`

Ângulo de texto, em graus

`iterator.textLength`

Número de caracteres do texto.

`iterator.text`

String de texto.

`iterator.blockName`

Nome do bloco inserido.

`iterator.xScale`

Escala X do bloco inserido.

`iterator.yScale`

Escala Y do bloco inserido.

`iterator.insertAngle`

Ângulo do bloco inserido em graus.

`iterator.level`

Nível do elemento

`iterator.color`

Cor (0..255) do elemento, possivelmente herdado do nível.

`iterator.colorRGB`

Cor RGB opcional do elemento

`iterator.style`

Estilo do elemento, possivelmente herdado do nível.

`iterator.levelLock`

(1) se o elemento é travado no nível.

`iterator.captureable`

(1) se o elemento é capturável.

`iterator.hasPlotData`

Error! Use the Home tab to apply Título 1 to the text that you want to appear here.25

(1) se informações de plotagem independentes disponíveis

`iterator.plotPen`

Pena de plotagem se disponível.

`iterator.plotWeight`

Peso de plotagem se disponível.

`iterator.plotStyle`

Estilo de plotagem se disponível.

`iterator.plotFont`

Fonte de plotagem se disponível.

`iterator.plotHatch`

Hachura de plotagem se disponível.

`iterator.objectName`

Nome do objeto de desenho, se objeto inteligente do editor.

`iterator.objectPointer`

Apontador do objeto de desenho, se objeto inteligente do editor.

`iterator.smartRebar`

Objeto SmartRebar se o objeto extraído é um Ferro Inteligente, ou None.

`iterator.inBlock`

(1) Se o elemento gráfico lido está dentro de um bloco.

`iterator.inXref`

(1) se o elemento gráfico lido está dentro de uma referência externa.

`iterator.isOpenObject`

(1) se o elemento gráfico lido está dentro de um objeto inteligente.

4.7.3. Métodos de TQSDwg.iterator

`iterator.Begin()`

Posiciona o desenho para leitura no início.

`iterator.Next()`

Lê o próximo elemento, retorna o tipo `iterator.DWGTYPE_XXXX`, com `DWGTYPE_EOF` no final.

`iterator.GetElementReadPosition()`

Retorna a posição `addr` do último elemento lido

`iterator.SetPosition (addr)`
 Posiciona para leitura na posição `addr`.

`iterator.GetElementWritePosition()`
 Retorna a posição do último elemento gráfico gravado.

`iterator.GetReadPosition()`
 Retorna a posição do próximo elemento a ser lido.

`iterator.GetWritePosition()`
 Retorna a posição do próximo elemento a ser gravado

`iterator.SetBlockPosition(blockname)`
 Posiciona para a leitura dos elementos internos ao bloco `blockname`. Retorna: (0) se teve sucesso ou (1) se o bloco não existe.

`iterator.CopyReadAtribbutes()`
 Copia os atributos do último elemento lido para uso no próximo elemento a ser gravado.

4.8. Tabela de níveis com `TQSDwg.levelstable`

O objeto do tipo `TQSDwg` tem o subobjeto `levelstable` da classe `TQSDwg.LevelsTable` que permite listar e alterar algumas propriedades da tabela de níveis de um desenho. Por exemplo, para atribuir a cor 4 ao nível 3, fazemos:

```
dwg.levelstable.SetColor(3, 4)
```

`levelstable.count`
 Número total de níveis numerados de 0 a N-1.

`levelstable.IsDefined(level)`
 Retorna (1) se o nível está definido

`levelstable.GetStyle(level)`
 Retorna o estilo associado ao nível ou (-1) para estilo padrão

`levelstable.SetStyle(level, ival)`
 Define o estilo associado ao nível ou (-1) para estilo padrão

`levelstable.IsOn(level)`
 Retorna estado ligado (1) ou desligado (0) de um nível

`levelstable.TurnOn(level, ival)`
 Define o estado ligado (1) ou desligado (0) de um nível

Error! Use the Home tab to apply Título 1 to the text that you want to appear here.27

`levelstable.GetColor(level)`

Retorna a cor (0..255) de um nível

`levelstable.SetColor(level, ival)`

Define a cor (0..255) de um nível

`levelstable.GetLock(level)`

Retorna o estado de trava de um nível (1) sim (0) não

`levelstable.SetLock(level, ival)`

Define o estado de trava de um nível (1) sim (0) não

`levelstable.GetCapture(level)`

Retorna o estado de captura de um nível (1) sim (0) não

`levelstable.SetCapture(level, ival)`

Define o estado de captura de um nível (1) sim (0) não

`levelstable.Create(level)`

Cria um nível numérico ou alfanumérico e retorna o correspondente numérico para uso em outros métodos

`levelstable.Name(level)`

Retorna o nível em formato alfanumérico

4.9. Tabela de blocos com TQSDwg.blockstable

O objeto do tipo TQSDwg tem o subobjeto `blockstable` da classe TQSDwg.BlocksTable que permite listar a biblioteca de blocos interna.

`blockstable.count`

Retorna o número de blocos da biblioteca interna

`blockstable.Name(index)`

Retorna o nome do bloco com índice entre 0 e `blockstable.count-1`.

`blockstable.IsDefined(name)`

Retorna (1) se o bloco `name` está definido

4.10. Referências externas e o TQSDwg.xreference

Referências externas são codificadas no DWG como blocos com nome especial, contendo um retângulo envolvente e um texto com o nome do arquivo que representa a referência. A inserção de uma referência externa é feita pela função `draw.XrefInsert`. A listagem e o controle de alguns parâmetros é feito pelo objeto `xreference` da classe XReference.

`xreference.count`

Retorna o número de referências externas definidas e inseridas.

`xreference.Read(index)`

Lê os dados de uma referência externa efetivamente inserida no desenho (`index==0.. xreference.count-1`)

Retorna: `dwgname` (o desenho associado), `ion` (1 se referência ligada) e `blockname` (nome do bloco que representa a referência no DWG).

`xreference.TurnOn(index, ival)`

Defina (`ival==1`) para ligar a visualização de uma referência externa.

`index==0.. xreference.count-1`)

`xreference.Disable(ival)`

Defina (0) para habilitar ou (1) para inibir a interpretação de referências externas no desenho.

4.11. Edição de elementos com TQSDwg.edit

Elementos gráficos podem ser modificados depois de criados. Para isto é necessário de um endereço de objeto gráfico que pode ser obtido com o `TQSDwg.iterator`, ou dentro do editor gráfico, com as rotinas interativas de seleção do editor. Por exemplo,

```
addr = iterator.GetElementReadPosition()
```

O objeto do tipo `TQSDwg` tem o subobjeto `edit` da classe `TQSDwg.Edit` que permite editar elementos gráficos. Nas rotinas a seguir, a variável `addr` corresponde a um endereço mostrado acima.

`edit.ModifyLevel(addr, level)`

Modifica o nível de um elemento.

`edit.ModifyColor(addr, icolor)`

Modifica a cor (0..255) de um elemento.

`edit.ModifyStyle(addr, istyle)`

Modifica o estilo de um elemento.

`edit.Erase(addr)`

Apaga um elemento, retorna (!=0) se erro.

`edit.Recover(addr)`

Recupera um elemento apagado, retorna (!=0) se erro.

Error! Use the Home tab to apply Título 1 to the text that you want to appear here.29

`edit.ModifyPoint(addr, index, x, y)`

Modifica as coordenadas de um ponto de um elemento

`index== 0..iterator. xySize-1`

Retorna (!=0) se erro.

`edit.ModifyText(addr, h, ang)`

Modifica altura `h` e o ângulo `ang` em graus de um texto. Para modificar o seu conteúdo é necessário apagar e criar outro.

`edit.ModifyArc(addr, r, angi, angf)`

Modifica dados de um arco: raio, ângulo inicial e ângulo final, em graus.

`edit.ModifyCircle(addr, r)`

Modifica o raio de um círculo.

`edit.ModifyBlock(addr, escx, escy, ang)`

Modifica dados de um bloco inserido: escala X e Y e ângulo em graus. retorna (!=0) se erro.

`edit.Move(addr, dx, dy)`

Movimenta um elemento qualquer por um deslocamento `dx,dy`. Retorna (!=0) se erro.

4.12. Leitura da tabela de plotagem associada com TQSDwg.plotting

O objeto do tipo TQSDwg tem o subobjeto `plotting` da classe TQSDwg. `Plotting` que permite listar o conteúdo da tabela de plotagem associada a um desenho.

`plotting.LoadPlottingTable()`

Torna disponíveis os dados de tabela de plotagem associados a este desenho.

Retorna: o nome da tabela e o status de leitura (!=0) se erro.

`plotting.AttributeRead(nivel)`

Lê os atributos de um determinado nível da tabela de plotagem

Retorna:

Pena	Índice da pena
peso	Índice do peso
estilo	Índice do estilo
hachura	Índice da hachura
fonte	Índice do fonte
titulo	Título do nível

4.13. Programa TSTDwg.py de teste do TQSDwg

Este programa testa a quase totalidade de recursos do módulo TQSDwg, com exceção dos ferros inteligentes. O programa abre uma janela de mensagens TQS e lista os resultados nesta janela. No final, é gravado o desenho TESTE1.DWG, com o resultado do teste.

O desenho é feito na instância de objeto dwg:

```
Dwg = TQSDwg.Dwg()
```

4.13.1. Função TestarDesenho ()

Ela começa atribuindo sistema TQSDwg.IEDFOR e subsistema 2 (planta de formas), e depois testa a maioria dos atributos e funções deste objeto: desenho de linhas, linhas poligonais, arcos, círculo, curva, poligonal preenchida, criação e inserção de blocos, textos, mistura de desenho e inserção de referência externa.

Tanto o desenho misturado quanto a referência externa são obtidos do desenho \TQSW\USUARIO\TESTE\COBERTURA.DWG, que é distribuído com o TQS. A pasta onde está o desenho é obtida por uma função de TQSUtil.

4.13.2. Função TestarEdit()

A função desenha alguns elementos e obtém seu endereço para alterá-los em seguida, demonstrando a capacidade de edição da biblioteca.

4.13.3. Função TestarCotagem()

Testa diversas propriedades e funções de cotagem.

```
Teste de cotagens
Altura do texto de cotagem      0.31
Extensão de linha de cotagem    0.32
Extensão da linha de chamada    0.33
Folga da linha de cotagem       0.34
....
```

4.13.4. Função TestarLeitura()

Esta função itera na leitura de todos os elementos de desenho, incluindo a mistura de desenhos e a referência externa, e lista os atributos de todos os elementos gráficos. Isto resulta em uma listagem extensa, onde podem ser verificados os atributos de cada objeto gráfico.

```
Elemento [Linha] (tipo 1) em bloco 0 xref 0 obj 0
  nivel 0 cor 7/0000000 estilo 0 ipreench 0 lock 0 captura 0
  plotdata 0 pen 0 peso 0 estilo 0 fonte 0 hatch 0
```


Error! Use the Home tab to apply Título 1 to the text that you want to appear here.31

```
pt1 0,0 pt2 500,0
.....
```

4.13.5. Função TestarGlobais()

Testa a leitura e atribuição de algumas variáveis de estado definidas em TQSDwg.settings.

```
Teste de variaveis globais
Aplicação TQS ..... 2/2
Subaplicação TQS..... 2/2
Escala ..... 50/75
....
```

4.13.6. Função TestarLerNiveis()

Aqui listamos a tabela de níveis de desenho.

```
Tabela de níveis
Nível [      ] Def Estilo Ligado Cor Lock Captura
      0 [      0]   1     0     1    7    0      0
      1 [      1]   1     0     1   15    0      1
....
```

4.13.7. Função TestarLerBlocos()

Listamos todos os blocos inseridos no desenho.

```
Tabela de blocos (4)
Bloco [          X] definido 1
Bloco [      $LAJPRE] definido 1
Bloco [          DOT2] definido 1
Bloco [      $REFE0001] definido 1
```

4.13.8. Função TestarXRef()

Lista as referências externas inseridas. Somente uma neste exemplo.

```
Tabela de referências externas (1)
Referência [C:\TQSWV22\TQSW\USUARIO\TESTE\COBERTURA.DWG] ligada 1
bloco [ $REFE0001]
```

4.13.9. Função TestarLerTabPlt()

Mostra todas os atributos da tabela de plotagem associada a este desenho.

Teste de tabela de plotagem

[C:\TQSWV22\TQSW\SUPORTE\NGE\TABPLT\TABPLTF.DAT]

ível	Pena	Peso	Estilo	Hachura	Fonte	Título
0	0	0	0	0	0	[Uso geral]
1	3	0	0	0	0	[Contorno de vigas]
2	6	0	0	6	0	[Pilares que nascem]

5. Ferros inteligentes e o objeto TQSDwg.SmartRebar

Ferros inteligentes são objetos dentro de um desenho DWG, mas a lógica de criação difere da dos demais elementos gráficos. É uma característica dos chamados "objetos inteligentes de edição gráfica", que são DLLs a parte, com lógica própria.

Nos elementos gráficos comuns, chamamos um método de um objeto `Dwg` para que o elemento gráfico seja criado dentro do desenho. Todos os atributos são assumidos (atributos atuais) ou passados como parâmetro da chamada. Por exemplo,

```
dwg.draw.Line (500, 300, 0, 300)
```

cria uma linha com coordenadas fornecidas, e nível, estilo e cor com valores atuais. Ferros inteligentes por sua vez são criados como um objeto independente e vazio:

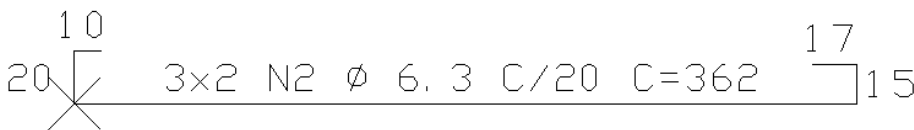
```
rebar = TQSDwg.SmartRebar (dwg)
```

Criamos acima o objeto `rebar`, instância da classe `TQSDwg.SmartRebar`, que é um ferro inteligente sem nenhuma informação. Os métodos e atributos para definir um ferro serão mostrados em seguida. Cada ferro diferente deve ter uma instância diferente da classe `TQSDwg.SmartRebar`.

5.1. Nomenclatura de ferros

Para entender os dados a fornecer, vamos ilustrar a nomenclatura usada. Os tipos de ferros e principais atributos usados são descritos a seguir. Temos quatro tipos de ferros. Os “X” mostrados nas figuras são os pontos de inserção do ferro nas chamadas de criação.

5.1.1. Ferros retos



Consistem em um trecho principal ou “horizontal” e zero, uma ou duas dobras ortogonais à esquerda e/ou direita.

O texto “3x2 N2 Ø 6.3 C/20 C=362” é o “*texto de descrição dos ferros*”. Se as coordenadas do texto de descrição não forem fornecidas, uma será assumida.

“3” é o multiplicador do número de ferros (ex: 3 nervuras com duas barras cada)

“2” é o número de ferros

“N2” é o número de posição. Cada ferro de características diferentes deve ter um número de posição diferente. Ferros diferentes, mas com mesmas características podem ter o mesmo número de posição.

“Ø 6.3” é a bitola em mm do ferro.

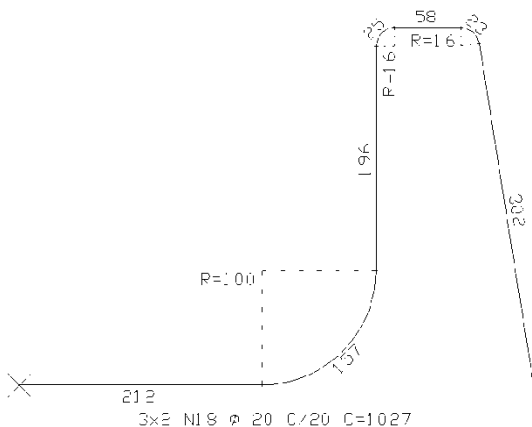
“C/20” é o espaçamento. Se um ferro não é distribuído, o valor do espaçamento deve ser zero.

“C=320” é o comprimento total. Este comprimento, conforme critérios, pode ser afetado por descontos devido a raio de curvatura das dobras. Como este comprimento é mostrado e depois passado para a tabela de ferros é controlado pela biblioteca.

As dobras recebem a “*identificação de dobras*”, que é o seu comprimento. As identificações podem ser mostradas ou não. Ferros retos são definidos por suas faces externas, e as dobras são as medidas por suas faces externas.

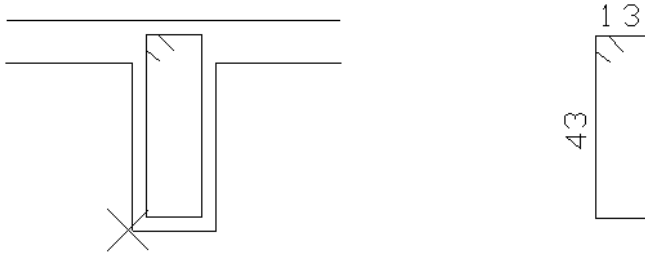
5.1.2. Ferros genéricos

São ferros definidos por uma poligonal qualquer formada por trechos retos e/ou em arco.



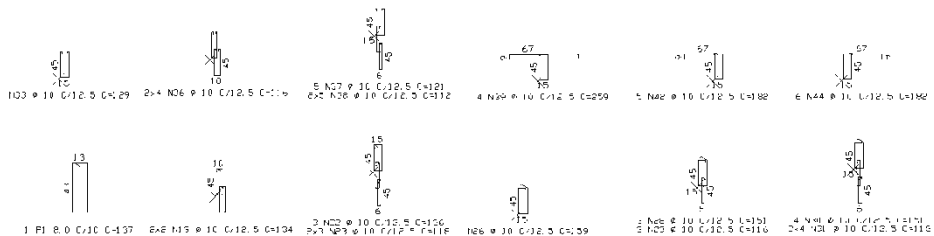
Ferros genéricos são definidos por seu eixo.

5.1.3. Estribos de vigas



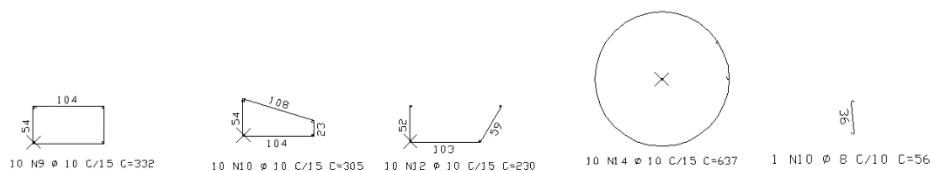
1 P1 8.0 C/10 C=137

Estribos inseridos por um canto da seção transversal. O cobrimento para posicionar o estribo é armazenado junto com o elemento. Estribos podem ser normais, fechados (para torção) ou abertos, e ainda ter a pata superior alongada para servir como negativo na laje. Estes estribos podem combinar com o número de ramos (2, 4 ou 6 ramos) e gerar uma ou duas posições de ferros.



5.1.4. Estribos genéricos (de pilares)

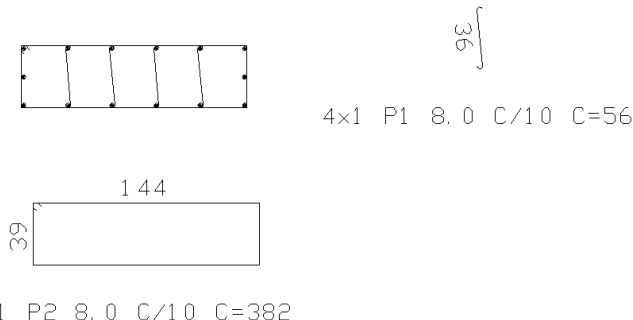
Estribos de pilares têm como referência os pontos de centro das armaduras longitudinais abraçados pelo estribo. Temos estribos fechados, abertos, grampos (por 2 pontos) e estribos circulares:



5.1.5. Inserção de linhas de ferros

Um ferro pode ser representado por várias linhas no desenho, independente da quantidade final ou multiplicador do número de ferros. O Editor de Armação tem um comando para duplicar linhas, que também incrementa o multiplicador de um ferro.

Na figura abaixo, o grampo com posição P1 tem sua linha inserida quatro vezes no desenho:

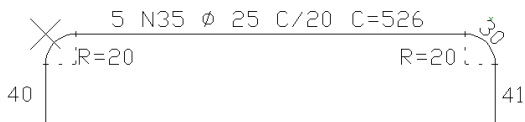


Quando um ferro tem várias linhas no desenho, por padrão a última linha inserida terá o texto de descrição do ferro.

5.1.6. Raio de curvatura

Ferros em geral são dobrados na central de corte e dobra com o uso de pinos definidos pela norma NBR6118:2003 9.4.2.3, mas existem armaduras longitudinais que devem resistir à força cortante ou nó de pórtico (principalmente de vigas) devem receber raio de dobra bem maior, definido pela NBR6118:2003 18.2.2. O tipo de uso do ferro deve ser informado à biblioteca para cálculo correto do raio.

Ferros com estes raios de dobra podem ser representados ou não, conforme critério:



As informações de curvatura são somente atributos, o programa que gera o ferro informa somente trechos retos. A representação é controlada pela biblioteca.

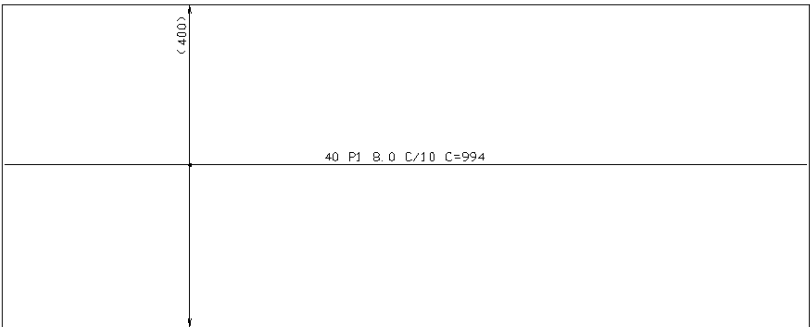
5.1.7. Ganchos

Também a NBR6118:2003 18.2 define comprimentos padrão de ancoragem nas extremidades das barras chamados de “ganchos”. Podem ser especificados ganchos nas pontas de ferros retos ou genéricos, a 180, 135 e 90 graus. Os ganchos são desenhados em escala real com o comprimento de acordo com a norma. O comprimento total é calculado pela biblioteca.

6 N25 Ø 16 C/17.5 C=330

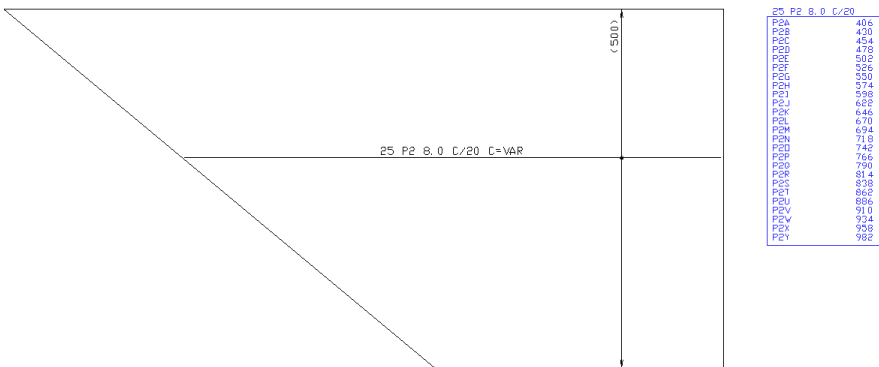
5.1.8. Faixas de distribuição

Ferros distribuídos com espaçamento podem ter a faixa de distribuição indicada. Se a faixa for associada ao ferro, e se o usuário alterar o comprimento da faixa, o número de ferros será alterado junto, automaticamente.



Um ferro pode ter múltiplas faixas de distribuição. O número de ferros será igual à soma dos ferros de todas as faixas. A associação de um ferro com as faixas é opcional.

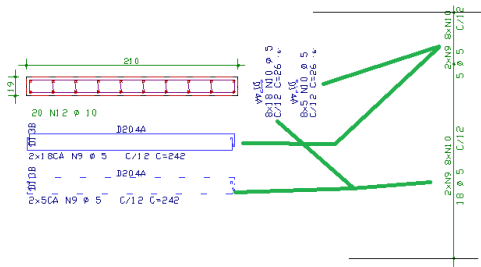
5.1.9. Faixas de distribuição com comprimento variável



Ferros retos ou genéricos podem ser distribuídos com variação de um dos trechos. A biblioteca fornece facilidades para gerar automaticamente as posições de ferro dentro de um contorno ou permite a entrada de cada comprimento de trecho variável. A

tabela de ferros de comprimentos variáveis também é gerada automaticamente, mas deve ser inserida pelo usuário ou programa.

5.1.10. Faixas de distribuição de múltiplas posições

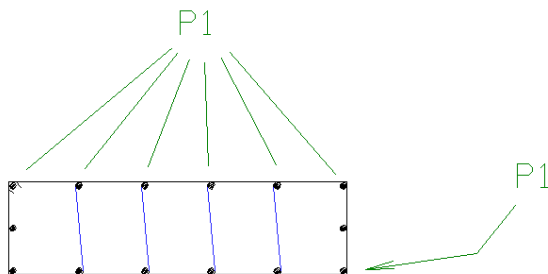


As faixas de distribuição de vigas e lajes são armazenadas junto com os dados do ferro. Isto é, podemos ter um ferro com múltiplas faixas. Faixas de múltiplas posições tem uma relação diferente: vários ferros podem ser associados a uma única faixa. Um exemplo são pilares, onde uma faixa de distribuição abrange diversos estribos e grampos da mesma seção do pilar.

Faixas de múltiplas posições são criadas independentemente dos ferros, e depois associadas a cada um deles.

5.1.11. Identificação de posição

"Identificação de posição" pode ser usada para identificar ferros em corte e outros. Se associadas a um ferro, alteram automaticamente o número da posição em caso de renumeração automática.



Um ferro pode ter várias identificações de posição

5.2. Atributos assumidos e arquivo de critérios

Diversos critérios e tabelas de bitolas são definidas no arquivo de critérios DESARM.DAT, sendo editado no gerenciador, na aba "Edifício", "Critérios gerais", Aço e desenhos de armação". Muitos destes critérios podem ser definidos por ferro no

Error! Use the Home tab to apply Título 1 to the text that you want to appear here.39

programa. Nos critérios não definidos, o programa assume como padrão os valores do arquivo de critérios mais próximo.

5.3. Definição de ferros

Como mostramos, um ferro é criado na memória e depois definido:

```
rebar = TQSDwg.SmartRebar (dwg)
```

Como parâmetro da criação, passamos o objeto TQSDwg do desenho onde o ferro está sendo inserido. Após a chamada, o ferro já está no desenho, faltam agora seus dados. Você pode manter múltiplos ferros abertos e guardar os objetos de cada um para associar atributos posteriormente. Estes objetos são válidos enquanto o desenho representado por dwg estiver aberto.

As constantes que podem ser fornecidas nas chamadas serão mostradas na primeira vez em que aparecerem.

Os métodos e propriedades mostrados a seguir devem ser chamados com o objeto do ferro, e pertencem à classe TQSDwg.SmartRebar.

5.3.1. Dados gerais

A primeira informação a ser fornecida com o ferro é o seu tipo:

type

Tipo do ferro, uma das constantes abaixo:

TQSDwg.**ICPFRT**

Ferro reto.

TQSDwg.**ICPFGN**

Ferro genérico.

TQSDwg.**ICPSTR**

Estribo.

TQSDwg.**ICPGRA**

Grampo de vigas.

TQSDwg.**ICPSTRGEN**

Estribo genérico, pilar.

TQSDwg.**ICPFAIMUL**

Faixa múltipla – a ser associada com outros ferros.

O fornecimento dos demais dados de ferros deve ser compatível com o tipo definido. O programa não verifica se a definição é inconsistente, e o comportamento poderá ser indefinido. A seguir, os outros dados comuns:

mark

Número da posição do ferro

quantity

Número de ferros

multiplier

Multiplicador de ferros

diameter

Bitola do ferro, mm

spacing

Espaçamento de ferros, cm

ribbed

(0) Em geral

(1) Em lajes nervuradas

showRibbed

(0) Não (1) Mostrar status de laje nervurada (C/NERV)

comment

Comentário de um ferro (texto que aparece com a descrição do ferro)

leftHook

Tipo de gancho à esquerda, uma das constantes a seguir:

TQSDwg.**ICPSGA**

Sem gancho

TQSDwg.**ICP090**

Gancho a 90 graus

TQSDwg.**ICP135**

Gancho a 135 graus

TQSDwg.**ICP180**

Gancho a 180 graus

leftHookInvert

(1) Gancho à esquerda invertido

Error! Use the Home tab to apply Título 1 to the text that you want to appear here.41

rightHook

Tipo de gancho definido à direita. Mesmas constantes do gancho à esquerda.

rightHookInvert

(1) Gancho à direita invertido

columnLevel

Número de lance de pilar

cover

Cobrimento, cm. Usado para remontar estribos tipo ICPENR/ICPEFC/ICPEAB/ICPENC.

5.3.2. Linha de ferro

Todo ferro é representado por uma ou mais linhas, cada uma inserida no desenho pela chamada abaixo.

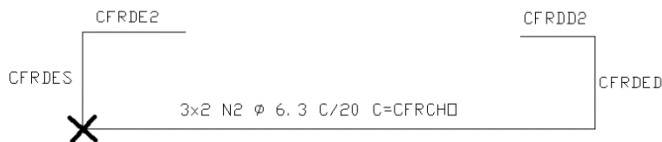
RebarLine(xins,yins,angle,scale,identify,identifyBends,ipatas,iexplode,ilevel,iestilo,icolor)

Entra linha de ferro de qualquer tipo com dados atuais. Um mesmo ferro pode ser representado por mais de uma linha no desenho.

xins	Ponto de inserção
yins	Ponto de inserção
angle	Ângulo de inserção graus
scale	Escala de inserção
identify	(1) Identificar o ferro
identifyBends	(1) Identificar dobras
ipatas	0 não 1 sim 2 45° 3 225° 4 invertido 0 e 1 valem para ICPFRT, ICPSTR, ICPSTRGEN e ICPGRA 2, 3 e 4 valem para ICPFRT
iexplode	(1) Explodir se estribo
ilevel	Nível 0..255 EAG (-1) padrão
iestilo	Estilo 0..5 EAG (-1) padrão
icolor	Cor 0..255 EAG (-1) padrão

5.3.3. Ferros retos

São definidos pelos comprimentos das dobras.



straightBarMainLength

Comprimento principal de ferro reto, cm

straightBarRightLength

Comprimento da dobra esquerda de ferro reto, cm

straightBarRightLength2

Comprimento da 2a dobra direita de ferro reto, cm

straightBarLeftLength

Comprimento da dobra esquerda de ferro reto, cm

straightBarLeftLength2

Comprimento da 2a dobra esquerda de ferro reto, cm

straightBarZone

Posição de ferro reto, definida por uma das constantes:

TQSDwg.ICPPOS

Armadura positiva (face inferior)

TQSDwg.ICPNEG

Armadura negativa (face superior)

straightBarBendType

Tipo de dobra de um ferro reto, definida por uma das constantes:

TQSDwg.ICPNSU

Dobra normal

TQSDwg.ICPDSU

Dobra de suspensão

TQSDwg.ICPDS2

Dobra de suspensão do mesmo lado

straightBarTextDirection

Direção dos textos de dobras:

(0): Direção do trecho

(1): Direção do ferro

straightBarTextPosition

Texto principal de ferros:

(0) Não colocar

(1) Acima da linha de ferro

(2) Abaixo da linha de ferro

straightBarContinuous

Ferro corrido: (0) Não (1) Sim

startCoupler

Luva no início da barra (0) Não (1) Sim

endCoupler

Luva no fim da barra (0) Não (1) Sim

mirrorMode

Espelhamento (0) restrito (1) completo

alternatingMode

Alternância de ferros retos, conforme as constantes:

TQSDwg.ICPSAL

Não

TQSDwg.ICPCAL

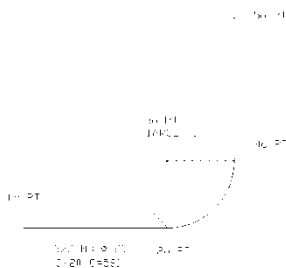
Sim

alternatingFactor

Fator de alternância de ferro reto

5.3.4. Ferros genéricos

As coordenadas dos ferros genéricos (e também estribos genéricos) são fornecidas ponto a ponto no sistema local, com as coordenadas Z atualmente ignoradas. As coordenadas são transformadas na inserção de cada linha de ferro. Para a definição de arcos, 3 pontos são definidos em sequência: o ponto inicial do arco (e final do trecho anterior), o centro (com `iarc==1`) e o ponto final (e início do próximo trecho).



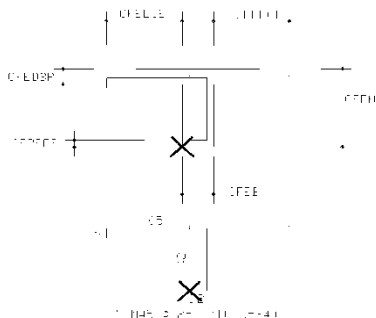
GenRebarPoint (xpt, ypt, zpt, iarc, identifyBends, indfrt)

Entra com um ponto de uma poligonal de ferro genérico. A poligonal será transformada na inserção em **RebarLine**. O Índice de trecho principal será o trecho variável caso o ferro seja distribuído com variação.

xpt	Ponto de inserção
ypt	Ponto de inserção
zpt	Ponto de inserção (ignorado)
iarc	(0) trecho reto (1) centro de arco
identify	(1) Identificar o ferro
identbend	(1) Identificar a dobra
indfrt	Índice de trecho principal (-1)

5.3.5. Estribos de vigas

Estribos de vigas tem referência nas medidas da seção menos o cobrimento. O cobrimento deve ser passado através dos parâmetros gerais. Cada linha deve ser inserida por IPOFER_ RebarLine nas posições mostradas abaixo.



stirrupType

Tipo de estribo de vigas, uma das constantes abaixo:

TQSDwg.**ICPENR**

Normal

TQSDwg.**ICPEFC**

Fechado

TQSDwg.**ICPEAB**

Aberto

TQSDwg.**ICPENC**

Normal com largura colaborante

Error! Use the Home tab to apply Título 1 to the text that you want to appear here.45

stirrupLegs

Número de ramos de estribos, uma das constantes abaixo:

TQSDwg. **ICPNR2**

Estribo de 2 ramos

TQSDwg. **ICPNR4**

Estribo de 4 ramos

TQSDwg. **ICPNR6**

Estribo de 6 ramos

TQSDwg. **ICPNR4B**

Estribo de 4 ramos config B

stirrupSectionWidth

Largura da seção transversal que contém o estribo, cm

stirrupSectionHeight

Altura da seção transversal que contém o estribo, cm

stirrupSectionWidth2

Largura final da seção transversal variável que contém o estribo, cm

stirrupSectionHeight2

Altura final da seção transversal variável que contém o estribo, cm

stirrupEffectiveLeftWidth

Largura esquerda colaborante da seção transversal que contém o estribo, cm

stirrupEffectiveRightWidth

Largura direita colaborante da seção transversal que contém o estribo, cm

stirrupSlabBendLength

Largura direita colaborante da seção transversal que contém o estribo, cm

stirrupHookLength

Comprimento da pata de estribos em número de bitolas

stirrupHookType

Tipo de pata de estribo, uma das constantes a seguir:

TQSDwg. **ICPTPPATA45**

Pata de estribo a 45 graus

TQSDwg.**ICPTPPATA90**

Pata de estribo a 90 graus

stirrupInternalLegDiameter

Bitola em mm diferente para estribo interno de 4 ou 6 ramos

5.3.6. Grampos de vigas

Poderiam ser descritos como ferros retos, mas são representados em perspectiva.



crosstiesDirection

Direção do grampo de vigas (1) esquerda (-1) direita

crosstiesLongLength

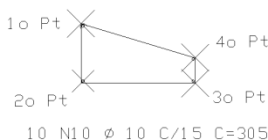
Comprimento longitudinal do grampo de vigas, em cm

crosstiesTransvLength

Comprimento transversal do grampo de vigas, em

5.3.7. Estribos genéricos (de pilares)

Estribos genéricos são definidos tendo como referência o centro da armadura longitudinal que abraçam. Para isto precisam da definição da bitola da armadura longitudinal em `genericStirrupLongDiameter`. O tipo de estribo é definido em `genericStirrupType`. Os pontos podem ser definidos por `GenRebarPoint`, ou pela `GenericStirrupPoint`, mas simples. Em estribos fechados, o ponto de fechamento não deve ser fornecido.



genericStirrupType

Tipo de estribo genérico, conforme uma das constantes:

TQSDwg.**ICPEGENFEC**

Estribo genérico fechado

TQSDwg.**ICPEGENABR**

Estribo genérico aberto

Error! Use the Home tab to apply Título 1 to the text that you want to appear here.47

TQSDwg.**ICPEGENGRA**

Grampo de pilar

TQSDwg.**ICPEGENCIR**

Estribo genérico circular

genericStirrupEntryMode

Modo de definição de estribo genérico, uma das constantes:

TQSDwg.**ICPEGPONTOSLONG**

Pontos longitudinais

TQSDwg.**ICPEGPONTOSSECA**

Pontos da seção longitudinal

TQSDwg.**ICPEGPONTOSEXTR**

Pontos externos

genericStirrupLongDiameter

Bitola longitudinal mm de referência para a definição dos pontos

GenericStirrupPoint(xlong, ylong)

Entrada de um ponto de estribo genérico. Os pontos são fornecidos no centro da armadura longitudinal. A bitola de armadura longitudinal é definida por **genericStirrupLongDiameter**. No estribo fechado, não fornecer o último ponto.

circularStirrupRadius

Raio de estribo circular, cm

StirrupAdditionalCriteria(compradgra, igrasck83, igrak77)

Diversos critérios que modificam um grampo de pilar:

compradgra	Somar valor em cm ao comprimento do grampo de pilar
igrasck83	Desenho de grampo (0) em S ou (1) C
igrak77	(1) Soma duas bitolas ao comprimento de grampo

5.3.8. Posição do texto de identificação

Textos de identificação são posicionados no desenho em posições padrão, não precisando a princípio ser fornecidos. Estas posições podem ser controladas pelas rotinas a seguir. As rotinas são redundantes para simular algumas chamadas da antiga biblioteca ARMLIB.

RebarTextDisplaySelection(iflnfr, iflpos, iflbit, iflesp, iflcmp, ialign, ibreak)

Seleciona os textos de identificação a serem mostrados:

<code>iflnfr</code>	(1) Número de ferros
<code>iflpos</code>	(1) Número da posição
<code>iflbit</code>	(1) Bitola
<code>iflesp</code>	(1) Espaçamento
<code>iflcmp</code>	Comprimento
<code>ialign</code>	Alinhamento – <code>ICPCENTR_xxx</code> abaixo
<code>ibreak</code>	Salto de linha – <code>ICPQUEBR_xxx</code> abaixo

`TQSDwg.ICPCENTR_CENTRAD`

Centrado

`TQSDwg.ICPCENTR_ESQUERD`

Esquerda

`TQSDwg.ICPCENTR_DIREITA`

Direita

`TQSDwg.ICPQUEBR_SEMQUEBRA`

Sem quebra

`TQSDwg.ICPQUEBR_SALTOCBCI`

Salto no espaçamento ou comprimento `C/` ou `C=`

`TQSDwg.ICPQUEBR_SALTOBITO`

Salto na posição da bitola

`TQSDwg.ICPQUEBR_SALTODECD`

Salto no espaçamento `C/`

`TQSDwg.ICPQUEBR_SALTONPOS`

Salto no número de ferros

`RebarTextDisplayPosition(xtex, ytex, htex, angle, imod, ialign, ibreak)`

Fornece a posição e formatação do texto de identificação

<code>xtex</code>	Coordenadas do texto
<code>ytex</code>	Coordenadas do texto
<code>htex</code>	Altura de texto em plotados
<code>angle</code>	Ângulo do texto em graus
<code>imod</code>	(0) normal (1) bitola/espacamento para tabela variável (2) posição (3) posição centrada, levanta o círculo da posição
<code>ialign</code>	Alinhamento – ICPCENTR_xxx acima
<code>ibreak</code>	Salto de linha – ICPQUEBR_xxx acima

A altura do texto dos textos de identificação também pode ser definida diretamente pelas variáveis:

textHeight

Altura do texto de ferros, cm de plotagem

distributionTextHeight

Altura do texto de descrição das faixas de distribuição, cm de plotagem

5.3.9. Linha de chamada para identificação longe do ferro

Textos de identificação afastados da linha de ferro podem ter linha de chamada apontado para o ferro gerada automaticamente. Os dois parâmetros seguintes controlam se a linha pode ser colocada, a partir de qual distância:

leaderLine

- (0) Não coloca linha de chamada
- (1) Será gerada acima da distância definida em **leaderLineDistance**

leaderLineDistance

Distância mínima, em cm de plotagem, do texto identificador para que a linha de chamada seja gerada.

5.3.10. Reposicionando textos de identificação existentes

Textos de identificação são posicionados em posição padrão, mas podem ter posição alterada pela rotina `RebarTextDisplayPosition` ou interativamente por edição gráfica. Para reposicionar textos de identificação nas posições padrão, use a rotina:

ResetTextPosition ()

Reposiciona os textos identificadores do ferro em posição padrão

5.3.11. Raios de curvatura e comprimentos de dobra

O programa permite combinar diversos modos de representação do ferro, mostrando ou não os raios de curvatura, e representando os comprimentos com ou sem o desconto da diferença entre o formato reto e com curvatura. O padrão adotado inicialmente vem do arquivo de critérios de armaduras.

bendType

Tipo de dobra

TQSDwg.ICPTPDOBGAN Gancho de tração NBR61182003 9.4.2.3

TQSDwg.ICPTPDOBNOP Nó de pórtico NBR61182003 18.2.2

bendRadius

Raio de dobra atual em cm ou (0) usar em função da bitola

bendTotalLengthMode

Modo de desconto de raios de dobra do comprimento total

TQSDwg.ICPDEF Conforme o arquivo de critérios

TQSDwg.ICPSRA Faces externas sem desconto

TQSDwg.ICPCRA Comprimento desenvolvido com desconto de raio de dobra

TQSDwg.ICPSMS Soma simples dos trechos

bendRadiusDisplay

Modo de apresentação dos raios de dobra

TQSDwg.ICPDEF Conforme o arquivo de critérios

TQSDwg.ICPMAN Não mostrar

TQSDwg.ICPAUT Mostrar

bendBendLengthMode

Modo representação de desconto das dobras

TQSDwg.ICPDEF Conforme o arquivo de critérios

TQSDwg.ICPDOBSRA Faces externas, sem desconto

TQSDwg.ICPDOBCRA Desenvolvimento com raios de dobras

TQSDwg.ICPDOBSMS Comprimento do trecho

bendDimRadius

Cotagem de raio de curvatura

(-1) Conforme o arquivo de critérios

(0) Não

(1) Sim

bendDimPerimeter

Cotagem de perímetro da curvatura

(-1) Conforme o arquivo de critérios

(0) Não

(1) Sim

Error! Use the Home tab to apply Título 1 to the text that you want to appear here.51

bendTickDisplay

Desenho de tick da cotagem de curvatura

- (-1) Conforme o arquivo de critérios
- (0) Não
- (1) Sim

bendTickSize

Tamanho do tick da cotagem da curvatura em cm de plotagem

5.3.12. Leitura de linha de ferro

Estas rotinas permitem tanto a leitura da linha de ferro desenvolvida em coordenadas absolutas e seus atributos, quanto a linha de um ferro genérico no sistema local, com descrição de arcos.

GetInsertionNumber ()

Retorna o número de linhas inseridas associadas a um ferro

GetInsertionData(indexins)

Retorna dados de uma linha inserida

indexins Índice da linha inserida, 0..GetInsertionNumber()-1

Retorna:

xins	Ponto de inserção
yins	Ponto de inserção
angins	Ângulo de inserção
escxy	Escala de inserção
identfer	(1) Identificar o ferro
identdobr	(1) Identifica dobras
ipatas	(0) não (1) sim (2) a 45°
iexplodir	(1) Se representação explodida
inivel	Nível EAG (-1) default
iestilo	Estilo EAG (-1) default
icor	Cor EAG (-1) default

GetInsertionPoints (indexins)

Retorna o número de pontos de uma inserção

indexins Índice da linha inserida, 0..GetInsertionNumber()-1

Retorna:

np	Número de pontos da linha de ferro
istat	(!=0) se não leu

GetInsertionPoint (indexins, ipt)

Retorna as coordenadas de um ponto de um ferro inserido

indexins Índice da linha inserida, 0..GetInsertionNumber()-1

ipt Ponto a ler 0..np-1

Retorna:

x	Abcissa X
y	Ordenada Y
istat	(!=0) se não leu

GetGenRebarPoints()

Retorna o número de pontos da poligonal de ferro genérico. Retorna zero para todos os outros tipos de ferros.

GetGenRebarPoint(indexpoint)

Retorna um ponto de ferro genérico

indexpoint Índice do ponto a ler, 0..GetGenRebarPoints()-1

Retorna:

xpt	X
ypt	Y
zpt	Z
iarco	(1) se centro de arco
identdobr	(1) p/identificar a dobra
indfrt	Índice de trecho equiv ou (-1)

5.3.13. Alteração da linha de ferro

Para alterar o ponto de inserção de uma linha de ferro, ou os pontos de um ferro genérico:

**SetInsertionData(indexins,xins,yins,angins,escxy,
identfer,identdobr,ipatas,iexplodir,inivel,
iestilo,icor)**

Redefine dados de uma linha inserida

indexins	Índice da linha inserida, 0..GetInsertionNumber()-1
xins	Ponto de inserção
yins	Ponto de inserção
angins	Ângulo de inserção
escxy	Escala de inserção
identfer	(1) Identificar o ferro
identdobr	(1) Identifica dobras
ipatas	(0) não (1) sim (2) a 45°
iexplodir	(1) Se representação explodida
inivel	Nível EAG (-1) default
iestilo	Estilo EAG (-1) default
icor	Cor EAG (-1) default

Error! Use the Home tab to apply Título 1 to the text that you want to appear here.53

SetInsertionPoint(indexins, ipt, x, y)

Redefine somente as coordenadas de um ponto de um ferro inserido

indexins	Índice da linha inserida, 0..GetInsertionNumber()-1
ipt	Ponto a ler 0..np-1
x	Abcissa X
y	Ordenada Y

SetGenRebarPoint(indexpoint, xpt, ypt, zpt, iarco, identdobr, indfrt)

Redefine um ponto de ferro genérico

indexpoint	Índice do ponto a ler, 0..GetGenRebarPoints()-1
xpt	X
ypt	Y
zpt	Z
iarco	(1) se centro de arco
identdobr	(1) p/identificar a dobra
indfrt	Índice de trecho equiv ou (-1)

5.3.14. Outras operações geométricas com ferros

Os comandos seguintes permitem mover, rodar, escalar e espelhar um ferro independente do seu formato.

Move(dx, dy)

Mover um ferro

dx	Deslocamento X cm
dy	Deslocamento Y cm

Rotate(cx, cy, ang)

Rodar um ferro

cx	Centro de rotação X cm
cy	Centro de rotação Y cm
ang	Ângulo de rotação graus antihorário

Scale(cx, cy, escala)

Escalar um ferro

cx	Centro de escala X cm
cy	Centro de escala Y cm
escala	Valor da escala

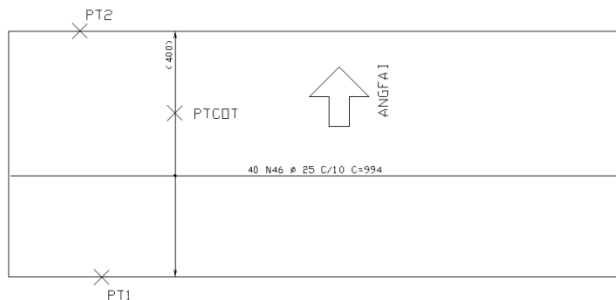
Mirror(x1, y1, x2, y2)

Espelhar um ferro

x1	Linha de espelho PT 1 cm
y1	Linha de espelho PT 1 cm
x2	Linha de espelho PT 2 cm
y2	Linha de espelho PT 2 cm

5.4. Faixas de distribuição

Faixas são associadas a ferros e podem fazer com que o cálculo de quantidade seja automático.



RebarDistrQuant(icfes1,distrLength,spacing)

Cálculo do número de ferros dado comprimento da faixa e espaçamento. Retorna o número de ferros calculado e status (!=0 se espaçamento zero).

icfes1	Número de ferros em função dos espaçamentos, conforme constantes TQSDwg.ICPxxx abaixo.
distrLength	Comprimento da faixa de distribuição, cm
spacing	Espaçamento da faixa de distribuição, cm

TQSDwg.**ICPE1P**

Espaçamentos + 1 ferro

TQSDwg.**ICPE1M**

Espaçamentos - 1 ferro

TQSDwg.**ICPESP**

Espaçamentos = número de ferros

RebarDistrAdd(icfes1, angdist, xpt1, ypt1, xpt2, ypt2, xcot, ycot, ifdcotc, iflnfr, iflpos, iflbit, iflesp, ialign, ibreak, ordem, k32vigas, k41vigas, ilinexten, ilinchama, itpponta, spacing, scale)

Adiciona uma faixa de distribuição ao ferro. Temos dois tipos de faixas de distribuição:

- Embutida em um ferro. Um ferro pode ter várias faixas. Não é usada em pilares. Uma faixa pertence a um ferro.
- Independente do ferro. Uma faixa pode ter vários ferros. Usadas em pilares, onde estribos e grampos partilham faixa única. Uma faixa é definida primeiro como um ferro tipo TQSDwg.ICPFAIMUL. Depois são definidos os dados de faixas (RebarDistrAdd) e depois associados os ferros ligados à faixa através da rotina RebarDistrLink.

Error! Use the Home tab to apply Título 1 to the text that you want to appear here.55

Uma vez que um ferro tem uma ou mais faixas associadas, o número de ferros passa a ser calculado pela soma dos ferros das faixas.

icfes1	Número de ferros em função dos espaçamentos, conforme constantes TQSDwg.ICPxxx
angdist	Ângulo em graus da faixa de distribuição ortogonal ao ferro
xpt1	Ponto 1 da faixa
ypt1	Ponto 1 da faixa
xpt2	Ponto 2 da faixa
ypt2	Ponto 2 da faixa
xcot	X de passagem da linha de cotagem
ycot	Y de passagem da linha de cotagem
ifdcotc	(1) p/cotar comprimento da faixa
iflnfr	(1) descrever número de ferros
iflpos	(1) descrever número da posição
iflbit	(1) descrever bitola
iflesp	(1) descrever espaçamento
ialign	Alinhamento TQSDwg.ICPCENTR xxxxxxxx
ibreak	Salto de linha TQSDwg.ICPQUEBR xxxxxxxxxx
ordem	Ordem dos textos (" padrão) "N" Número de ferros "n" só o número "M" só o multiplicador "P" Posição "B" Bitola "E" Espaçamento "C" Comprimento "F" Comprimento de faixa
k32vigas	Critério K32 CAD/Vigas p/m_ordem=="
ilnexten	(1) linha de extensão automática
ilinchama	(1) se linha de chamada
itpponta	(0) flexa (1) círculo (2) traço
spacing	Espaçamento em se diferente do ferro
scale	Escala de inserção (multiplica todas as dimensões)

5.4.1. Leitura de faixa de distribuição

Os mesmos dados usados na criação de faixas associadas a um ferro podem ser lidos.

GetRebarDistrNum()

Retorna o número de faixas de distribuição

GetRebarDistrInfo(ifaixa)

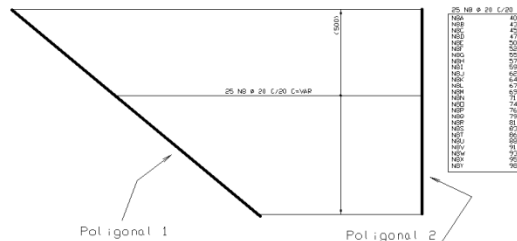
Retorna dados da faixa de distribuição ifaixa == 0..GetRebarDistrNum()-1

Retorna:

icfes1	Número de ferros distribuídos
angfai	Ângulo da faixa de distribuição sistema global
xpt1	Ponto 1 da faixa
ypt1	Ponto 1 da faixa
xpt2	Ponto 2 da faixa
ypt2	Ponto 2 da faixa
xcot	X de passagem da linha de cotagem
ycot	Y de passagem da linha de cotagem
ifdcotc	(1) p/cotar comprimento da faixa
iflnfr	(1) descrever número de ferros
iflpos	(1) descrever número da posição
iflbit	(1) descrever bitola
iflesp	(1) descrever espaçamento
icentr	Alinhamento TQSDwg.ICPCENTR_XXXXXXX
iquebr	Salto de linha TQSDwg.ICPQUEBR_XXXXXXX
ordem	Ordem dos textos (" padrão)
	"N" Número de ferros
	"n" só o número
	"M" só o multiplicador
	"P" Posição
	"B" Bitola
	"E" Espaçamento
	"C" Comprimento
	"F" Comprimento de faixa
k32vigas	Critério K32 CAD/Vigas p/m_ordem=="
	(0) K32=0 (1) K32=1 (2) K32=2
k41vigas	Critério K41 CAD/Vigas p/m_ordem=="
	(0) ã cotar faixa (1) cotar (2) cota+muda Pn
ilnexten	(1) linha de extensão automática
ilinchama	(1) se linha de chamada
itpponta	(0) flexa (1) círculo (2) traço
espac	Espaçamento cm se diferente do ferro
escxy	Escala de inserção (multiplica dimensões)

5.4.2. Faixas de distribuição variável

Ferros variáveis podem ser definidos de maneira completamente manual ou com ajuda da biblioteca de ferros. No modo mais automático, o programa calcula as quantidades e comprimentos da seguinte maneira:



- Defina dados gerais, incluindo espaçamento.
- Defina o cobrimento que será subtraído do contorno por cover.
- Defina as poligonais de contorno como na figura acima. Use `VariableRebarContoursInit`, `VariableRebarOneContourInit`, `VariableRebarOneContourPoint` e `VariableRebarOneContourEnd` para a definição dos contornos.
- Nos ferros genéricos, defina a dobra variável por `VariableRebarMainSegIndex`.
- Chame `RebarDistrAdd` para definir a extensão da faixa de distribuição.
- Chame `VariableRebarGenerate` para calcular cada comprimento variável e obter um comprimento médio e coordenadas para representar a linha do ferro.
- Chame `RebarLine` para inserir uma linha que representa o ferro no desenho. Para desenhar com o comprimento médio obtido no ferro reto, use `straightBarMainLength`.
- Chame `VariableRebarScheduleInsert` se quiser inserir a tabela de comprimentos variáveis no desenho.

VariableRebarContoursInit()

Inicializa lista de contornos para a definição de ferros variáveis.

VariableRebarOneContourInit()

Inicializa um novo contorno atual para ferros variáveis.

VariableRebarOneContourPoint(x, y)

Entra um ponto no contorno atual

VariableRebarOneContourEnd()

Finaliza a entrada de pontos no contorno atual.

VariableRebarMainSegIndex(idobrarvar)

Define o índice da dobra variável (dobras numeradas a partir de zero).

VariableRebarGenerate(icfes1, angdist, scale)

Geração automática de ferros variáveis.

`icfes1` Número de ferros distribuídos: `TQSDwg.ICPE1P/ICPE1M/ICPESP`
`angdist` Ângulo em graus da faixa de distribuição ortogonal ao ferro
`scale` Escala de inserção

Retorna:

<code>xins</code>	Posição X média de inserção do ferro
<code>yins</code>	Posição y inserção ferro médio
<code>meanLength</code>	Compr do ferro médio cm
<code>angle</code>	Ângulo do ferro em graus
<code>istat</code>	(0) Ok
	(1) Contorno inválido
	(2) O número gerado não bate com o número de espaçamentos
	(3) Espaçamento zero

VariableRebarQuantity()

Retorna o número de ferros variáveis gerados.

VariableRebarScheduleInsert(xins,yins)

Insere tabela de ferros variável no desenho.

VariableStirrupGenerate()

Gera a lista de comprimentos variáveis de estribos.

- Somente estribos retangulares (estribos de vigas).
- Somente na largura ou altura
- Variação entre `stirrupSectionWidth` e `stirrupSectionWidth2` ou `stirrupSectionHeight` e `stirrupSectionHeight2`
- A quantidade de ferros é calculada pela faixa ou por número fixo.
- Se não tem faixa definida, `icfes1==TQSDwg.ICPE1P`.

5.5. Faixa de distribuição de múltiplas posições

Faixas para múltiplas posições são faixas independentes, definidas como se fossem um ferro, e depois associadas aos ferros distribuídos na faixa. A criação de uma faixa é como esta:

```
rebar = TQSDwg.SmartRebar (dwg)
rebar.type = TQSDwg.ICPFAIMUL
rebar.RebarDistrAdd (TQSDwg.ICFESP, angdist, xpt1, ypt1,
xpt2, ypt2, xcot, ycot, ifdcotc, iflnfr, iflpos, iflbit,
iflesp, ialign, ibreak, ordem, k32vigas, k41vigas,
ilinxten, ilinchama, itpponta, spacing, scale)
```

Ou seja: cria-se um ferro com tipo `TQSDwg.ICPFAIMUL`, e define-se a geometria da faixa. Em seguida, os ferros são associados à faixa em chamadas como esta:

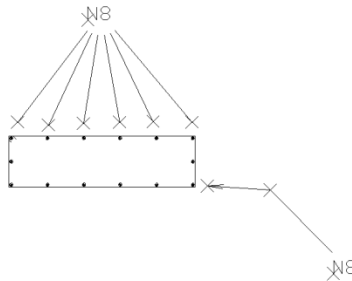
```
rebar2 = TQSDwg.SmartRebar (dwg)
rebar.RebarDistrLink (rebar2)
```

5.6. Identificação de posições

Error! Use the Home tab to apply Título 1 to the text that you want to appear here.59

Identificação de posições são cotagens que identificam linhas ou ferros em corte no desenho associados a uma posição de ferro. Um ferro pode ter uma ou mais identificações associadas. Se o número da posição do ferro se altera, a identificação também se altera de maneira automática.

O mesmo comando gera identificação com dois formatos diferentes: múltiplas linhas definidas pelos pontos da extremidade ou uma única linha com uma flecha na ponta:



A função `RebarMarkIdentify` cria a identificação com zero linhas. Para entrar os pontos das múltiplas linhas ou de uma poligonal, usamos `RebarMarkIdentifyPoint`.

RebarMarkIdentify(`imultiple`,`xtex`,`ytex`,`iflnfr`,`iflpos`,`iflbit`,`iflesp`)

Identificação de uma posição de ferro. Uma ou mais linhas de chamada com um texto de descrição de ferro. Cada chamada a `RebarMarkIdentify` cria uma nova identificação. As chamadas `RebarMarkIdentifyPoint` em seguida se referem à última identificação criada.

```
10 P1          10 P1
+ 2+-->3      + + + +
| /           / | \ \
+---+         / | \ \
0 1           0 1 2 3
(imultiple == 0) (imultiple == 1)
```

<code>imultiple</code>	poligonal e flexa (1) múltiplas linhas
<code>xtex</code>	Posição do texto de identificação
<code>ytex</code>	Posição do texto de identificação
<code>iflnfr</code>	(1) descrever número de ferros
<code>iflpos</code>	(1) descrever número da posição
<code>iflbit</code>	(1) descrever bitola
<code>iflesp</code>	(1) descrever espaçamento

RebarMarkIdentifyPoint(`xid`, `yid`)

Entra novo ponto de linha cotagem. O primeiro está na posição de texto.

5.7. Cotagem de ponta de ferro

É uma cotagem automática colocada em uma ou duas pontas de uma linha de ferro.

DimRebarEndPoint(insindex, idimpoint, xpt, ypt) :

Cotagem de uma ponta de ferro, com o índice (0..n-1) da linha do ferro.

insindex Índice da da linha a cotar (0..)

idimpoint (0) Não (1) cotar 1o ponto (2) cotar pontos 1 e 2

xpt X

ypt Y

5.8. Linhas adicionais de ferro

São linhas arbitrárias associadas a um ferro, são movimentadas e apagadas junto com o ferro. Não precisam ter relação com o ferro, e não tem comprimento alterado com ele.

AdditionalLineInit(inivel, iestilo, icor)

Abre linha adicional de ferro. Os pontos serão definidos por

AdditionalLineInitPoint. Para abrir nova linha, chame novamente esta rotina.

inivel Nível EAG (-1) default

iestilo Estilo EAG (-1) default

icor Cor EAG (-1) default

AdditionalLineInitPoint(xpt, ypt, zpt, iarco)

Entra ponto na linha adicional aberta.

xpt X

ypt Y

zpt Z (atualmente sem uso)

iarco (1) se centro de arco

5.9. Duplicação de um ferro

Você pode duplicar um ferro existente e alterar seus atributos para facilitar a programação.

DeepCopy ()

Retorna uma cópia SmartRebar do objeto atual

5.10. Controle de posições e o objeto TQSDwg. GlobalRebar

Error! Use the Home tab to apply Título 1 to the text that you want to appear here.61

Embora o número da posição de um ferro possa ser atribuído, o mais fácil é usar a renumeração de ferros que atua no desenho inteiro, ordena as posições de cima para baixo e da direita para a esquerda, e agrupa posições iguais.

O objeto do tipo TQSDwg tem o subobjeto globalrebar da classe TQSDwg.GlobalRebar, que trata os ferros do desenho como um conjunto. Por exemplo, o número da primeira posição livre de ferros para ser usada é obtida com:

```
iprimpos = dwg.globalrebar.FreeMark
```

firstMark

Primeira posição para renumeração.

RenumerateMarks ()

Renumeração de posições, a partir da primeira posição definida em firstMark.

FreeMark ()

Retorna próximo número de posição livre para uso

5.11. Iteração com ferros inteligentes

É possível ler o conteúdo de todos os ferros de um desenho, até mesmo gerar uma tabela de ferros. Para isto, itera-se no desenho até encontrar ferros, e lista-se os atributos de cada um.

Para identificar um ferro no desenho, é necessário saber que é um elemento gráfico tipo TQSDwg.DWGTYPE_OBJECT, e que o nome do objeto (também sua DLL) é "IPOFER". A rotina RebarScheduleInfo facilita a leitura dos dados do ferro, somando quantidades e comprimentos de acordo com os critérios de aplicação de raios de curvatura. Uma iteração típica seria como esta:

```
dwg.iterator.Begin ()
while True:
    itipo = dwg.iterator.Next ()
    if itipo == TQSDwg.DWGTYPE_EOF:
        break
    if itipo != TQSDwg.DWGTYPE_OBJECT:
        continue
    if dwg.iterator.objectName != "IPOFER":
        continue
    rebar = dwg.iterator.smartRebar
    numdescr = rebar.RebarScheduleNumDescr ()
    for idescr in range (0, numdescr):
        ipos, bitola, nfer, mult, itpcorbar, ivar, rdval,
        rddsc, compr, igane, igand, observ, ilance, itipo99, icftppata,
        icorrido, iluvai, iluvaf = rebar.RebarScheduleInfo (idescr)
```

A classe `TQSDwg.Iterator` tem uma propriedade que retorna um objeto tipo `SmartRebar` caso o elemento gráfico extraído seja um, ou `None`:

`iterator.smartRebar`

Retorna objeto `SmartRebar` se o objeto extraído é um ferro inteligente, ou `None`

A Função abaixo, de `SmartRebar`, retorna o "número de descrições de um ferro". Este número é necessário apenas para estribos de 4 ou 6 ramos, que agrupam ferros diferentes.

`RebarScheduleNumDescr()`

Retorna o número de descrições de um ferro.

E a função `RebarScheduleInfo`, retorna informações de um ferro usadas para gerar uma tabela de ferros;

`RebarScheduleInfo(idescr)`

`idescr` é o número da descrição de `0..RebarScheduleNumDescr()-1`

Retorna:

<code>ipos</code>	Número da posição
<code>bitola</code>	Bitola mm
<code>nfer</code>	Número de ferros
<code>mult</code>	Multiplicador de ferros
<code>itpcorbar</code>	Tipo de ferro CORBAR
<code>ivar</code>	(1) Se ferro variável
<code>rdval</code>	Raio de dobra
<code>rddsc</code>	Deconto total raio de dobra
<code>compr</code>	Comprimento total sem desconto
<code>igane</code>	Gancho esquerdo tipo <code>TQSDwg.ICPSGA/ICP090/ICP135/ICP180</code>
<code>igand</code>	Gancho direito tipo <code>TQSDwg.ICPSGA/ICP090/ICP135/ICP180</code>
<code>observ</code>	Observação associada
<code>ilance</code>	Lance de pilar
<code>itipo99</code>	(1) Se ferro tipo 99 é estribo (informação para o G-Bar)
<code>icftppata</code>	Pata de estribo <code>ICPTTPATA45/90</code>
<code>icorrido</code>	(1) Ferro corrido
<code>iluvai</code>	(1) Luva inicial
<code>iluvaf</code>	(1) Luva final

`InUse()`

Retorna (`!=0`) se a posição do ferro está em uso por outro ferro.

5.12. Programa TSTFer.py de teste de geração de ferros inteligentes no TQSDwg

Error! Use the Home tab to apply Título 1 to the text that you want to appear here.63

Este programa é uma adaptação de um programa em C++ desenvolvido para testar ao máximo todas as funcionalidades da biblioteca de ferros inteligentes. Assim, é um pouco extenso (e mais difícil de entender), gerando no total mais de 300 ferros. Entretanto, é uma referência para entender como os ferros são gerados.

Ele grava o desenho TESTE2.DWG, usando as seguintes rotinas:

Rotina	Função
DesFerroReto()	Todas as variações de ferros retos
FerroGeneri()	Ferros genéricos
FerroEstrib()	Estribos de vigas
FerroGrampo()	Grampos de vigas
FerroVariavel()	Ferros retos com variação de comprimento
EstribGen()	Estribos de pilares com faixa de distribuição comum
EstribRetVar()	Estribo de viga com variação de seção
FerroCorrido()	Ferros corridos
DesenharEscada()	Desenho simples de escadas para teste interativo no EAGFER
DesenharArco()	Ferros genéricos com trecho em arco
DesenharLinhas()	Linhas arbitrárias para teste interativo de linhas em ferros.
DesenharFerCor()	Identificação de ferros em corte, usando ferros retos como exemplo.
ExtrairTabela()	Exemplo de extração de tabela de ferros.

6. Módulo TQSTJan

O módulo TQSTJan mostra um objeto tipo TQSDwg.Dwg em uma janela Windows, usando um objeto TQSTJan.Window.

A apresentação de janelas Windows em Python em geral depende de uma biblioteca GUI de interface de usuário, e existem várias disponíveis. A ligação de um desenho com uma janela depende do tratamento de alguns eventos. Para não tornar o módulo TQSTJan dependente de uma biblioteca em particular, os eventos devem ser ligados manualmente à biblioteca escolhida. Isto é mostrado no programa TSTJan.py, que lê o desenho TESTE1.DWG gerado pelo programa TSTDwg.py, e o coloca em uma janela da biblioteca wxPython. Esta biblioteca precisa ser baixada para rodar o exemplo.

A criação de um objeto TQSTJan.Window é feito com o uso de dois parâmetros: o *handle* da janela Windows, que é uma variável que caracteriza toda janela no Windows, e um objeto Dwg, criado com o módulo TQSDwg:

```
import wx
from TQS import TQSTJan, TQSDwg
.....
    dwg = TQSDwg.Dwg ()
    dwgname = "TESTE1"
    if dwg.file.Open (dwgname) != 0:
.....
    tqstjan = TQSTJan.Window (hWnd, dwg)
```

Por exemplo, na biblioteca wxPython, uma janela derivada da classe wx.Window pode obter este handle pela chamada:

```
hWnd = self.GetHandle ()
```

6.1. Métodos e propriedades de TQSTJan

dwg

Retorna objeto Dwg() associado à esta janela

DestroyView()

Destroi e desaloca a janela atual

ChangeDwg (dwg)

Define novo desenho associado à janela atual

orthogonal

Modo ortogonal (1) ou normal (0)

Error! Use the Home tab to apply Título 1 to the text that you want to appear here.⁶⁵

SetWindow(self, xmin, ymin, xmax, ymax)

Define a janela atual (canto esquerdo inferior, canto direito superior).

GetWindow()

Retorna xmin, ymin, xmax, ymax da janela atual(canto esquerdo inferior, canto direito superior).

WorldToScreen(x, y)

Converte coordenadas x, y do mundo real para coordenadas de tela, Retorna ix, iy, as coordenadas de tela.

ScreenToWorld(ix, iy)

Converte coordenadas ix, iy de tela para coordenadas do mundo real. Retorna x, y no mundo real.

Regen()

Regera o desenho na janela Windows.

Window2P(x1, y1, x2, y2)

Redefine a janela atual do desenho por 2 pontos.

WindowScale(scale)

Redefine a janela atual afastando por uma escala.

WindowPan(vx, vy)

Movimenta a janela atual por um vetor.

ZoomTotal()

Zoom total do desenho atual.

ZoomOut()

Afasta a visualização por um fator de escala de 0.5.

ZoomPrevious()

Mostra a janela anterior. Retorna istat != 0 se não há mais janelas

6.2. Métodos que interagem com o usuário

Os métodos abaixo esperam que o usuário escolha um ponto na tela e aperte <B1>.

GetPoint()

Retorna x, y, istat de um ponto. istat != 0 se o usuário apertou <Esc>.

GetPointRubberBand(x1, y1)

Retorna x, y, istat de um ponto, ligando linha elástica com x1, y1. istat != 0 se o usuário apertou <Esc>.

Zoom2P(imod, posxy=[0., 0.])

Pede 2 pontos para uma janela entre eles conforme imod:

imod == (0) 2 pontos (1) cursor + segundo ponto

posxy = [x, y] primeiro ponto para imod == 1

ZoomPan(imod, irub, posxy=[0., 0.])

Faz Pan com 1 ou 2 pontos do usuário conforme imod:

imod (0) Lê do usuário (1) usa posxy

irub TQSJan.RUB_xxx, uma das constantes abaixo

posxy [ix, iy] Posição do cursors na janela

TQSJan.RUB_LINEAR

Rubberband linear

TQSJan.RUB_RETANG

Rubberband retangular

TQSJan.RUB_NAO

Sem rubberband

TQSJan.RUB_PANDIN

Caso particular: pan dinamico

6.3. Eventos de TQSJan

OnPaint(hdc)

Regera o desenho na janela Windows em um evento WM_PAINT. Hdc é um "handle para um device contexto", estrutura do Windows.

OnSize(cx, cy)

Chamar quando a janela mudar de tamanho, durante evento WM_SIZE.

"""

OnKey(ikeycode, posxy)

Processa eventos WM_KEYDOWN e captura algumas teclas de atalho típicas dos editores gráficos:

Error! Use the Home tab to apply Título 1 to the text that you want to appear here.⁶⁷

<F8>	Janela por 2 pontos
<Shift><F8>	Janela total
<Ctrl><F8>	Janela anterior
<Alt><F8>	Deslocamento de janela
<F11>	Zoom out
<F10>	Liga/desliga ortogonal

OnMouseMiddle (posxy)

Trata botão do meio do mouse (rolete pressionado).

OnWheel (idelta, posxy)

Processa deslocamentos do rolete do mouse.

6.4. Baixando o módulo wxPython

Este programa foi testado com o módulo wxPython baixado e compatível com o Python 3.8. O comando mais comum para baixar e instalar o wxPython é:

```
pip install -U wxPython
```

6.5. Programa TSTJan.py para teste de TQSJan

Usando o wxPython, o programa tem que ter no mínimo um objeto de aplicação derivado da classe wxApp, e um objeto de janela derivado da classe wx.Frame. A aplicação principal então será um objeto desta classe:

```
class MainApp (wx.App):
    def OnInit(self):
        self.m_frame = SampleFrame ()
        self.m_frame.Show ()
        return True
```

O nosso programa carrega um desenho na memória (o TESTE1.DWG gerado pelo TSTDwg.py), cria o objeto da aplicação e a janela da classe SampleFrame:

```
def main():
    dwg = TQSDwg.Dwg ()
    dwgname = "TESTE1"
    if dwg.file.Open (dwgname) != 0:
        return
    app = MainApp ()
    app.m_frame.m_panel.CreateView (dwg)
    app.MainLoop ()
```

Após criar o objeto da aplicação wxPython, é chamada a criação da janela e o loop principal de mensagens. A classe SampleFrame criada em MainApp gera um

container para um painel (uma janela) e um menu, e define uma rotina para tratar o evento de acionamento do menu:

```
class SampleFrame(wx.Frame):
    def __init__(self):
        super().__init__(parent=None, title='Desenho TQS',
            size=(1024, 1024))
        self.SetMenuBar (DrawMenuBar())
        self.m_panel = DrawingPanel (self)
        self.Bind (wx.EVT_MENU, self.m_panel.OnMenu)
```

A rotina `CreateView` efetivamente cria o objeto de janela TQS e liga este objeto ao desenho carregado na memória:

```
def CreateView (self, dwg):
    self.m_dwg = dwg
    self.m_tqsjan = TQSTJan.Window (self.m_hwnd, self.m_dwg)
```

A classe `DrawingPanel` define as características da janela (tamanho 1024 x 1024) e as rotinas de tratamento de eventos:

```
class DrawingPanel(wx.Window):
    def __init__(self, parent):
        super().__init__(parent, -1)
        self.SetSize(1024, 1024)
        self.SetBackgroundColour(wx.Colour(0, 0, 0))
        self.m_hwnd = self.GetHandle()
        self.Bind (wx.EVT_PAINT, self.OnPaint)
        self.Bind (wx.EVT_KEY_DOWN, self.KeyDown)
        self.Bind (wx.EVT_MOUSEWHEEL, self.OnWheel)
        self.Bind (wx.EVT_MOUSEWHEEL, self.OnWheel)
        self.Bind (wx.EVT_MIDDLE_DOWN, self.OnMouseMiddle)
```

As demais rotinas recebem os eventos declarados e chamam as funções correspondentes em `TQSTJan`.

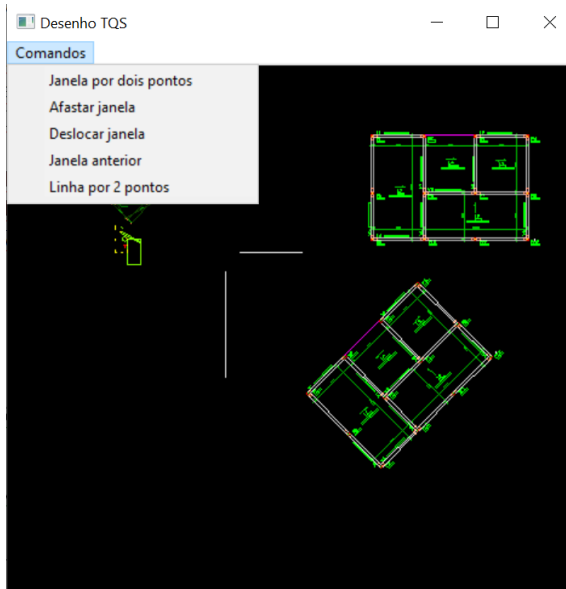
6.5.1. O que faz o programa TSTJan.py

Apesar de pequeno, o programa `TSTJan.py`:

- Permite a visualização do desenho `TESTE1.DWG`.
- Permite operações de janela com teclas de atalho – Janela por 2 pontos (<F8>), Janela total (<Shift><F8>), Janela anterior (<Ctrl><F8>), Deslocamento de janela (<Alt><F8>), Zoom out (<F11>) e controle de modo ortogonal (<F10>).

Error! Use the Home tab to apply Título 1 to the text that you want to appear here.69

- Aceita controle de janela seja por movimento do rolete do mouse (zoom in e out) ou por movimento com o botão do rolete apertado.
- Define um menu *dropdown* com os comandos de janela acionados por menu, mais um comando para desenhar linhas.



7. Módulo TQSEag

O Módulo TQSEag permite o desenvolvimento de programas de dentro dos editores gráficos, com acesso aos desenhos e janelas de edição durante a sessão gráfica. Naturalmente usamos no TQSEag os módulos já mostrados para manipular desenhos (TQSDwg) e janelas (TQSJan). Inicialmente a interface de usuário para chamada de programas está restrita aos menus suspensos (*dropdown menus*).

Embora o Editor de Aplicações Gráficas TQS (EAG) seja um único programa executável, EAGW.EXE, o editor é estendido para diferentes aplicações, com diferentes comandos e menus. Atualmente existem mais de 20 aplicações diferentes de edição. A programação do editor é comandada por menus, e a programação em Python, da mesma maneira, é chamada conforme a aplicação. Mostraremos antes como são chamados os menus.

7.1. Como uma aplicação do EAG é escolhida

Todo desenho TQS tem os atributos de sistema e subsistema definidos. Estes atributos são acessados pelas propriedades de TQSDwg:

`settings.systemId`

Número do sistema

`settings.subSystemId`

Número do subsistema

Quando damos um duplo clique em um desenho no gerenciador, o editor EAG será chamado e um menu de aplicação carregado conforme as propriedades acima. Eis alguns menus disponíveis:

Menu	Sistema TQSDwg.	Sub	Aplicação
EAG.MEN	IEDOUT	1	Desenho qualquer
EAGEDP.MEN	IEDOUT	4	Layout de plantas
EAGFOR.MEN	IEDFOR	2	Planta de formas
EAGGRE.MEN	IEDFOR	5	Edição de desenhos de grelhas
EAGME.MEN	IEDFOR	11	Modelador Estrutural
EAGSCAT.MEN	IEDFOR	14	Seções catalogadas
EAGFER.MEN	IEDLAJ	4	Armaduras de lajes
EAGPIL.MEN	IEDPIL	3	Editor de seções de pilares
EAGTELS.MEN	IEDTAB	5	Planta de telas
EAGTESQ.MEN	IEDTAB	11	Esquema de telas soldadas
EAGMCRUZP.MEN	IEDMAD	1	Edição de formas de madeira
EAGMAD.MEN	IEDMAD	2	Desenho de painéis de lajes
EAGELMS.MEN	IEDMAD	6	Desenho de painéis de vigas

Error! Use the Home tab to apply Título 1 to the text that you want to appear here.71

EAGALV.MEN	IEDALV	1	Alvenaria em planta
EAGELV.MEN	IEDALV	12	Alvenaria em elevação
EAGADG.MEN	IEDALV	3	Diversas funções de alvenaria
EAGFAC.MEN	IEDPRE	5	Fachadas pré-moldadas
EAGPOR.MEN	IEDPOR	1	Visualizador de pórtico/Grelha
EAGPRCA.MEN	IEDPAR	1	Entrada gráfica de Paredes de concreto
EAGELVPC.MEN	IEDPAR	2	Elevação de Parede de concreto
EAGGDG.MEN	IEDPAR	3	Diversas funções de alvenaria
EAGELVPC.MEN	IEDPAR	17	Paredes de concreto

O que é um menu de editor? É um arquivo de script, que define as interfaces gráficas de menus e barras de ferramentas de botões, acionando comandos (Os menus tipo Ribbon tem uma programação à parte). Todos os menus do editor são armazenados na pasta TQSW\EXEC\EAGMENU. Os comandos são funções com assinatura padrão, dentro de DLLs C++. O arquivo .MEN é dividido em seções. A primeira seção declara as DLLs que serão chamadas:

```
[DLL]
EAGME
```

Nesta declaração, temos a seção [DLL] e a declaração da EAGME.DLL. As DLLs ficam obrigatoriamente na pasta TQSW\EXEC. Em seguida, temos a seção [CMD] de comandos, num formato como este:

```
[CMD]
ID_ARQUIVO_SALVAR,EAGME,aplic_arquivosalvar
Salvar desenho na tela em arquivo DWG

ID_ARQUIVO_SALVARCOMO,EAGME,aplic_arquivosalvar
Salvar o desenho atual com um nome a ser fornecido
```

Um comando é formatado por 4 campos em duas linhas: na primeira linha temos um identificador arbitrário que será usado em outros lugares, o nome da DLL, e o nome de uma função C++ dentro da DLL que será chamada se o comando for acionado. Na segunda linha temos uma mensagem de ajuda que aparece em tooltips e na área de status do editor.

Finalmente, os identificadores entram na descrição de menus e barras de ferramentas, dentro da seção [MENU]:

```
[MENU]
SUBMENU, &Arquivo
    MENUITEM, ID_ARQUIVO_SALVARCOMO, Salvar &DWG
    MENUITEM, ID_ARQUIVO_SALVARCOMOIMAGEM, Salvar como ima&gem
    SEPARADOR
    SUBMENU, &Utilidades
        MENUITEM, ID_ARQUIVO_UTILIDADES_PURGEBLK, Limpar blocos
    FIMSUBMENU
FIMSUBMENU
```

O submenu "Arquivo" entrará no final da lista de menus, e os itens restantes serão pendurados abaixo. Os identificadores apontam para os comandos dentro das DLLs, e assim elas são chamadas.

Mostramos em linhas gerais a programação dos editores por DLLs em C++. A programação em Python é semelhante.

7.2. Como uma extensão em Python é carregada

Junto com os módulos para interface Python são distribuídos exemplos. Usaremos o menu `EAGFor.PYMEN` para mostrar a programação do editor.

Após a carga de um menu tipo `.MEN`, o EAG verifica se existem menus Python, e se existirem, os carrega no final da lista de menus. O nome do menu Python é o mesmo do arquivo `.MEN` carregado, mas com tipo `.PYMEN`. A localização do arquivo é a pasta `TQSW\EXEC\Python`.

O arquivo `EAGFor.PYMEN` está na pasta `\TQSW\EXEC\Python`. Isto significa que quando o menu `EAGFOR.MEN` for carregado, o EAG tentará a carga do `EAGFor.PYMEN`. Este arquivo será carregado sempre que um desenho de formas for lido. Para gerar um desenho de formas e chamar este menu, vá no "TQS Formas", "Processar", "Geração de desenhos", "Planta de formas". Depois de gerado, edite o desenho com nome tipo `FORnnnn.DWG`.

7.3. Menu PYMEN comum a todos os editores

A partir da V24, os editores gráficos passaram a carregar o menu `EAG.PYMEN` antes de tentar carregar o menu específico da aplicação. Assim, é possível definir um conjunto de comandos em Python comum a todos os editores gráficos.

7.4. Chamada do programa Python pelo editor

O menu `EAGFor.PYMEN` de exemplo tem estrutura semelhante aos dos outros menus:

Error! Use the Home tab to apply Título 1 to the text that you want to appear here.73

```
[PYTHON]
EAGFOR.PY
```

```
[CMD]
ID_PYTHON_CMD1,EAGFOR.PY,aplic_cmd1
Teste de geração de desenhos
```

```
ID_PYTHON_CMD2,EAGFOR.PY,aplic_cmd2
Acionamento do editor por identificadores de comando
```

```
ID_PYTHON_CMD3,EAGFOR.PY,aplic_cmd3
Teste de interação
```

```
ID_PYTHON_CMD4,EAGFOR.PY,aplic_cmd4
Teste de seleção e movimentação de elementos
```

```
ID_PYTHON_CMD5,EAGFOR.PY,aplic_cmd5
Mostrar os estados do editor
```

```
[MENU]
SUBMENU,Teste Python
    MENUITEM,ID_PYTHON_CMD1,&Geração de desenho
    MENUITEM,ID_PYTHON_CMD2,&Comandos do editor
    SEPARADOR
    MENUITEM,ID_PYTHON_CMD3,&Entrada de pontos
    MENUITEM,ID_PYTHON_CMD4,&Seleção e movimentação
    SEPARADOR
    MENUITEM,ID_PYTHON_CMD5,Estados do edito&r
FIMSUBMENU
```

A diferença é que agora em vez do nome de DLLs, temos a seção [PYTHON], com nomes dos diversos módulos em Python chamados, e nos identificadores, temos o nome de um módulo seguido do nome da função que tratará o comando. Esta função tem uma assinatura padrão que mostraremos em seguida.

7.4.1. Reutilização de menus de editores

Para utilizar um código comum a vários editores sem repetir o conteúdo, separe o código comum, e inclua dentro dos menus desejados com o comando INCLUIR. Se o código comum estiver no arquivo COMUM.PYMEN, use o seguinte comando para que ele seja incluído dentro de outro menu:

```
INCLUIR,COMUM.PYMEN
```

7.4.2. Rotinas Python chamado pelo .pymen

Vemos acima a seção [MENU] com nome "Teste Python", declarando 5 comandos (com dois separadores entre eles). No editor de planta de formas (EAGFOR), o resultado será como este:



A declaração da primeira linha deste menu é:

```
MENUIITEM, ID_PYTHON_CMD1, &Geração de desenho
```

O sinal "&" na frente da letra "G" em "Geração de desenho", simplesmente faz com que o editor sublinhe esta letra, e aceite as teclas <Alt><G> de atalho para acionamento deste menu, quando estiver aberto.

Quando o usuário acionar este comando, entrará em ação o identificador:

```
ID_PYTHON_CMD1
```

Este por sua vez foi declarado na seção [CMD] como:

```
ID_PYTHON_CMD1, EAGFOR.PY, aplic_cmd1
Teste de geração de desenhos
```

Isto fará com que a rotina `aplic_cmd1` seja chamada do módulo Python `EAGFOR.PY`, necessariamente localizado na pasta `\TQSW\EXEC\Python`. Neste arquivo de exemplo, está a rotina:

```
def aplic_cmd1 (eag, tqsjan):
    TSTDWG.Test(tqsjan.dwg)
    tqsjan.ZoomTotal()
```

A assinatura da rotina é comum à todas as chamadas que o editor faz em Python: o primeiro parâmetro é um objeto da classe `TQSEag.Eag`, que permite acesso ao editor gráfico; o segundo parâmetro é da classe `TQSEag.Window` (já mostrado), que permite a manipulação de um desenho em uma janela Windows. Os dois parâmetros são separados, pois o editor gráfico tem uma arquitetura multidocumento e multijanela. A janela e desenho recebidos pela rotina são os ativos no momento da chamada.

Complementado, para manipular o desenho associado à janela, basta obter o objeto da classe `TQSDwg.Dwg` a partir de `tqsjan`:

```
dwg = tqsjan.dwg
```

Assim, o primeiro comando da rotina acionou o teste inteiro `TSTDWG.py`, passando o `dwg` aberto. Veja que este teste pode rodar tanto dentro quanto fora do editor gráfico para gerar desenhos.

Em linhas gerais, dentro do editor gráfico, você pode manipular o desenho editado, lendo e gravando. Você pode controlar a janela gráfica do editor, e chamar programas da interface do editor, `TQSEag` para interagir com o usuário.

7.5. Pré-entrada de dados e o `TQSEag.entry`

O objeto tipo `TQSEag.Eag` tem um subobjeto `entry` da classe `TQSEag.Entry`, que serve para simular a entrada de dados do usuário. A ideia é simular a entrada de coordenadas e teclas por parte do usuário para em seguida acionar comandos disponíveis do editor gráfico.

É importante ressaltar que os comandos usam os dados definidos previamente – então, primeiro alimentamos as respostas esperadas por um comando a ser chamado, depois chamamos o comando.

Por exemplo, para uma definir uma linha entre dois pontos, acionamos o comando de desenho de linhas, entramos com dois pontos e terminamos com `<Enter>`. Isto pode ser simulado como abaixo:

```
eag.entry.Point      (0., 0.)
eag.entry.Point      (200., 0.)
eag.entry.KeyEnter   ()
eag.exec.Command     ("ID_INSERIR_LINHA")
```

As funções que simulam entrada do usuário são:

```
entry.KeyEscape()
Entra tecla de <Escape>
```

```
entry.KeyEnter()
Entra tecla de <Enter>
```

```
entry.KeyFunction(ishift,icontrol,ialt,ifn)
Entra tecla de função Fn, opcionalmente com <Shift>, <Control> e <Alt> acionados
```

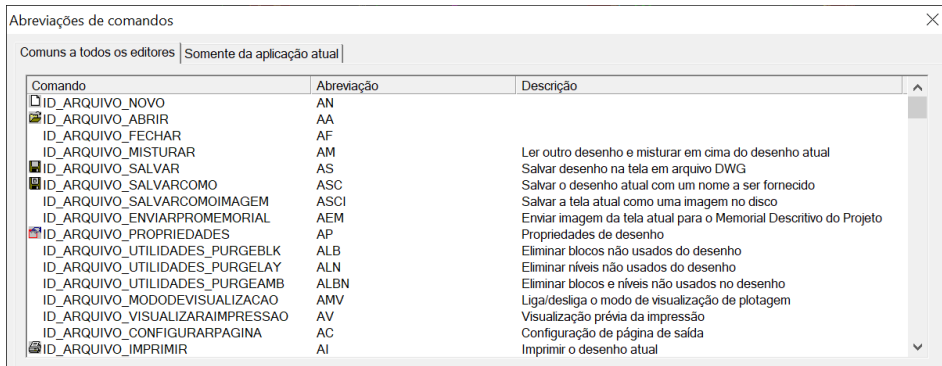
```
entry.String(str)
Entra um texto, que será interpretado conforme o comando
```

```
entry.Point(x, y)
```

Entra um par de coordenadas

7.6. Execução de comandos com TQSEag.exec

O subobjeto `exec` do objeto `TQSEag.Eag` permite executar comandos e controlar a interface deles no editor. Os identificadores de comandos disponíveis podem ser determinados examinando-se os arquivos `.MEN` do TQS, ou simplesmente acionando o comando "Editar, Abreviações".



`exec.Command(ident)`

Executa um comando do editor.

`ident` identificador tipo `ID_xxx` declarado no arquivo `.MEN`

`exec.EnableCommand(ident, ienable)`

Habilita ou desabilita um comando do editor.

`Ident` identificador tipo `ID_xxx` declarado no arquivo `.MEN`

`Ienable` (0) desabilita (1) habilita comando

`exec.CheckCommand(ident, icheck)`

Coloca marca "Check" no comando, quando aplicável

`Ident` identificador tipo `ID_xxx` declarado no arquivo `.MEN`

`Icheck` (0) não (1) marca check

`exec.EditW(name)`

Chama o editor `EDITW.EXE` para edição de um arquivo Ascii fornecido

`Name` Nome do arquivo, com ou sem path

Retorna (0) se editou (1) não

7.7. Mensagens na janela do editor com TQSEag.msg

Error! Use the Home tab to apply Título 1 to the text that you want to appear here.⁷⁷

O subobjeto `msg` da classe `TQSEag.Msg` permite a emissão de mensagens na janela de mensagens do editor e na área de status.

`msg.Print(*args)`

Emite mensagens no estilo do "print" do Python, na janela do editor

`msg.ClearMessageWindow()`

Limpa a janela de mensagens do editor

`msg.PrintStatus(*args)`

Emite mensagens no estilo do "print" do Python, na área de status do editor

`msg.WindowTitle(tqsjan, title)`

Define o título da janela atual de desenho

`Tqsjan` Objeto da classe `TQSJan.Window` passado pelo EAG

`Title` Novo título da janela

7.8. Leitura de pontos e objetos com `TQS.locate`

O subobjeto `locate` da classe `TQS.Locate` tem funções para interação com o usuário – leitura de coordenadas e seleção de elementos.

`locate.GetPoint(tqsjan, msg)`

Lê um ponto do mouse/teclado.

`tqsjan` Objeto da classe `TQSJan.Window` passado pelo EAG

`msg` Mensagem para leitura do ponto

Retorna: `icod, x, y`

`icod` (0)<Enter> (-1)<Esc> (1)<B1> (2)<B2> (3)<B3>
(>20) Teclas de função
(<-1) Código Ascii negativo ("A"=-65)

`locate.GetSecondPoint(tqsjan, x1, y1, irubber, itprubret, msg)`
"""

Lê um segundo ponto com mouse/teclado, fazendo rubberband com `x1, y1`

`tqsjan` Objeto da classe `TQSJan.Window` passado pelo EAG

`x1, y1` Ponto de referência para rubberband

`irubber` Tipo de linha elástica – uma das constantes abaixo.

`Itprubret` Preenchimento de linha elástica retangular – constantes abaixo

`msg` Mensagem para leitura do ponto

Retorna `icod, x, y`

`icod` (0)<Enter> (-1)<Esc> (1)<B1> (2)<B2> (3)<B3>
(>20) Teclas de função
(<-1) Código Ascii negativo ("A"=-65)

Constantes de linha elástica	Finalidade
TQSEag.EAG_RUBNAO	Sem linha elástica
TQSEag.EAG_RUBLINEAR	Linha elástica linear
TQSEag.EAG_RUBRETANG	Linha elástica retangular
TQSEag.EAG_RUBPANDIN	Pan dinâmico

Preenchimento de linha elástica	Finalidade
TQSEag.EAG_RUBRET_NAOPREEN	Não preencher
TQSEag.EAG_RUBRET_JANW	Preenche como uma janela tipo W
TQSEag.EAG_RUBRET_JANC	Preenche como janela tipo C
TQSEag.EAG_RUBRET_JANUSR	Preenche como W (à direita) ou C (à esquerda)

`locate.SetNextZ(z)`

Define coordenadas Z que podem ser usadas pelo próximo comando.

A coordenada é zerada após o próximo `locate.GetPoint`.

`locate.GetLastZ()`

Retorna (idisp, z) o valor da cota Z da última entrada, se disponível

Idisp (1) Se cota Z disponível

z Valor, se disponível

`locate.GetLabel(tqsjan, msg)`

Lê texto do usuário, com mensagem.

tqsjan Objeto da classe TQSJJan.Window passado pelo EAG

msg Mensagem

Retorna str, istat

Str Texto lido

Istat (0) Ok (1)<Enter> (2) <Esc>

""""

`locate.GetValue(tqsjan, msg, valmin, valmax, valdef)`

Leitura de número em ponto flutuante.

tqsjan Objeto da classe TQSJJan.Window passado pelo EAG

msg Mensagem

valmin Valor mínimo

valmax Valor máximo

valdef Valor padrão

Retorna val, istat

Val Valor lido

Istat (0) Ok (1) Não leu

`locate.GetAngle(tqsjan, msg, angdef)`

Leitura de um ângulo em graus. Aceita entrada de <R> 3 pontos e <L> linha.

tqsjan Objeto da classe TQSJJan.Window passado pelo EAG

msg Mensagem

Error! Use the Home tab to apply Título 1 to the text that you want to appear here.79

angdef	Valor padrão
Retorna	val, istat
Ang	Ângulo lido
Istat	(0) Ok (1) Não leu

locate.GetEntryType()

Retorna (0) para última entrada alfanumérica ou (1) gráfica

locate.GetYesNo(tqsjan, msg, r1, r2)

Mostrar mensagem msg e retorna se r1 ou r2 foram escolhidos

tqsjan	Objeto da classe TQSSJan.Window passado pelo EAG
msg	Mensagem
r1,r2	String de resposta, como "S" e "N"
Retorna	istat
Istat	(0)r1 (1)r2 (2)<Escape>

locate.GetCursor(tqsjan)

tqsjan Objeto da classe TQSSJan.Window passado pelo EAG

Retorna coordenadas x, y na posição do cursor no mundo real

locate.SetRubberIdle(tqsjan, irubberidle, xrubbidle, yrubbidle)

Liga ou desliga linha elástica com ponto de referência fornecido.

A linha fica acesa até o próximo comando ou <Esc>

tqsjan	Objeto da classe TQSSJan.Window passado pelo EAG
irubberidle	(0) desligada (1) ligada
xrubbidle	Coordenadas para ativar a linha elástica
yruddidle	Coordenadas para ativar a linha elástica

locate.GetPolyline(tqsjan)

Lê uma poligonal fechada ou aberta por pontos.

tqsjan Objeto da classe TQSSJan.Window passado pelo EAG

Retorna

xy[[2] Matriz de pontos lidos. Não leu se size== 0

locate.Select(tqsjan,msg,itploc)

Faz seleção de elementos conforme itploc. Emite mensagem do usuário. Retorna endereço do 1o elemento e outros dados. Para mais elementos, use BeginSelect/GetSelect.

tqsjan	Objeto da classe TQSSJan.Window passado pelo EAG
msg	Mensagem ao usuário
itploc	Tipo de seleção – uma das constantes abaixo
Retorna:	
addr	Handle do 1o elemento gráfico selecionado (Dwg)
x	Coordenadas de seleção
y	Coordenadas de seleção

np Índice do ponto se poligonal
 istat (!=0) Se não selecionou

Tipo de seleção	Uso
TQSEag.EAG_INORM	Padrão (por ponto, modificável)
TQSEag.EAG_IJANEL	Seleção por janela
TQSEag.EAG_ICURS	Coordenada do cursor
TQSEag.EAG_IMULTP	Seleção múltipla
TQSEag.EAG_IALTJAN	Tenta seleção por ponto, depois janela

locate.**BeginSelection**(tqsjan)

Prepara para a leitura de elementos selecionados em locate.Select.

tqsjan Objeto da classe TQSJJan.Window passado pelo EAG

Retorna (1) se não há elementos

locate.**NextSelection**(tqsjan)

tqsjan Objeto da classe TQSJJan.Window passado pelo EAG

Retorna handle para o próximo elemento gráfico selecionado ou None

locate.**DragSelection**(tqsjan, xref, yref)

Acende elementos os selecionados e os arrasta junto com o cursor

tqsjan Objeto da classe TQSJJan.Window passado pelo EAG

xref, yref é o ponto de inserção inicial da lista

locate.**DragOff**(tqsjan)

Desliga elementos ligados em DragSelection.

tqsjan Objeto da classe TQSJJan.Window passado pelo EAG

7.9. Estados do editor com TQSEag.state

O subobjeto state da classe TQSEag.State permite controlar diversos estados globais do editor que afetam a entrada de dados gráficos.

state.**GetOrtho**(tqsjan)

Retorna o modo ortogonal

tqsjan Objeto da classe TQSJJan.Window passado pelo EAG

Retorna:

Iortho (0) desligado (1) modo ortogonal

Angle Ângulo em graus do modo ortogonal

state.**SetOrtho**(tqsjan, iortho, angle)

Define o modo ortogonal

tqsjan Objeto da classe TQSJJan.Window passado pelo EAG

iortho (0) Desligado (1) Modo ortogonal

angle Ângulo em graus do modo ortogonal

state.GetLevelLock (tqsjan)

Retorna o nível travado

tqsjan Objeto da classe TQSSJan.Window passado pelo EAG

Retorna:

Ilevellock (0) Desligado (1) Nível travado

state.SetLevelLock (tqsjan, ilevellock)

Define o nível travado

tqsjan Objeto da classe TQSSJan.Window passado pelo EAG

Ilevellock (0) Desligado (1) Nível travado

state.ClearAllLevelsLock (tqsjan)

Limpa travas individuais de níveis

tqsjan Objeto da classe TQSSJan.Window passado pelo EAG

state.GetOneLevelLock (tqsjan, ilevel)

Lê a trava de um nível

tqsjan Objeto da classe TQSSJan.Window passado pelo EAG

ilevel Nível considerado

Retorna:

Ilock (1) Se este nível está travado

state.SetOneLevelLock (tqsjan, ilevel, ilock)

Define a trava de um nível

tqsjan Objeto da classe TQSSJan.Window passado pelo EAG

ilevel Nível considerado

Ilock (1) Se este nível está travado

state.GetActiveLevel (tqsjan)

Retorna o nível atual

tqsjan Objeto da classe TQSSJan.Window passado pelo EAG

Retorna:

iaactivelevel (0) Desligado (1) Nível travado

state.SetActiveLevel (tqsjan, iactivelevel)

Define o nível atual

tqsjan Objeto da classe TQSSJan.Window passado pelo EAG

iaactivelevel (0) Desligado (1) Nível travado

state.GetFastCurve (tqsjan)

Retorna (1) se curva rápida ligada

tqsjan Objeto da classe TQSSJan.Window passado pelo EAG

Retorna:

ifastcurve (0) modo normal (1) curva rápida (curvas mostradas como linhas)

`state.SetFastCurve(tqsjan, ifastcurve)`

Define (1) se curva rápida ligada

`tqsjan` Objeto da classe `TQSJan.Window` passado pelo EAG

`ifastcurve` (0) modo normal (1) curva rápida (curvas mostradas como linhas)

`state.GetGrid(tqsjan)`

Retorna dados de grade

`tqsjan` Objeto da classe `TQSJan.Window` passado pelo EAG

Retorna:

`igrade` (0) Não (1) Grade ligada

`grdorx` X origem da grade cm

`grdory` Y origem da grade cm

`grdesx` X espaçamento da grade cm

`grdesy` Y espaçamento da grade cm

`grdang` Ângulo da grade, em graus

`iespacponto` Número de espaçamentos por ponto da grade

`igraderubber` (1) Se grade em linha elástica

`state.SetGrid(tqsjan, igrade, grdorx, grdory, grdesx, grdesy, grdang, iespacponto, igraderubber)`

Define dados da grade

`tqsjan` Objeto da classe `TQSJan.Window` passado pelo EAG

`igrade` (0) Não (1) Grade ligada

`grdorx` X origem da grade cm

`grdory` Y origem da grade cm

`grdesx` X espaçamento da grade cm

`grdesy` Y espaçamento da grade cm

`grdang` Ângulo da grade, em graus

`iespacponto` Número de espaçamentos por ponto da grade

`igraderubber` (1) Se grade em linha elástica

`state.InvertCapture(tqsjan)`

Inverte a captura de coordenadas (ligada ou desligada).

`tqsjan` Objeto da classe `TQSJan.Window` passado pelo EAG

`state.GetFatText(tqsjan)`

Retorna o modo de texto rápido (1) ligado

`tqsjan` Objeto da classe `TQSJan.Window` passado pelo EAG

`state.SetFatText(tqsjan, ifast)`

Define o modo de texto rápido

`tqsjan` Objeto da classe `TQSJan.Window` passado pelo EAG

`ifast` (1) se modo de texto rápido

Error! Use the Home tab to apply Título 1 to the text that you want to appear here.83

state.GetBlockInsertion(tqsjan)

Retorna dados para inserção de um bloco novo

tqsjan Objeto da classe TQSSJan.Window passado pelo EAG

Retorna:

xscale Escala X

yscale Escala Y

angle Ângulo de inserção em graus

state.SetBlockInsertion(tqsjan,xscale,yscale,angle)

Define dados para inserção de um bloco novo

tqsjan Objeto da classe TQSSJan.Window passado pelo EAG

xscale Escala X

yscale Escala Y

angle Ângulo de inserção em graus

state.GetParallelDist(tqsjan)

Retorna distância padrão no comando "Paralela a elemento".

tqsjan Objeto da classe TQSSJan.Window passado pelo EAG

state.SetParallelDist(tqsjan,distance)

Define distância padrão no comando "Paralela a elemento".

tqsjan Objeto da classe TQSSJan.Window passado pelo EAG

state.GetFilletRadius(tqsjan)

Retorna a distância de arredondamento padrão de arcos.

tqsjan Objeto da classe TQSSJan.Window passado pelo EAG

state.SetFilletRadius(tqsjan,radius)

Define a distância de arredondamento padrão de arcos

tqsjan Objeto da classe TQSSJan.Window passado pelo EAG

state.GetFilletlDist(tqsjan)

Retorna as distâncias padrão de chanfragem

tqsjan Objeto da classe TQSSJan.Window passado pelo EAG

Retorna:

dist1, dist2 Distâncias de chanfragem

state.SetFilletlDist(tqsjan,dist1,dist2)

Define as distâncias padrão de chanfragem

tqsjan Objeto da classe TQSSJan.Window passado pelo EAG

dist1, dist2 Distâncias de chanfragem

7.10. Menus de demonstração EAGFOR.PYMEN e programa EAGFor.py.

Como já mostramos, o menu `EAGFor.PYMEN` e o programa `EAGFor.py` ligado a ele devem ficar na pasta `TQSW\EXEC\Python`, ou na pasta correspondente conforme a instalação TQS. Já falamos sobre o princípio de funcionamento de um menu Python para os editores EAG. Vamos descrever apenas os cinco comandos neste menu de exemplo.

7.10.1. Comando "Geração de desenho"

Este comando mostra a facilidade de gerar desenhos usando o objeto de desenho `TQSDwg.Dwg`, que pode ser extraído do `TQSJan.Window`. Neste exemplo, o exemplo inteiro definido no módulo `TQSDwg` é rodado, passando o objeto de desenho aberto. Uma longa listagem de extração de elementos gráficos é emitida por este comando, que também acrescenta alguns elementos gráficos no desenho atual, e mistura e insere um desenho como referência externa.

7.10.2. Comando "Acionamento de comandos do editor"

Aqui mostramos como acionar comandos padrão do editor, fornecendo antes os dados que o usuário entraria interativamente. O programa simula a entrada interativa, usando funções da classe `TQSEag.Entry`. Um comando do editor é marcado como "Checado" e a janela de mensagens é limpa.

7.10.3. Comando "Entrada de pontos"

Aqui são testados os comandos de entrada de pontos, ângulos, valores e texto. Um texto, uma linha e uma linha poligonal fechada e preenchida são construídos. No final, uma linha elástica é deixada ligada nas coordenadas do texto.

7.10.4. Comando "Seleção de elementos e movimentação"

Testamos o comando para locação de elementos, e mostramos como iterar pelos elementos selecionados. No final, a lista selecionada se acende, e movimentamos os elementos com o fornecimento de novo ponto.

7.10.5. Comando "Estados do editor"

Diversos estados do editor são alterados, e o resultado é mostrado antes e depois da alteração. A grade de coordenadas é deixada ligada no final do comando.

8. Módulo TQSBUILD

O módulo TQSBUILD permite listar e editar dados de edifícios existentes, assim como criar edifícios novos. Por dados do edifício no TQS entendemos as informações de pisos, cotas, materiais, cobrimentos, cargas e alguns critérios. O TQS tem um editor interativo específico para esta tarefa, chamado de "Editor de Dados do Edifício", que tem uma tela como essa:

Uma estrutura de dados de edifício é criada na memória com a classe `Building()`:

```
from TQS import TQSBUILD
build = TQSBUILD.Building()
```

Opcionalmente o edifício pode ser criado a partir de um modelo salvo, fornecendo-se o nome do modelo:

```
build = TQSBUILD.Building("TQS")
```

A atribuição de dados do edifício segue lógica aproximada das abas do Editor, com subobjetos correspondentes como mostra a tabela:

Objeto	Aba	Dados e funções tratadas
<code>file</code>		Ler e gravar arquivos
<code>project</code>	Gerais	Dados gerais
<code>model</code>	Modelo	Modelo estrutural, P-Delta, análise dinâmica, interação solo-estrutura, efeito incremental
<code>model.precastmodel</code>	Modelo, pré-moldados	Fases e etapas construtivas de estruturas pré-moldadas
<code>floorsplan</code>	Pavimentos	Dados por pavimento
<code>materials</code>	Materiais	Classes de concreto por tipo de elemento
<code>materials.precast</code>	Materiais, pré-moldados	Classes de concreto pré-moldado
<code>covers</code>	Cobrimentos	Cobrimentos por tipo de elemento estrutural

<code>covers.aci</code>	Cobrimentos	Cobrimentos definidos no ACI-318
<code>covers.precast</code>	Cobrimentos, pré-moldados	Cobrimentos de concreto pré-moldado
<code>covers.floorplan</code>	Cobrimentos, valores por planta	Cobrimentos diferenciados por planta
<code>loads</code>	Cargas	Dados de casos simples de carregamento
<code>loads.windloads</code>	Cargas, Vento	Dados de casos de vento
<code>loads.precastloads</code>	Cargas, Pré-moldados	Carregamentos em etapas de estruturas pré-moldadas.
<code>management</code>	Gerenciamento	Modos de ligação com o GerPrE

8.1. Funções globais sem o objeto Building

As seguintes funções podem ser chamadas antes de que qualquer edifício seja aberto.

BuildingContext ()

Retorna dados do contexto do edifício, se pasta atual de edifício.

`nprjpv` Número do projeto do pavimento
`nprjed` Número do projeto do edifício
`nombde` Nome completo do EDIFÍCIO.BDE
`nomedi` Nome do edifício
`nompav` Nome do pavimento
`istat` (!=0) se fora do contexto do edifício

BuildingRoot ()

Retorna a raiz geral da árvore de edifícios

8.2. Funções globais sem edifício aberto

A seguinte função pode ser chamada sem abrir o edifício através do `file.Open()`:

Build.RootFolder (filename)

Entra na pasta raiz de um edifício de nome filename

Retorna (!=0) se o edifício não existe

8.3. Leitura e gravação com TQSBuilding.file

Ao criar um objeto `TQSBuild.Building()`, o módulo cria um edifício vazio na memória. Este objeto então pode ler e salvar um edifício – permitindo então listar ou modificar dados de edifícios existentes ou gravar novos edifícios. Esta lógica torna simples gravar uma família de edifícios com pequenas alterações de um para outro, a

Error! Use the Home tab to apply Título 1 to the text that you want to appear here.⁸⁷

partir de um edifício original. Um edifício novo recebe atributos iniciais iguais aos de um edifício criado no Editor de Dados do Edifício.

`file.Open(filename)`

Carrega um edifício com nome filename

Retorna (!=0) se não leu.

`file.Save()`

Salva o edifício atual aberto com Open

Retorna (!=0) se não gravou.

`file.SaveAs(filename)`

Salva um edifício com nome filename

Retorna (!=0) se não gravou.

`file.RootFolder (filename)`

Entra na pasta raiz de um edifício de nome filename

`file.SuppressMessages (isuprimir)`

Suprime mensagens de criação e eliminação de arquivos no salvamento do edifício (0)

Não (1) Sim

8.4. Dados gerais com TQSBldgng.project

`project.projectTitle`

Título do edifício

`project.clientTitle`

Título do cliente

`project.projectAddress`

Endereço da obra

`project.projectNumber`

Número arbitrário de projeto 1..9999

`project.blueprintPrefix`

Prefixo arbitrário para plantas plotadas

`project.projectDescription`

Descrição do projeto

`project.structureType`

Tipo da estrutura:

(0) Concreto armado/protendido

(1) Concreto pré-moldado

(2) Alvenaria estrutural

(3) Paredes de concreto

```
project.formworkActive
```

(1) Se com projeto de formas de madeira

```
project.concreteCode
```

Norma de concreto TQSBuild.NORMA_ xxxx

Constante	Norma
TQSBuild.NORMA_INDEF	Indefinida
TQSBuild.NORMA_NB1_78	NB1-78
TQSBuild.NORMA_NBR_6118_2003	NBR-6118:2003
TQSBuild.NORMA_NBR_6118_2014	NBR-6118:2014
TQSBuild.NORMA_CIRSOC	CIRSOC-2005
TQSBuild.NORMA_ACI_318_05	ACI-318-05
TQSBuild.NORMA_ACI_318_14	ACI-318-14

```
project.masonryCode
```

Norma de alvenaria estrutural TQSBuild.NORMALVI_ xxxx

Constante	Norma
TQSBuild.NORMALVI_10837	NBR 10837 : 1989 - Blocos vazados de Concreto
TQSBuild.NORMALVI_15961	NBR 15961-1 : 2011 - Blocos de Concreto
TQSBuild.NORMALVI_15812	NBR 15812-1 : 2010 - Blocos Cerâmicos
TQSBuild.NORMALVI_16868	NBR 16868-1:2020 - Blocos de Concreto e Cerâmicos

```
project.concreteWallsCode()
```

Norma de paredes de concreto TQSBuild.NORMAPAREDI_ xxxx

Constante	Norma
TQSBuild.NORMAPAREDI_NBR16055	NBR 16055:2012 - Paredes de Concreto

```
project.codeStrictUse
```

(1) Se forçar o uso de critérios de norma

```
project.floorHeight
```

Cota inicial da planta (m)

```
project.country
```

País TQSBuild.PAIS_ xxx

Constante	País
TQSBuild.PAIS_BRASIL	Brasil

TQSBuild. PAIS_ARGENTINA	Argentina
TQSBuild. PAIS_COLOMBIA	Colômbia
TQSBuild. PAIS_BOLIVIA	Bolívia
TQSBuild. PAIS_EUA	Estados unidos
TQSBuild. PAIS_PARAGUAI	Paraguai

8.5. Modelo de cálculo com TQSBuilding.model

`model.structuralModel`

Modelo TQSBuild.PORTICO_XXX

Constante	País
TQSBuild. PORTICO_HORVER	Pórtico Modelo IV
TQSBuild. PORTICO_LAJES	Pórtico Modelo VI

`model.expansionJoints`

- (0) Estrutura única (1) Duas ou mais estruturas separadas por junta de dilatação (1)

`model.transferBeamModel`

- (1) Dois modelos com vigas de transição: inércia enrijecida e inércia real

`model.transferBeamModelInertiaMultiplier`

Para modelo de viga de transição enrijecida, o multiplicador de inércia à flexão

`model.transferBemModelPolarInertiaDivider`

Para modelo de viga de transição enrijecida, o divisor da inércia à torção

`model.secondOrderEffect`

Efeitos globais de segunda ordem calculados por (0) GamaZ (1) P-Delta

`model.structuralDynamicsAnalysis`

- (0) Análise estática (1) Análise dinâmica da estrutura

`model.soilStructureInteraction`

- (1) Se integrado ao sistema de interação solo estruturas SISEs

`model.soilStructureIntegrateFoundations`

- (1) Agregar a fundação discretizada no SISEs no pórtico espacial

`model.incrementalAnalysis`

- (1) Analisar a estrutura considerando efeito incremental

8.5.1. Modelo de pré-moldados

As etapas e as regiões construtivas são separadas no subobjeto

`TQSBuilding.model.precastmodel.`

`model.precastmodel.Clear()`

Limpa a lista de etapas e regiões construtivas

`model.precastmodel.stagesNumber`

Número de etapas construtivas de estrutura pré-moldada

`model.precastmodel.regionsNumber`

Número de regiões construtivas de estrutura pré-moldada

`model.precastmodel.GetFloor(istage, iregion)`

Retorna para uma etapa e região, até que piso será construído e até que piso será solidarizado

`istage` Etapa construtiva 0..stagesNumber-1

`iregion` Região construtiva 0..regionsNumber-1

Retorna:

`ifloor` Piso até onde será construído

`ifloormonolithic` Piso até onde será solidarizado

`model.precastmodel.SetFloor(istage, iregion, ifloor, ifloormonolithic)`

Define para uma etapa e região, até que piso será construído e até que piso será solidarizado

`istage` Etapa construtiva 0..stagesNumber-1

`iregion` Região construtiva 0..regionsNumber-1

`ifloor` Piso até onde será construído

`ifloormonolithic` Piso até onde será solidarizado

8.6. Dados do pavimento - TQSBUILDING.floorsplan

Plantas são organizadas em um vetor de dados com base zero, os dados são fornecidos por planta. Plantas podem ser criadas, lidas ou apagadas sempre com base no índice da planta.

`floorsplan.floorsPlanNumber`

Número de plantas. Plantas com repetição são uma única planta (somente leitura).

`floorsplan.Clear()`

Elimina todas as plantas do edifício

`floorsplan.Create(ipla)`

Cria nova planta de índice ipla com base zero

`ipla` Índice da planta com base zero

`floorsplan.CreateAbove(nompla, pd, iclass):`

Cria uma planta acima de todas as outras. Define nome e pé-direito

Error! Use the Home tab to apply Título 1 to the text that you want to appear here.91

nompla (string) Nome e título da planta
pd Altura da planta (m)
iclass Classe do piso ICLxxx

floorsplan.**Erase** (ipla)
Apaga a planta ipla (0..floorsPlanNumber-1)
ipla Índice da planta com base zero

floorsplan.**GetRepetition** (ipla)
Retorna o número de repetições de uma planta
ipla Índice da planta com base zero
Retorna:
irep Número de repetições da planta

floorsplan.**SetRepetition** (ipla, irep)
Define o número de projeto da planta
ipla Índice da planta com base zero
irep Número de repetições da planta

floorsplan.**GetFloorHeight** (ipla)
Retorna a diferença de cota entre a planta atual e a de baixo (m)
ipla Índice da planta com base zero
Retorna:
pd Altura da planta

floorsplan.**SetFloorHeight** (ipla, pd)
Retorna a diferença de cota entre a planta atual e a de baixo (m)
ipla Índice da planta com base zero
pd Altura da planta

floorsplan.**GetClass** (ipla)
Retorna a classe de uma planta do tipo TQSBuild.ICLxxx
ipla Índice da planta com base zero
Retorna:
iclass Classe tipo TQSBuild.ICLxxx

floorsplan.**SetClass** (ipla, iclass)
Define a classe de uma planta do tipo TQSBuild.ICLxxx
ipla Índice da planta com base zero
iclass Classe tipo TQSBuild.ICLxxx

Constante	Significado
TQSBUILD. ICLFUN	Fundação
TQSBUILD. ICLSUB	Subsolo
TQSBUILD. ICLTER	Térreo
TQSBUILD. ICLMEZ	Mezzanino
TQSBUILD. ICLTRN	Transição
TQSBUILD. ICLPRI	Primeiro
TQSBUILD. ICLTIP	Andar tipo
TQSBUILD. ICLCOB	Cobertura
TQSBUILD. ICLATI	Ático
TQSBUILD. ICLDUP	Duplex
TQSBUILD. ICLTRI	Triplex

`floorsplan.GetProjectNumber (ipla)`

Retorna o número de projeto

`ipla` Índice da planta com base zero

Retorna:

`nprj` Número do projeto da planta

`floorsplan.SetProjectNumber (ipla, nprj)`

Define o número de projeto da planta

`ipla` Índice da planta com base zero

`nprj` Número arbitrário de projeto da planta

Usado em alguns nomes de arquivos.

`floorsplan.GetName (ipla)`

Retorna o nome da planta

`ipla` Índice da planta com base zero

Retorna:

`sname` Nome da planta.

Usado também como nome da pasta dos arquivos da planta

`floorsplan.SetName (ipla, sname)`

Define o nome da planta.

`ipla` Índice da planta com base zero

`sname` Nome da planta.

Usado também como nome da pasta dos arquivos da planta

`floorsplan.GetDescription (ipla)`

Retorna a descrição da planta

`ipla` Índice da planta com base zero

Retorna:

`sdescription` Descrição da planta

Error! Use the Home tab to apply Título 1 to the text that you want to appear here.93

floorsplan.SetDescription (ipla, sdescription)

Define a descrição da planta

ipla Índice da planta com base zero

sdescription Descrição da planta

floorsplan.GetTitle (ipla)

Retorna a o título da planta

ipla Índice da planta com base zero

Retorna:

stitle Título da planta

floorsplan.SetTitle (ipla, stitle)

Define a o título da planta

ipla Índice da planta com base zero

stitle Título da planta

floorsplan.GetBlueprintPrefix (ipla)

Retorna o prefixo de plantas

ipla Índice da planta com base zero

Retorna:

sprefix Prefixo opcional para a planta nos layouts de plotagem

floorsplan.SetBlueprintPrefix (ipla, sprefix)

Define o prefixo de plantas

ipla Índice da planta com base zero

sprefix Prefixo opcional para a planta nos layouts de plotagem

floorsplan.GetPlaneFrameModel (ipla)

Retorna o modelo estrutural da planta

ipla Índice da planta com base zero

Retorna:

igrpla Modelo estrutural tipo TQSBUILD.GRELHA_xxxx

floorsplan.SetPlaneFrameModel (ipla, igrpla)

Define o modelo estrutural da planta

ipla Índice da planta com base zero

igrpla Modelo estrutural tipo TQSBUILD.GRELHA_xxxx

Constante	Modelo de grelha
TQSBUILD.GRELHA_INDEF	Modelo de grelha indefinido
TQSBUILD.GRELHA_VIGAS	Vigas contínuas
TQSBUILD.GRELHA_LAJNER	Lajes nervuradas
TQSBUILD.GRELHA_LAJPLA	Lajes maciças
TQSBUILD.GRELHA_GREVIG	Grelha só de vigas
TQSBUILD.GRELHA_PLACAS	Lajes planas com placas
TQSBUILD.GRELHA_AUTOMA	Modo automático – lajes nervuradas ou maciças

`floorsplan.GetPreStressed (ipla)`

Retorna se o pavimento é protendido

`ipla` Índice da planta com base zero

Retorna:

`iprpla` (1) Se o pavimento é protendido

`floorsplan.SetPreStressed (ipla, iprpla)`

Define o modelo de grelha

`ipla` Índice da planta com base zero

`iprpla` (1) Se o pavimento é protendido

`floorsplan.GetTemperatureAnalysis (ipla)`

Retorna se o pavimento é calculado com carregamento de temperatura

`ipla` Índice da planta com base zero

Retorna:

`itemper` (1) Definida por elemento no Modelador (2) Uniforme na planta

`floorsplan.SetTemperatureAnalysis (ipla, itemper)`

Define se o pavimento é calculado com carregamento de temperatura

`ipla` Índice da planta com base zero

`itemper` (1) Definida por elemento no Modelador (2) Uniforme na planta

`floorsplan.GetTemperatureTransVariation (ipla)`

Retorna o valor da variação transversal de temperatura uniforme na planta

`ipla` Índice da planta com base zero

Retorna:

`temvartra` Variação transversal de temperatura uniforme na planta em graus

`floorsplan.SetTemperatureTransVariation (ipla, temvartra)`

Define o valor da variação transversal de temperatura uniforme na planta

`ipla` Índice da planta com base zero

`temvartra` Variação transversal de temperatura uniforme na planta em graus

`floorsplan.GetTemperatureLongVariation (ipla)`

Retorna o valor da variação longitudinal de temperatura uniforme na planta

`ipla` Índice da planta com base zero

Retorna:

`temvarlon` Variação longitudinal de temperatura uniforme na planta em graus

`floorsplan.SetTemperatureLongVariation (ipla, temvarlon)`

Define o valor da variação longitudinal de temperatura uniforme na planta

`ipla` Índice da planta com base zero

`temvarlon` Variação longitudinal de temperatura uniforme na planta em graus

Error! Use the Home tab to apply Título 1 to the text that you want to appear here.95

floorsplan.GetTemperatureAnalysis2 (ipla)

Retorna se o pavimento tem um segundo caso de temperatura definido

ipla Índice da planta com base zero

Retorna:

itemper2 (1) Segundo caso de temperatura definido

floorsplan.SetTemperatureAnalysis2 (ipla, itemper2)

Define se o pavimento tem um segundo caso de temperatura definido

ipla Índice da planta com base zero

itemper2 (1) Segundo caso de temperatura definido

floorsplan.GetTemperatureTransVariation2 (ipla)

Retorna o valor da variação transversal do segundo caso de temperatura

ipla Índice da planta com base zero

Retorna:

temvartra2 Variação transversal de temperatura do segundo caso

floorsplan.SetTemperatureTransVariation2 (ipla, temvartra2)

Define o valor da variação transversal do segundo caso de temperatura

ipla Índice da planta com base zero

temvartra2 Variação transversal de temperatura do segundo caso

floorsplan.GetTemperatureLongVariation2 (ipla)

Retorna o valor da variação longitudinal do segundo caso de temperatura

ipla Índice da planta com base zero

Retorna:

temvarlon2 Variação longitudinal de temperatura do segundo caso

floorsplan.SetTemperatureLongVariation2 (ipla, temvarlon2)

Define o valor da variação longitudinal do segundo caso de temperatura

ipla Índice da planta com base zero

temvarlon2 Variação longitudinal de temperatura do segundo caso

floorsplan.GetShrinkageAnalysis (ipla)

Retorna se o pavimento é calculado com carregamento de retração

ipla Índice da planta com base zero

Retorna:

ishrink (1) Definida por elemento no Modelador (2) Uniforme na planta

floorsplan.SetShrinkageAnalysis (ipla, ishrink)

Define se o pavimento é calculado com carregamento de retração

ipla Índice da planta com base zero

ishrink (1) Definida por elemento no Modelador (2) Uniforme na planta

`floorsplan.GetShrinkageVariation (ipla)`

Retorna se o pavimento é calculado com carregamento de retração

`ipla` Índice da planta com base zero

Retorna:

`shrinkage` Variação transversal de temperatura equivalente em graus

`floorsplan.SetShrinkageVariation (ipla, shrinkage)`

Define se o pavimento é calculado com carregamento de retração

`ipla` Índice da planta com base zero

`shrinkage` Variação transversal de temperatura equivalente em graus

`floorsplan.GetDegreesOfFreedom (ipla)`

Retorna se o tipo de modelo de grelha da planta

`ipla` Índice da planta com base zero

Retorna:

`iporgre` (0) Automático (1) 3 graus de liberdade (2) 6 graus de liberdade

`floorsplan.SetDegreesOfFreedom (ipla, iporgre)`

Define se o pavimento é calculado com carregamento de retração

`ipla` Índice da planta com base zero

`iporgre` (0) Automático (1) 3 graus de liberdade (2) 6 graus de liberdade

`floorsplan.GetVehicleLoad (ipla)`

Retorna se o tipo de modelo de grelha da planta

`ipla` Índice da planta com base zero

Retorna:

`ipavveic` Veículos (0) não (1) <= 30kN (2) >30kN

`floorsplan.SetVehicleLoad (ipla, ipavveic)`

Define se o pavimento é calculado com carregamento de retração

`ipla` Índice da planta com base zero

`ipavveic` Veículos (0) não (1) <= 30kN (2) >30kN

`floorsplan.GetSlopedElements (ipla)`

Retorna (1) se a planta contém elementos inclinados

`ipla` Índice da planta com base zero

Retorna:

`islopedelem` (1) se a planta contém elementos inclinados

`floorsplan.SetSlopedElements (ipla, islopedelem)`

Retorna (1) se a planta contém elementos inclinados

`ipla` Índice da planta com base zero

`islopedelem` (1) se a planta contém elementos inclinados

Error! Use the Home tab to apply Título 1 to the text that you want to appear here.97

`floorsplan.GetAuxiliaryFloorsNumber (ipla)`

Retorna o número de pisos auxiliares associados à esta planta

`ipla` Índice da planta com base zero

Retorna:

`numfloors` Número de pisos auxiliares

`floorsplan.SetAuxiliaryFloorsNumber (ipla, numfloors)`

Define o número de pisos auxiliares associados à esta planta

`ipla` Índice da planta com base zero

`numfloors` Número de pisos auxiliares

`floorsplan.GetAuxiliaryFloorUnderneath (ipla, ifloor)`

Retorna o rebaixo de um piso auxiliar (m)

`ipla` Índice da planta com base zero

`ifloor` Índice do piso auxiliar (0..GetAuxiliaryFloorsNumber-1)

Retorna:

`undereneath` Rebaixo do piso undereneath (m)

`floorsplan.SetAuxiliaryFloorUnderneath (ipla, ifloor, undereneath)`

Define o rebaixo de um piso auxiliar (m)

`ipla` Índice da planta com base zero

`ifloor` Índice do piso auxiliar (0..GetAuxiliaryFloorsNumber-1)

`undereneath` Rebaixo do piso undereneath (m)

`floorsplan.GetFloorsNumber`

Retorna o número de pisos. É diferente do número de plantas, pois uma planta pode ter repetições.

`floorsplan.GetFloorPath (ipla)`

Retorna a pasta da planta

`ipla` Índice da planta (0..floorsPlanNumber-1)

Retorna:

`path` Pasta/diretório

`floorsplan.SetFloorPath (ipla, path)`

Define a pasta da planta

`ipla` Índice da planta (0..floorsPlanNumber-1)

`path` Pasta/diretório

`floorsplan.GetBeamsPath (ipla)`

Retorna a pasta de vigas se diferente do padrão

`ipla` Índice da planta (0..floorsPlanNumber-1)

Retorna:

`path` Pasta/diretório

`floorsplan.SetBeamsPath(ipla, path)`

Define a pasta de vigas se diferente do padrão

`ipla` Índice da planta (0..floorsPlanNumber-1)

`path` Pasta/diretório

`floorsplan.GetFormworkPath(ipla)`

Retorna a pasta de madeira se diferente do padrão

`ipla` Índice da planta (0..floorsPlanNumber-1)

Retorna:

`path` Pasta/diretório

`floorsplan.SetFormworkPath(ipla, path)`

Define a pasta de madeira se diferente do padrão

`ipla` Índice da planta (0..floorsPlanNumber-1)

`path` Pasta/diretório

`floorsplan.GetMasonryPath(ipla)`

Retorna a pasta de alvenaria estrutural se diferente do padrão

`ipla` Índice da planta (0..floorsPlanNumber-1)

Retorna:

`path` Pasta/diretório

`floorsplan.SetMasonryPath(ipla, path)`

Define a pasta de alvenaria estrutural se diferente do padrão

`ipla` Índice da planta (0..floorsPlanNumber-1)

`path` Pasta/diretório

`floorsplan.GetStairsPath(ipla)`

Retorna a pasta de escadas se diferente do padrão

`ipla` Índice da planta (0..floorsPlanNumber-1)

Retorna:

`path` Pasta/diretório

`floorsplan.SetStairsPath(ipla, path)`

Define a pasta de escadas se diferente do padrão

`ipla` Índice da planta (0..floorsPlanNumber-1)

`path` Pasta/diretório

`floorsplan.GetOthersPath(ipla)`

Retorna a pasta de elementos especiais se diferente do padrão

`ipla` Índice da planta (0..floorsPlanNumber-1)

Retorna:

`path` Pasta/diretório

`floorsplan.SetOthersPath(ipla, path)`

Define a pasta de elementos especiais se diferente do padrão

Error! Use the Home tab to apply Título 1 to the text that you want to appear here.99

`ipla` Índice da planta (0..floorsPlanNumber-1)
`path` Pasta/diretório

8.7. Materiais e o objeto TQSBUILDING.MATERIALS

Atualmente somente a associação de classes de concreto e elementos estruturais ou plantas e pisos estão definidos nos dados do edifício.

`materials.disableConcreteCheck`

(1) Para desativar verificação do fck mínimo

`materials.environmentalClass`

Classe de agressividade I..IV (valores 0..3)

`materials.dryClimate`

(1) Se clima seco

`materials.qualityControl`

(1) Se concreto realizado com controle de qualidade

`materials.beamConcreteStrength`

Concreto de vigas e lajes

`materials.columnConcreteStrength`

Concreto de pilares

`materials.foundationConcreteStrength`

Concreto de fundações

`materials.columnConcreteVariations`

Número de variações de resistência de concreto em pilares

`materials.GetColumnConcreteVariation` (index)

Retorna um concreto e o último piso onde ele vale

index Índice da variação 0..columnConcreteVariations-1

Retorna:

sfck Título do fck até o piso abaixo

iuntilfloor Último piso no qual vale o Fck

`materials.SetColumnConcreteVariation` (index, sfck, iuntilfloor)

Define um concreto e o último piso onde ele vale

index Índice da variação 0..columnConcreteVariations-1

sfck Título do fck até o piso abaixo

iuntilfloor Último piso no qual vale o Fck

`materials.GetFloorPlanConcreteStrength` (ipla)

Concreto de vigas e lajes específicos de uma planta

`ipla` Índice da planta (`0..floorsPlanNumber-1`)

Retorna:

`sfck` Título do concreto

```
materials.SetFloorPlanConcreteStrength(ipla, sfck)
```

Concreto de vigas e lajes específicos de uma planta

`ipla` Índice da planta (`0..floorsPlanNumber-1`)

`sfck` Título do concreto

```
materials.masonryMaterial
```

Material de alvenaria `TQBuilding.MATRALVI_xxx`

Constante	Valor
<code>TQSBuild.MATRALVI_CONVAZ</code>	Blocos vazados de CONCRETO
<code>TQSBuild.MATRALVI_CONMAC</code>	Blocos maciços de CONCRETO
<code>TQSBuild.MATRALVI_CERVAZ</code>	Blocos CERÂMICOS vazados
<code>TQSBuild.MATRALVI_CERMAC</code>	Blocos CERÂMICOS maciços
<code>TQSBuild.MATRALVI_SILVAZ</code>	Blocos SILICO-CALCÁREOS vazados
<code>TQSBuild.MATRALVI_SILMAC</code>	Blocos SILICO-CALCÁREOS maciços
<code>TQSBuild.MATRALVI_OUTROS</code>	Outro (Bloco/Tijolo)

```
materials.aciExposureClass
```

Classe de exposição ambiental do ACI-318 `TQSBuild.CLAGCIRSOC_xxx`

Constante	Valor
<code>TQSBuild.CLAGCIRSOC_A1</code>	Não agressiva
<code>TQSBuild.CLAGCIRSOC_A2</code>	Moderada
<code>TQSBuild.CLAGCIRSOC_A3</code>	Quente úmido
<code>TQSBuild.CLAGCIRSOC_CL</code>	Submerso
<code>TQSBuild.CLAGCIRSOC_M1</code>	Marinho 1
<code>TQSBuild.CLAGCIRSOC_M2</code>	Marinho 2
<code>TQSBuild.CLAGCIRSOC_M3</code>	Marinho 3
<code>TQSBuild.CLAGCIRSOC_C1</code>	Gelo e degelo 1
<code>TQSBuild.CLAGCIRSOC_C2</code>	Gelo e degelo 2
<code>TQSBuild.CLAGCIRSOC_Q1</code>	Agressividade química 1
<code>TQSBuild.CLAGCIRSOC_Q2</code>	Agressividade química 2
<code>TQSBuild.CLAGCIRSOC_Q3</code>	Agressividade química 3

8.7.1. Materiais pré-moldados `TQBuilding.materials.precast`

Os concretos para elementos pré-moldados e pré-fabricados são definidos através do subobjeto `materials.precast`.

Error! Use the Home tab to apply Título 1 to the text that you want to appear here.101

`materials.precast.pbeamConcreteStrength`

Concreto das vigas pré-moldadas

`materials.precast.pbeamDemouldingConcreteStrength`

Concreto das vigas pré-moldadas no saque

`materials.precast.pbeamLiftingConcreteStrength`

Concreto das vigas pré-moldadas no içamento

`materials.precast.pcolumnConcreteStrength`

Concreto dos pilares pré-moldados

`materials.precast.pcolumnDemouldingConcreteStrength`

Concreto dos pilares pré-moldados no saque

`materials.precast.pcolumnLiftingConcreteStrength`

Concreto dos pilares pré-moldados no içamento

`materials.precast.pslabConcreteStrength`

Concreto das lajes pré-moldadas

`materials.precast.pslabDemouldingConcreteStrength`

Concreto das lajes pré-moldadas no saque

`materials.precast.pslabLiftingConcreteStrength`

Concreto das lajes pré-moldadas no içamento

`materials.precast.pfoundationConcreteStrength`

Concreto das fundações pré-moldadas

`materials.precast.pfoundationDemouldingConcreteStrength`

Concreto das fundações pré-moldadas no saque

`materials.precast.pfoundationLiftingConcreteStrength`

Concreto das fundações pré-moldadas no içamento

8.8. Cobrimentos TQSBuilding.covers

`covers.disableCoverCheck`

(1) Para desativar verificação do cobrimento mínimo

`covers.beamCover`

Cobrimento cm vigas

`covers.columnCover`

Cobrimento cm pilares

`covers.foundationCover`

Cobrimento em fundações

`covers.foundationSecCover`

Cobrimento secundário em fundações

`covers.slabLowerCover`

Cobrimento em laje inferior

`covers.slabUpperCover`

Cobrimento em laje superior

`covers.slabSecLowerCover`

Cobrimento secundário em laje inferior

`covers.slabSecUpperCover`

Cobrimento secundário em laje superior

`covers.prestressedSlabLowerCover`

Cobrimento em protendido inferior

`covers.prestressedSlabUpperCover`

Cobrimento em protendido superior

`covers.prestressedSlabSecCover`

Cobrimento em protendido secundário

`covers.beamAndSlabSoilCover`

Cobrimento em vigas e lajes contato com o solo

`covers.columnSoilCover`

Cobrimento em pilares contato com o solo

8.8.1. Cobrimentos ACI-318 TQSBUILDING.covers.aci

Cobrimentos específicos do ACI-318 são tratados pelo subobjeto

`TQSBUILDING.covers.aci`.

`covers.aci.aciPrestressClass`

Classe de protensão `TQSBUILD.CLPRCIRSOC_XXX` para consideração de cobrimentos

Constante	Valor
<code>TQSBUILD.CLPRCIRSOC_FtU</code>	$F_t \leq 0.7 \times \text{SQRT}(F'c)$
<code>TQSBUILD.CLPRCIRSOC_FTT</code>	$0.7 \times \text{SQRT}(F'c) < F_t \leq \text{SQRT}(F'c)$
<code>TQSBUILD.CLPRCIRSOC_FTC</code>	$F_t > \text{SQRT}(F'c)$

`covers.aci.slabNotprestressedExposedCover`

Error! Use the Home tab to apply Título 1 to the text that you want to appear here.103

Cobrimento cm lajes não protendidas expostas ao ambiente

`covers.aci.slabPrestressedSoilCover`

Cobrimento cm lajes protendidas em contato com o solo

`covers.aci.slabPrestressedExposedCover`

Cobrimento cm lajes protendidas em contato com o solo

`covers.aci.beamSoilCover`

Cobrimento cm vigas em contato com o solo

`covers.aci.beamExposedCover`

Cobrimento cm vigas não protendidas expostas ao ambiente

`covers.aci.beamPrestressedExposedCover`

Cobrimento cm vigas pré-moldadas expostas ao ambiente

`covers.aci.columnExposedCover`

Cobrimento cm pilar não protendido exposto ao ambiente

`covers.aci.columnPrecastExposedCover`

Cobrimento cm pilar pré-moldado exposto ao ambiente

8.8.2. Cobrimentos pré-moldados TQSBuilding.covers.precast

Cobrimentos específicos de elementos pré-moldados são tratados pelo subobjeto TQSBuilding.covers.precast.

`covers.precast.pbeamCover`

Cobrimento cm viga pré-moldada

`covers.precast.pcolumnCover`

Cobrimento cm pilares pré-moldados

`covers.precast.pfoundationCover`

Cobrimento cm fundações pré-moldadas

`covers.precast.pfoundationSecCover`

Cobrimento cm fundações pré-moldadas adicional secundário (<0 primário)

`covers.precast.pslabLowerCover`

Cobrimento cm laje inferior

`covers.precast.pslabUpperCover`

Cobrimento cm laje superior

`covers.precast.pslabSecLowerCover`
 Cobrimento cm laje adicional secundário inferior

8.8.3. Cobrimentos por planta `TQSBldg.covers.floorplan`

Cobrimentos específicos por planta são tratados pelo subobjeto
`TQSBldg.covers.floorplan`.

`covers.floorplan.GetFloorPlanBeamsCover (ipla)`
 Retorna o cobrimento (cm) de vigas para a planta, se ($\neq 0$)
 ipla Índice da planta ($0 \dots \text{floorsPlanNumber}-1$)
 Retorna:
 cover Valor do cobrimento em cm ou (0)

`covers.floorplan.SetFloorPlanBeamsCover (ipla, cover)`
 Define o cobrimento (cm) de vigas para a planta, se ($\neq 0$)
 ipla Índice da planta ($0 \dots \text{floorsPlanNumber}-1$)
 cover Valor do cobrimento em cm ou (0)

`covers.floorplan.GetFloorPlanSlabLowerCover (ipla)`
 Retorna o cobrimento inferior (cm) de lajes para a planta, se ($\neq 0$)
 ipla Índice da planta ($0 \dots \text{floorsPlanNumber}-1$)
 Retorna:
 cover Valor do cobrimento em cm ou (0)

`covers.floorplan.SetFloorPlanSlabLowerCover (ipla, cover)`
 Define o cobrimento inferior (cm) de lajes para a planta, se ($\neq 0$)
 ipla Índice da planta ($0 \dots \text{floorsPlanNumber}-1$)
 cover Valor do cobrimento em cm ou (0)

`covers.floorplan.GetFloorPlanSlabUpperCover (ipla)`
 Retorna o cobrimento superior (cm) de lajes para a planta, se ($\neq 0$)
 ipla Índice da planta ($0 \dots \text{floorsPlanNumber}-1$)
 Retorna:
 cover Valor do cobrimento em cm ou (0)

`covers.floorplan.SetFloorPlanSlabUpperCover (ipla, cover)`
 Retorna o cobrimento superior (cm) de lajes para a planta, se ($\neq 0$)
 ipla Índice da planta ($0 \dots \text{floorsPlanNumber}-1$)
 cover Valor do cobrimento em cm ou (0)

`covers.floorplan.GetFloorPlanSlabSecLowerCover (ipla)`
 Retorna o cobrimento secundário inferior (cm) de lajes para a planta, se ($\neq 0$)
 ipla Índice da planta ($0 \dots \text{floorsPlanNumber}-1$)
 Retorna:

cover Valor do cobrimento em cm ou (0)

`covers.floorplan.SetFloorPlanSlabSecLowerCover (ipla, cover)`

Define o cobrimento secundário inferior (cm) de lajes para a planta, se (!=0)

ipla Índice da planta (0..floorsPlanNumber-1)

cover Valor do cobrimento em cm ou (0)

`covers.floorplan.GetFloorPlanSlabSecUpperCover (ipla)`

Retorna o cobrimento secundário superior (cm) de lajes para a planta, se (!=0)

ipla Índice da planta (0..floorsPlanNumber-1)

Retorna:

cover Valor do cobrimento em cm ou (0)

`covers.floorplan.SetFloorPlanSlabSecUpperCover (ipla, cover)`

Define o cobrimento secundário superior (cm) de lajes para a planta, se (!=0)

ipla Índice da planta (0..floorsPlanNumber-1)

cover Valor do cobrimento em cm ou (0)

`covers.floorplan.GetFloorPlanPrestressedSlabLowerCover (ipla)`

Retorna o cobrimento inferior (cm) de lajes protendidas para a planta, se (!=0)

ipla Índice da planta (0..floorsPlanNumber-1)

Retorna:

cover Valor do cobrimento em cm ou (0)

`covers.floorplan.SetFloorPlanPrestressedSlabLowerCover (ipla, cover)`

Define o cobrimento inferior (cm) de lajes protendidas para a planta, se (!=0)

ipla Índice da planta (0..floorsPlanNumber-1)

cover Valor do cobrimento em cm ou (0)

`covers.floorplan.GetFloorPlanPrestressedSlabUpperCover (ipla)`

Retorna o cobrimento superior (cm) de lajes protendidas para a planta, se (!=0)

ipla Índice da planta (0..floorsPlanNumber-1)

Retorna:

cover Valor do cobrimento em cm ou (0)

`covers.floorplan.SetFloorPlanPrestressedSlabUpperCover (ipla, cover)`

Define o cobrimento superior (cm) de lajes protendidas para a planta, se (!=0)

ipla Índice da planta (0..floorsPlanNumber-1)

cover Valor do cobrimento em cm ou (0)

`covers.floorplan.GetFloorPlanPrestressedSlabSecCover (ipla)`

Retorna o cobrimento secundário (cm) de lajes protendidas para a planta, se (!=0)

ipla Índice da planta (0..floorsPlanNumber-1)

Retorna:

cover Valor do cobrimento em cm ou (0)

```
covers.floorplan.SetFloorPlanPrestressedSlabSecCover(ipla, cover)
```

Define o cobrimento secundário (cm) de lajes protendidas para a planta, se (!=0)

ipla Índice da planta (0..floorsPlanNumber-1)

cover Valor do cobrimento em cm ou (0)

8.9. Cargas e o objeto TQSBUILDING.LOADS

Os casos simples de cargas aceitos pelo TQS já são definidos através de *prefixos* convencionados. Casos que não se enquadram na lista de pré-definidos podem ser criados como *casos adicionais*. O acesso aos casos de carregamento é feito sempre tendo como chave o prefixo do caso.

```
loads.GetLoad(prefixo)
```

Retorna os parâmetros comuns de um caso simples de carregamento pré-definido

prefixo Prefixo TQSBUILD.PREFIXOxxxx

Retorna:

titulo Título do carregamento

ipermacidi Grupo (0) permanente (1) acidental (2) excepcional

ivigtrans (1) Se afetado pelo caso de viga de transição enrigecida

ireduzida (1) Se afetado pelo caso de carga acidental reduzida

imulaxi (1) Se aplicar multiplicador de área de pilares MULAXI

gamaf Ponderador de cargas GamaF favorável

igamafd (1) Se aplicar ponderador Gamaf desfavorável

gamafd Ponderador de cargas GamaF desfavorável

psi0 Fator de redução das combinações ELU

psi1 Fator de redução das combinações frequentes ELS

psi2 Fator de redução das combinações quase permanentes ELS

ipavimento (1) Se aplicado nas grelhas do pavimento

numcasos Número de casos diferentes gerados por este carregamento

istat (!= 0) Se prefixo indefinido ou erro

Prefixo de carga	Valor
TQSBUILDING.PREFIXOTODAS	Todas as cargas verticais
TQSBUILDING.PREFIXOPP	Peso próprio
TQSBUILDING.PREFIXOPEM	Cargas permanentes
TQSBUILDING.PREFIXOACID	Cargas acidentais
TQSBUILDING.PREFIXOEMPU	Empuxo
TQSBUILDING.PREFIXORETR	Retração
TQSBUILDING.PREFIXOSISM	Sismo
TQSBUILDING.PREFIXOTEMP	Temperatura
TQSBUILDING.PREFIXOVENT	Vento
TQSBUILDING.PREFIXODESA	Desaprumo
TQSBUILDING.PREFIXOHIPER	Hiperestático de protensão
TQSBUILDING.PREFIXOFORALI	Forças de alívio de protensão

TQSBuilding.PREFIXOADIC	Cargas adicionais do usuário
TQSBuilding.PREFIXOGELO	Gelo
TQSBuilding.PREFIXOFLUI	Pressão de fluídos
TQSBuilding.PREFIXOENCH	Cargas de enchente
TQSBuilding.PREFIXOTELH	Cargas acidentais em telhado
TQSBuilding.PREFIXOCHUV	Chuva
TQSBuilding.PREFIXONEVE	Neve
TQSBuilding.PREFIXOVEGE	Vento sobre gelo

loads.**SetLoad**(prefixo, titulo, ipermacid, ivigtrans, ireduzida, imulaxi, gamaf, igamafd, gamafd, psi0, psi1, psi2, ipavimento, numcasos, icasadi)

Fixa os parâmetros comuns de um caso simples de carregamento pré-definido TQS

prefixo Prefixo TQSBuild.PREFIXOxxxx
 titulo Título do carregamento
 ipermacid Grupo (0) permanente (1) acidental (2) excepcional
 ivigtrans (1) Se afetado pelo caso de viga de transição enrigecida
 ireduzida (1) Se afetado pelo caso de carga acidental reduzida
 imulaxi (1) Se aplicar multiplicador de área de pilares MULAXI
 gamaf Ponderador de cargas GamaF favorável
 igamafd (1) Se aplicar ponderador Gamaf desfavorável
 gamafd Ponderador de cargas GamaF desfavorável
 psi0 Fator de redução das combinações ELU
 psi1 Fator de redução das combinações frequentes ELS
 psi2 Fator de redução das combinações quase permanentes ELS
 ipavimento (1) Se aplicado nas grelhas do pavimento
 numcasos Número de casos diferentes gerados por este carregamento
 icasadi (1) Se este é um caso adicional definido pelo usuário

Retorna:

istat (!= 0) Se prefixo indefinido ou erro

loads.**CreateLoad**(prefixo)

Cria carregamento com novo prefixo

Retorna:

istat (!=0) se erro

loads.**GetAdditionalCasesNumber**(self)

Retorna o número de casos adicionais definidos. São casos com icasadi!=0 definido

loads.**GetAadditionalCasePrefix**(icasadi)

Retorna o prefixo de um caso adicional

icasadi Caso adicional 0..GetAdditionalCasesNumber()-1

Retorna:

prefixo Prefixo do caso adicional

`loads.GetLiveLoadReduction(iffloor)`

Retorna um redutor de sobrecargas para um piso

`iffloor` Número do piso do edifício, começando em zero

Retorna:

`reductor` Redutor de sobrecargas. (0.) Não reduz (1.) Reduz 100%

`loads.SetLiveLoadReduction(iffloor, reductor)`

Define um redutor de sobrecargas para um piso

`iffloor` Número do piso do edifício, começando em zero

`reductor` Redutor de sobrecargas. (0.) Não reduz (1.) Reduz 100%

`loads.outOfPlumbCases`

Número de casos de desaprumo

`loads.GetOutOfPlumbAngle(ioopcase)`

Retorna o ângulo do caso fornecido em graus

`ioopcase` Número do caso de desaprumo 0..`loads.outOfPlumbCases()`-1

Retorna:

`angle` Ângulo em planta do caso de desaprumo em graus

`loads.SetOutOfPlumbAngle(ioopcase, angle)`

Define o ângulo do caso fornecido em graus

`ioopcase` Número do caso de desaprumo 0..`loads.outOfPlumbCases()`-1

`angle` Ângulo em planta do caso de desaprumo em graus

8.9.1. Cargas de vento `TQSBUILD.loads.windloads`

Dados adicionais de vento são definidos pelo subobjeto

`TQSBUILD.loads.windloads`.

`loads.windloads.GetWindLoadParametersNBR ()`

Parâmetros de cálculo de caso de vento pela NBR-6123

Retorna:

`windv0` Velocidade básica m/s

`winds1` Fator topográfico

`iwndrug` Categoria de rugosidade I..IV (valores 0..3)

`winds3` Fator estatístico

`iwindclass` Classe (0)A (1)B (2)C

`numcaswind` Número de casos em direções diferentes

Error! Use the Home tab to apply Título 1 to the text that you want to appear here.109

```
loads.windloads.SetWindLoadParametersNBR(windv0, winds1, iwndrug,  
winds3, iwindclass, numcaswind)
```

Define os parâmetros de cálculo de caso de vento pela NBR-6123

windv0	Velocidade básica m/s
winds1	Fator topográfico
iwndrug	Categoria de rugosidade I..IV
winds3	Fator estatístico
iwindclass	Classe (0)A (1)B (2)C
numcaswind	Número de casos em direções diferentes

```
loads.windloads.GetWindLoadCase(iwindcase)
```

Retorna dados de um caso de vento em uma direção

iwindcase Número do caso de vento 0..numcaswind-1

Retorna:

ca	Coefficiente de arrasto
ang	Ângulo de vento em graus
icti	(1) Se cota inicial definida
cti	Cota inicial (m)

```
loads.windloads.SetWindLoadCase(iwindcase, ca, ang, icti, cti)
```

Define dados de um caso de vento em uma direção

iwindcase	Número do caso de vento 0..numcaswind-1
ca	Coefficiente de arrasto
ang	Ângulo de vento em graus
icti	(1) Se cota inicial definida
cti	Cota inicial (m);

```
loads.windloads.InitWindEccentricity(iwindcase)
```

Inicializa excentricidades de vento e outros dados para o caso fornecido

iwindcase Número do caso de vento 0..numcaswind-1

```
loads.windloads.GetWindEccentricityNumber(iwindcase)
```

Retorna o número de excentricidades de vento associadas a um caso

iwindcase Número do caso de vento 0..numcaswind-1

Retorna:

numwindecc	Número de excentricidades do caso iwindcase
------------	---

```
loads.windloads.AddWindEccentricity (iwindcase, ipisoi, ipisof,
ipil, excven, alrgpis, forven, s2b, s2fr, s2p, forlat, amomz)
```

Adiciona excentricidade e outros dados no caso de vento fornecido

iwindcase	Número do caso de vento 0..numcaswind-1
ipisoi	Piso inicial (-1) o primeiro
ipisof	Piso final (-1) o último
ipil	Número do pilar (-1) distriui em todos
excven	Excentricidade porcentual %
alrgpis	Largura imposta do piso (m)
forven	Força total de vento no piso, tf
s2b	Parâmetro meteorológico
s2fr	Fr- Fator de rajada
s2p	p - Expoente da lei potencial S2
forlat	Força lateral de vento (tf)
amomz	Momento torsor de vento (tfm)

```
loads.windloads.GetWindEccentricity (iwindcase, iwindexc)
```

Retorna a excentricidade e outros dados no caso de vento fornecido

iwindcase	Número do caso de vento 0..numcaswind-1
iwindexc	Número da excentricidade 0..GetWindEccentricityNumber()-1

Retorna:

ipisoi	Piso inicial (-1) o primeiro
ipisof	Piso final (-1) o último
ipil	Número do pilar (-1) distriui em todos
excven	Excentricidade porcentual %
alrgpis	Largura imposta do piso (m)
forven	Força total de vento no piso, tf
s2b	Parâmetro meteorológico
s2fr	Fr- Fator de rajada
s2p	p - Expoente da lei potencial S2
forlat	Força lateral de vento (tf)
amomz	Momento torsor de vento (tfm)

```
loads.windloads.windInPlanFloor
```

(1) Se caso de vento na planta de formas e grelha

```
loads.windloads.windTunnelGlobalSystem
```

Túnel de vento: (1) Se sistema global

```
loads.windloads.windTunnelInvertSignal
```

Túnel de vento: (1) Se inverter sinal

```
loads.windloads.windTunnelIgnoreColumns
```

Túnel de vento: Número de colunas a desprezar

Error! Use the Home tab to apply Título 1 to the text that you want to appear here.111

`loads.windloads.windTunnelIgnoreLines`

Túnel de vento: Número de linhas a desprezar

`loads.windloads.windTunnelTopToBottom`

Túnel de vento: (1) Se de cima para baixo

`loads.windloads.windTunnelFirstFloor`

Túnel de vento: Primeiro piso da lista ou (-1)

`loads.windloads.windTunnelForcesUnits`

Túnel de vento: (0) tf e tfm (1) se kN e kNm

`loads.windloads.windTunnelGlobalSystem`

Túnel de vento: (0) Forças no sistema local (1) sistema global.

Somente aplicável no momento da leitura da tabela de forças.

`loads.windloads.windTunnelInvertExcc`

Túnel de vento: (1) Excentricidades armazenadas com sinal invertido

`loads.windloads.aciBasicWindSpeed`

ACI-318 Velocidade básica de vento (m/s)

`loads.windloads.aciNaturalFrequency`

ACI-318 Frequência natural Hz (`aciStructureStiffness()` != 0)

`loads.windloads.aciDampingRate`

ACI-318 Taxa amortecimento % valor crítico (`aciStructureStiffness()` != 0)

`loads.windloads.aciStructureStiffness`

ACI-318 (0) Edifício rígido (1) flexível

`loads.windloads.aciTopographicFactor`

ACI-318 Fator topográfico Kzt

`loads.windloads.aciMinimumWindPressure`

ACI-318 Pressão mínima de vento, tf/m²

`loads.windloads.aciOccupancyCategory`

ACI-318 Natureza de ocupação (0)I (1)II (2)III (3)IV

`loads.windloads.aciExposureCategory`

ACI-318 Categoria de exposição (0)A (1)B (2)C (3)D

8.9.2. Cargas em etapas construtivas pré-moldadas

As cargas atribuídas nas fases construtivas das estruturas pré-moldadas são definidas pelo objeto `TQSBuild.loads.precastloads`.

`loads.precastloads.fracSelfLoadConsStage`

Fração % da carga de peso próprio engastado nas etapas construtivas

`loads.precastloads.fracDeadLoadConsStage`

Fração % da carga permanente nas etapas construtivas

`loads.precastloads.fracLiveLoadConsStage`

Fração % de sobrecargas nas etapas construtivas

`loads.precastloads.fracWindConsStage`

Fração % de carga de vento nas etapas construtivas

`loads.precastloads.fracEarthPressureStage`

Fração % de carga de empuxo de terra nas etapas construtivas

`loads.precastloads.fracTemperatureStage`

Fração % de carga de temperatura nas etapas construtivas

`loads.precastloads.fracShrinkageStage`

Fração % de carga de empuxo de terra nas etapas construtivas

`loads.precastloads.outOfPlumbNumber`

Número de casos de desaprumo

`loads.precastloads.outOfPlumbInverseAngle`

Inverso do ângulo de desaprumo

8.10. Sincronização com o GerPrE

A sincronização de arquivos com o sistema GerPrE para Gerenciamento de Produção de Estruturas é feita pelo subobjeto `TQSBuild.management`.

`management.gerpreActive`

(1) Se controle e transferência para o GerPrE ativada

`management.checkVariableRebar`

(1) Verificar a existência de ferros corridos e variáveis

`management.pdfRequired`

(1) PDF de plantas requerido

Error! Use the Home tab to apply Título 1 to the text that you want to appear here.113

management.**dxfRequired**

(1) DWF de plantas requerido

management.**dxgRequired**

(1) DXF de plantas requerido

9. Módulo TQSGrid

O TQSGrid é uma ferramenta para confecção de tabelas (ou planilhas) em desenho DWG. Ele permite que o programador crie tabelas formatando células com atributos, em vez de calcular coordenadas e separar linhas e colunas. A tabela abaixo é gerada pelo programa TSTGRID.py distribuído como exemplo.

Aço	Pos	Bit	Quant	Comprimento	
				Unit	Total
V1					
50A	1	12.50	3	590	1770
50A	2	12.50	3	455	1365
60B	3	5.00	12	363	4356
50A	4	6.30	5	364	1820
50A	5	6.30	16	427	6832
V2					
50A	1	12.50	3	590	1770
50A	2	12.50	3	455	1365
60B	3	5.00	12	363	4356
50A	4	6.30	5	364	1820
50A	5	6.30	16	427	6832
				Total	32286

Este módulo permite também que as tabelas sejam salvas em formato HTML para visualização em browser, e em formato LST com delimitador, que pode ser importado por planilhas.

9.1. Modo de funcionamento

O elemento básico da planilha são as células; elas são distribuídas em linhas e colunas numeradas a partir de zero. As células recebem atributos, que podem ser definidos por célula, coluna, linha ou para a tabela inteira. Os atributos assumidos por uma célula dependem donde foram definidos atributos, com a pesquisa também nesta ordem. Você pode misturar a forma de definir atributos, de maneira que o programa assumirá aquele que achar primeiro.

Você cria uma tabela com uma chamada como:

```
from TQS import TQSGrid
griddwg = TQSGrid.GridDwg ()
```

Depois dimensiona o número de linhas e colunas, e cria atributos para as células, como em:

```
griddwg.format.columns= numcolunas
griddwg.format.lines  = numlinhas
atrib = TQSGrid.GridDwgAttrib (griddwg)
griddwg.attrib.SetAttribGrid (atrib)
```

Error! Use the Home tab to apply Título 1 to the text that you want to appear here.115

O preenchimento das células será feito com chamadas como:

```
griddwg.format.SetCell (0, 1, "Pos")
```

E finalmente você salva o DWG ou outros formatos:

```
griddwg.file.SaveAsDwg (nomdwg)  
griddwg.file.SaveAsLst (nomlst)  
griddwg.file.SaveAsHtml (nomhtm)  
griddwg.file.SaveAsPdf (nompdf)
```

9.2. Atributos de uma célula

Os atributos de uma célula são definidos por um objeto tipo GridDwgAttrib:

```
atrib = TQSGrid.GridDwgAttrib (griddwg)
```

TQSGrid.GridDwgAttrib (griddwg)

Cria um objeto tipo GridDwgAttrib, para ser usado como atributo de tabela

Retorna: o objeto GridDwgAttrib criado.

level

Nível de desenho

hText

Altura de texto, cm. A escala padrão do desenho é criado é 1:1.

iHorizontalAlignment

Alinhamento de texto GridDwgAttrib.ALIN_Hxxxx, definidos a seguir:

GridDwgAttrib.**ALIN_HINVAL**

Alinhamento inválido

GridDwgAttrib.**ALIN_HESQ**

Alinhamento horizontal à esquerda

GridDwgAttrib.**ALIN_HCEN**

Alinhamento horizontal central

GridDwgAttrib.**ALIN_HDIR**

Alinhamento horizontal à direita

iVerticalAlignment

Alinhamento de texto GridDwgAttrib.ALIN_Vxxxx, definidos a seguir:

`GridDwgAttrib.ALIN_VINVAL`
Alinhamento inválido

`GridDwgAttrib.ALIN_VINF`
Alinhamento vertical inferior

`GridDwgAttrib.ALIN_VCEN`
Alinhamento vertical central

`GridDwgAttrib.ALIN_VSUP`
Alinhamento vertical superior

cellHeight

Altura da célula, cm

cellWidth

Largura da célula, cm

cellFormat

Formato da célula, na convenção da linguagem C, família de rotinas tipo printf. Tem que bater o tipo usado. Ex: "%7.2f" para um número real, "%05d" para um número inteiro. Se não definido, um padrão será usado. A definição de um formato inválido poderá causar invasão de memória.

9.3. Atributos das células com TQSGrid.attrib

Atributos podem ser associados às células, colunas, linhas ou tabela inteira, sendo que o programa usa por célula o primeiro atributo que achar, nesta ordem. Os atributos são fornecidos como um objeto `GridDwgAttrib`.

`attrib.SetAttribGrid (attrgrid)`

Define os atributos gerais do grid

`attrib.SetAttribLine (line, attrgrid)`

Define os atributos da linha com `line= 0..lines-1`

`attrib.SetAttribColumn (column, attrgrid)`

Define os atributos da coluna com `column= 0..columns-1`

`attrib.SetAttribCell (line, column, attrgrid)`

Define os atributos da célula com a linha `line= 0..lines-1` e a coluna `column= 0..columns-1`

9.4. Formatação e preenchimento com TQSGrid.Format

Estas rotinas definem o tamanho do grid e permitem mesclar linhas e colunas, assim como definir o valor das células.

9.4.1. Propriedades de dimensionamento da tabela

Podem ser lidos ou gravados, definem o tamanho da tabela.

`format.columns`

Número de colunas do grid.

`format.lines`

Número de linhas do grid.

9.4.2. Métodos de formatação e preenchimento

São as rotinas que permitem juntar colunas dentro de linhas, ou linhas dentro de colunas, e o de definição de valores de células.

`format.MergeLine(line, startColumn, endColumn)`

Mistura as colunas `startColumn` a `endColumn` inclusive da linha `line`.

As linhas misturadas não têm linha divisória.

`format.MergeColumn(column, startLine, endLine)`

Mistura as linhas `startLine` a `endLine` inclusive, da coluna `column`.

As colunas misturadas não têm linha divisória.

`format.SetCell(line, column, val)`

Valor de uma célula. Pode ser inteiro, real ou texto.

9.5. Salvamento do grid

O grid gerado pode ser salvo em 4 formatos.

`file.SaveAsDwg (nomdwg)`

Salva o grid em formato DWG TQS

`file.SaveAsLst (nomlst)`

Salva o arquivo em formato delimitado por vírgulas, que pode ser importado pela planilha Excel®.

`file.SaveAsHtml (nomhtm)`

Formato HTM, que pode ser lidos por browsers

`file.SaveAsPdf (nompdf)`

Salvamento em formato PDF. O sistema de plotagem é usado para fazer este salvamento, utilizando os parâmetros de plotagem e tabela de plotagem correspondente. Por isto, é possível que uma assinatura com o nome do cliente seja gerada do lado esquerdo do desenho, e um número de revisão seja somado ao nome do arquivo.

9.6. Programa TSTGRID.py para teste do TQSGrid

Este programa gera o desenho mostrado no início do capítulo. Ele gera 3 arquivos de saída:

TSTDWGGRID.DWG	Tabela em DWG, para leitura pelos editores EAG.
TSTDWGGRID.HTM	Tabela em HTML, para leitura por browsers.
TSTDWGGRID.LST	Tabela em listagem, delimitada, para leitura por planilhas.

Note que atualmente os browsers têm dificuldade para interpretar tabelas com mesclagem de linhas ou colunas. Para ver corretamente o arquivo HTML em um browser, é necessário tirar toda a mesclagem dos programas. Alternativamente o programa NTQSHTM.EXE distribuído com o TQS visualiza corretamente este arquivo.

10. Execução do TQS com o módulo TQSExec

O módulo `TQSExec` permite a execução do gerenciador TQS e alguns de seus comandos, como o processamento global. É possível montar uma fila de edifícios a processar e processá-los forma automática.

10.1. Modo de funcionamento

A execução de comandos no gerenciador é feita pelo objeto `TQSExec.Job`. Você entra tarefas tipo `TQSExec.Task` na fila, e no final o comando `Job.Execute` chama o gerenciador e executa os comandos.

10.2. Definição de tarefas

As tarefas a serem realizadas são objetos derivados da classe `TQSExec.Task`. Cada objeto derivado define uma tarefa diferente. A tarefa é definida na construção do objeto.

10.2.1. Pasta atual e a classe TaskFolder

Alguns comandos são realizados a partir de uma pasta determinada. Use este objeto para posicionar o gerenciador na pasta desejada.

```
TQSExec.TaskFolder (buildingName, folderType=FOLDER_FRAMES,  
                    folderName="", subFolder=SUBFOLDER_NONE)
```

Define a pasta atual do edifício

`buildingName` Nome do edifício

`folderType` Pasta tipo `FOLDER_xxx`

`folderName` Nome da pasta (string) para `FOLDER_FLOORS`

`subFolder` Subpasta tipo `SUBFOLDER_xxxx` para `FOLDER_FLOORS`

As constantes que podem ser usadas em `folderType` são:

`TaskFolder.FOLDER_FRAMES`

Pasta Espacial

`TaskFolder.FOLDER_COLUMNS`

Pasta Pilares

`TaskFolder.FOLDER_FOUNDATIONS`

Pasta Fundacoes

`TaskFolder.FOLDER_FLOORS`

Pasta do pavimento

`TaskFolder.FOLDER_GENERAL`

Pasta Gerais

`TaskFolder.FOLDER_INFRASTRUCTURE`

Pasta Infraestrutura

As constantes que podem ser usadas em `subFolder` são:

`TaskFolder.SUBFOLDER_NONE`

Pasta do pavimento

`TaskFolder.SUBFOLDER_BEAMS`

Subpasta vigas, quando `folderName = TaskFolder.FOLDER_FLOORS`

`TaskFolder.SUBFOLDER_STAIRS`

Subpasta escadas, quando `folderName = TaskFolder.FOLDER_FLOORS`

10.2.2. Pasta raiz do edifício

`TQSExec.TaskRootFolder(rootfolder)`

Define a pasta raiz de edifícios

10.2.3. Processamento global

Esta é a principal tarefa definida, com processamento global de toda a estrutura. O preenchimento de dados é semelhante à janela de dados do gerenciador.

```
TQSExec.TaskGlobalProc(floorPlan=2, floorDraw=0, slabs=0,
                        beams=1, columnsData=1, columns=0, columnsReport=1,
                        gridModel=1, gridDraw=1, gridExtr=1, gridAnalysis=1,
                        gridBeamsTrnsf=0, gridSlabsTrnsf=1, gridNonLinear=0,
                        frameModel=1, frameAnalysis=1, frameBeamsTrnsf=1,
                        frameColumnsTrnsf=1, foundations=0, stairs=0,
                        fire=0, precastPhases=0)
```

Processamento global conforme parâmetros escolhidos

<code>FloorPlan</code>	(0) Não (1) Extrair plantas (2) Extrair e processar
<code>floorDraw</code>	(0) Não (1) Desenhar formas
<code>slabs</code>	(0) Não (1) Esforços (2) Esforços e desenho
<code>beams</code>	(0) Não (1) Esforços (2) Dimensionamento, detalh (3) E desenho
<code>columnsData</code>	(0) Não (1) gravação de dados de pilares
<code>columns</code>	(0) Não (1) Dimensionamento, detalhamento (2) E desenho
<code>columnsReport</code>	(0) Não (1) Relatório geral de pilares
<code>gridModel</code>	(0) Não (1) Geração do modelo
<code>gridDraw</code>	(0) Não (1) Desenho de dados de grelha

Error! Use the Home tab to apply Título 1 to the text that you want to appear here.121

gridExtr (0) Não (1) Extração do desenho
gridAnalysis (0) Não (1) Análise de esforços
gridBeamsTrnsf (0) Não (1) Transferência de esforços para vigas
gridSlabsTrnsf (0) Não (1) Transferência de esforços para lajes
gridNonLinear (0) Não (1) Análise não linear
frameModel (0) Não (1) Geração do modelo
frameAnalysis (0) Não (1) Análise de esforços
frameBeamsTrnsf (0) Não (1) Transferência de esforços para vigas
frameColumnsTrnsf (0) Não (1) Transferência de esforços para pilares
foundations (0) Não (1) Dimensionamento, detalhamento (2) E desenho
stairs (0) Não (1) Dimensionamento, detalhamento e desenho
fire (0) Não (1) Verificação à incêndio
precastPhases (0) Não (1) Pré-moldados: Todas as etapas construtivas

10.2.4. Planta de cargas

Entre este objeto na lista de tarefas para gerar a planta de cargas

TQSExec.**TaskLoadsReport** ()

Planta de cargas com as últimas combinações fornecidas

10.2.5. Desenho de lajes

Além dos desenhos de flexão positiva e negativa, punção e cisalhamento, este comando permite também gerar as faixas de esforços correspondentes.

TQSExec.**TaskSlabDrawings**(rebartop=1, rebarbot=1, rebarpunch=1,
rebarshear=1, forcestop=1, forcesbot=1,
forcespunch=1, forcesshear=1)

Desenhos de lajes

rebartop (0) Não (1) Ferros positivos
rebarbot (0) Não (1) Ferros negativos
rebarpunch (0) Não (1) Ferros de punção
rebarshear (0) Não (1) Ferros de cisalhamento
forcestop (0) Não (1) Faixas positivos
forcesbot (0) Não (1) Faixas negativos
forcespunch (0) Não (1) Faixas de punção
forcesshear (0) Não (1) Faixas de cisalhamento

10.2.6. Desenho de lajes treliçadas

Geração dos 4 desenhos típicos no pós-processamento de lajes treliçadas. Exige que as armaduras tenham sido previamente calculadas.

```
TQSExec.TaskLatticeSlabs(latticeList=1, latticeBeams=1,
                           rebarList=1, fillingList=1)
```

Desenhos de lajes treliçadas

<code>latticeList</code>	(0) Não (1) Tabela de vigotas treliçadas
<code>latticeBeams</code>	(0) Não (1) Planta de fabricação de vigotas
<code>rebarList</code>	(0) Não (1) Tabela de ferros complementares
<code>fillingList</code>	(0) Não (1) Tabela de enchimentos

10.2.7. Desenho de lajes mistas nervuradas

Defina primeiro a pasta antes de acionar o desenho de lajes mistas nervuradas.

```
TQSExec.TaskCompositeSlabs()
```

Desenhos de lajes mistas nervuradas

10.2.8. Tabela de ferros com todos os desenhos de armação

Este comando não existe diretamente no gerenciador. Ele cria uma planta na pasta GERAIS com todos os desenhos de armação de todas as pastas do edifício, e gera uma tabela de ferros única. Esta tabela é interessante para avaliação da quantidade total de aço no edifício e para comparações de quantitativos.

```
TQSExec.TaskRebarSchedule()
```

Tabela de ferros com todos os desenhos de armadura do edifício

10.2.9. Processamento de escadas

Caso o dimensionamento, detalhamento e desenho de escadas não tenha sido feito no processamento global, pode ser feito aqui. O processamento é de um único piso, que precisa ser definido antes na pasta de ESCADAS.

```
TQSExec.TaskStairs()
```

Dimensionamento, detalhamento e desenho de escadas

10.2.10. Desenho de formas

Gera os desenhos típicos de formas de um pavimento. A pasta atual precisa ser definida previamente, a menos dos cortes do edifício, que são sempre gerados na pasta espacial.

```
TQSExec.TaskFloorPlanDrawings(floor=1, columns=1, slabs=1,
                                slabsDims=1, beams=1, loads=1, sections=1)
```

Desenho de formas

<code>floor</code>	(0) Não (1) Planta de formas
<code>columns</code>	(0) Não (1) Planta de pilares
<code>slabs</code>	(0) Não (1) Verificação de nós de lajes

Error! Use the Home tab to apply Título 1 to the text that you want to appear here.123

slabsDims	(0) Não (1) Verificação de medidas de lajes
beams	(0) Não (1) Nós e cargas em vigas
loads	(0) Não (1) Cargas em lajes
sections	(0) Não (1) Cortes do edifício

10.2.11. Desenhos de pré-moldados

Adicione esta tarefa para gerar os desenhos de formas de pré-moldados

TQSExec.TaskPreCastDrawings ()

Desenho de pré-moldados

10.2.12. Transferência de esforços para vigas pré-moldadas

TQSExec.TaskPreCastBeams ()

Transferência de esforços para cálculo de vigas pré-moldadas

10.2.13. Transferência de esforços para fundações pré-moldadas

TQSExec.TaskPreCastFoundations ()

Transferência de esforços para cálculo de fundações pré-moldadas

10.2.14. Dimensionamento, detalhamento e desenho de pilares pré-moldados

TQSExec.TaskPreCastColumns ()

Dimensionamento, detalhamento e desenho de pilares pré-moldados

10.2.15. Dimensionamento, detalhamento e desenho de consolos pré-moldados

TQSExec.TaskPreCastCorbels ()

Dimensionamento, detalhamento e desenho de consolos pré-moldados

10.2.16. Quantitativos de pré-moldados

TQSExec.TaskPreCastMaterials ()

Quantitativos de pré-moldados

10.2.17. Resumo Estrutural

`TQSExec.TaskStructuralReport()`

Realiza o resumo estrutural e chama o resultado em HTML na tela

10.3. Execução de tarefas – classe Job

A objetos da classe `Job()` acumulam tarefas, que são objetos derivados da classe `Task()`, e são executados. Cada execução carrega o gerenciador no ar, processa e depois sai do gerenciador. Você cria o objeto com:

```
Job = TQSExec.Job()
```

10.3.1. Entrada de uma tarefa

As tarefas são colocadas em uma fila e executadas na mesma sequência de entrada.

```
job.EnterTask(task)
```

Acumula uma tarefa da classe `Task` para execução

10.3.2. Execução das tarefas

Acione o comando `job.Execute()` para executar todas as tarefas.

```
job.Execute()
```

Executa os comandos acumulados, chama o TQS e sai no final

10.4. Programa de teste TSTEXEC.py para a classe TQSExec

Este programa usa como exemplo o edifício “Mod-Padrão” distribuído para teste do programa TQS. Se não estiver na sua árvore de edifícios, primeiro faça a instalação. Descompacte o modelo MODPLA.TQS, localizado na pasta USUARIO\NGE\TESTE da instalação TQS.

O programa duplica este edifício, alterando o fck (C25, C30, C35 e C40) e o método de cálculo (Modelo IV e VI), no total de 8 combinações. Cada edifício duplicado recebe um nome distinto, identificando fck e modelo estrutural.

O edifício é carregado através de um objeto `TQSBuild.Building`:

```
Building = TQSBuild.Building ()  
istat    = building.file.Open (nomedi)
```

Depois, é duplicado com alterações no nome, concreto e modelo estrutural:

```
nomediteste = nomedi + "_" + concrete + "_" + modeldesc  
building.model.structuralModel = structuralModel  
building.materials.beamConcreteStrength      = concrete  
building.materials.columnConcreteStrength    = concrete  
building.materials.foundationConcreteStrength= concrete  
building.file.SaveAs (nomediteste)
```

Finalmente é feito o processamento global, incluindo uma tabela de ferros na pasta GERAIS:

```
job                = TQSExec.Job ()  
job.EnterTask(TQSExec.TaskFolder(nomediteste,  
                                   TQSExec.TaskFolder.FOLDER_FRAMES))  
job.EnterTask(TQSExec.TaskGlobalProc(floorDraw=1,slabs=2,  
                                       beams=3, columns=2, foundations=1, stairs=1))  
job.EnterTask (TQSExec.TaskRebarSchedule ())  
job.Execute      ()
```

11. Módulo `TQSModel` e o Modelo Estrutural

O modelo estrutural, criado através do Modelador em modo interativo, pode ser totalmente lido e gravado pelo módulo `TQSModel`. Existem inúmeras possibilidades para uso do `TQSModel`, entre criar um modelo novo, modificar um modelo existente e extrair dados de modelos. Podemos testar várias soluções estruturais diferentes de um único edifício, com geração e processamento automático, combinando diferentes critérios, dimensões e cargas. Basta combinar este módulo com os módulos `TQSBuild` e `TQSExec`.

Este módulo é maior que os demais vistos. Os diversos elementos estruturais do Modelador são representados por objetos. Considerando apenas 15 objetos do Modelador definidos inicialmente (de um total de 54), foram definidas cerca de 100 classes e 1200 funções.

11.1. Modelo Estrutural

Fica mais fácil entender o funcionamento do `TQSModel`, se entendermos como são relacionados os dados do edifício e os diversos objetos do Modelo Estrutural. É ideal que o programador tenha experiência no uso do TQS e do Modelador Estrutural. Como sempre, um ótimo meio de iniciar o uso deste módulo é examinando e copiando partes dos exemplos distribuídos (`TSTMODELn.PY`).

A forma de criar e alterar objetos através do `TQSModel` reflete o funcionamento do TQS – por exemplo, a planta atual e a criação de elementos estruturais usando *dados atuais*. Muitas estruturas de dados correspondem aproximadamente à abas de edição de dados de elementos estruturais.

11.1.1. Dados do Edifício e Modelo Estrutural

No TQS, o modelo estrutural completo é dividido em duas partes: uma chamada de *Dados do Edifício* e outra *Modelo Estrutural*. A separação não tem a ver com o modelo em si, mas como o TQS foi desenvolvido ao longo do tempo.

Os Dados do Edifício contém a definição dos pavimentos e suas cotas, casos de carregamento, materiais, cobrimentos e critérios. O modelo estrutural contém os objetos estruturais como vigas, pilares, lajes, fundações, cargas e outros elementos, por planta, assim como pilares e elementos inclinados entre plantas diferentes. Antes da criação ou edição de um modelo estrutural, é necessário definir os dados do edifício. Isto é feito interativamente ou pelo módulo `TQSBuild` mostrado neste manual. Desde a V24, é possível editar os dados do edifício dentro do Modelador. Esta função não está disponível no módulo `TQSModel`.

Ao criar os dados do edifício, é criado um conjunto de pastas para trabalho. O Modelador precisa ter a pasta atual como uma das pastas de trabalho do edifício. Estas pastas são chamadas de *Contexto do Edifício*.

11.1.2. Organização de dados do Modelo Estrutural

Os dados estão distribuídos por classes, que são instanciadas em objetos. Os objetos são o que se espera do modelo: vigas, pilares, lajes, fundações, etc. Os objetos são armazenados em listas de dois tipos:

Lista	Conteúdo
Pavimentos	Uma lista de objetos por pavimento. Vigas, lajes, contornos, cargas, cotagens, dados por pavimento, etc
Lista espacial	Objetos que passam por mais de uma planta – pilares, elementos inclinados, dados globais.

Para facilitar o acesso, certos objetos espaciais aparecem na lista de objetos do pavimento, conforme o contexto. Por exemplo, se estamos trabalhando com o pavimento térreo e um pilar passa por ele, você pode acessar os dados do pilar neste pavimento. Se ele morre antes do térreo, não aparecerá nesta lista.

11.1.3. Criando elementos novos e dados atuais

Seguindo a lógica do Modelador Estrutural, a criação de um elemento estrutural tem poucos dados. A maioria deles é pré-definida em *Dados Atuais*. Por exemplo, os dados atuais de vigas:

Temos então duas maneiras de definir elementos estruturais:

- Preenchemos os dados atuais e criamos o elemento. Esta é a maneira de criar elementos no Modelador;
- ou, criamos um elemento sem verificar os dados atuais, e depois modificamos o elemento criado.

Na criação de cada elemento, mostraremos onde estão os dados atuais usados na criação, e as variáveis de acesso correspondentes no elemento criado.

11.2. Objeto TQSModel.Model

Este objeto representa todo o modelo estrutural, e todos os demais objetos serão acessados começando através dele. O acesso a um modelo estrutural é sempre sobre um edifício existente, com *dados do edifício* já criados. A pasta atual para abrir um modelo necessariamente é uma das pastas do edifício.

O módulo TQSBuild tem uma função para entrar na pasta raiz do edifício. Para abrir um modelo existente de nome TESTE, podemos fazer:

```
from TQS import TQSBuild, TQSModel
....
    build          = TQSBuild.Building ()
    istat          = build.RootFolder ("TESTE")
    if             istat == 0:
        model      = TQSModel.Model ()
```

Neste exemplo, criamos um objeto tipo TQSBuild.Building e entramos na pasta raiz do edifício TESTE (ex: C:\TQS\TESTE). Então, abrimos o modelo estrutural. O modelo tem 3 subobjetos:

Objeto	Uso
file	Leitura e salvamento do modelo
type	Lista os tipos de objeto do Modelador
floors	Acesso aos pavimentos e objetos FloorsList
current	Dados atuais de uso global CurGlobalData (parte dos dados atuais é definida também por pavimento)
visData	Modos de visualização do Modelador VisData

11.2.1. Leitura e gravação de modelos com o Model.file

O nome do edifício não é fornecido com as funções de leitura e salvamento. Ele é deduzido pela pasta atual do edifício, ou contexto do edifício.

```
file.OpenModel ()
```

Abre modelo existente ou cria um novo
Retorna: istat!=0 Se erro

```
file.OpenNewModel ()
```

Abre um modelo novo, apaga o velho se já existir
Retorna: istat!=0 Se erro

Error! Use the Home tab to apply Título 1 to the text that you want to appear here.129

`file.Save ()`

Salva o modelo estrutural

Retorna: `istat!=0` Se erro

`file.Close ()`

Fecha modelo estrutural aberto

11.2.2. Lista de tipos de objeto com `Model.type`

O `Model.type` lista os tipos de objeto do Modelador. Os tipos são numerados sequencialmente. Nem todos os tipos tem tratamento previsto dentro do `TQSModel`.

`type.GetNumTypes ()`

Retorna o número de tipos de objetos do Modelador

`type.GetTypeClass (itype)`

Retorna o nome de classe de um tipo de objeto (classe C++, não do `TQSModel`)

`itype` `TYPE_XXXX`

Retorna

`Nomeclasse` (string) Nome da classe

`type.GetTypeDescription (itype)`

Retorna a descrição de um tipo de objeto

`itype` `TYPE_XXXX`

Retorna

`Nomeobjeto` (string) Nome do objeto

Os tipos descritos pelas constantes `TYPE_XXX` são os seguintes:

Constante	Tipo
<code>TQSModel.TYPE_VIGAS</code>	Vigas
<code>TQSModel.TYPE_PILARES</code>	Pilares
<code>TQSModel.TYPE_LAJES</code>	Lajes
<code>TQSModel.TYPE_FUNDAC</code>	Fundações
<code>TQSModel.TYPE_FUROS</code>	Furos
<code>TQSModel.TYPE_CAPITEIS</code>	Capiteis
<code>TQSModel.TYPE_FORNER</code>	Formas de lajes nervuradas
<code>TQSModel.TYPE_CTRAUX</code>	Contorno auxiliar
<code>TQSModel.TYPE_EIXOS</code>	Eixos rotulados
<code>TQSModel.TYPE_COTAGENS</code>	Cotagens
<code>TQSModel.TYPE_CORTES</code>	Cortes
<code>TQSModel.TYPE_DADPAV</code>	Dados comuns do pavimento
<code>TQSModel.TYPE_DADEDI</code>	Dados comuns do edifício
<code>TQSModel.TYPE_SECPIL</code>	Seções de pilares
<code>TQSModel.TYPE_LISMAL</code>	Lista de malhas de lajes
<code>TQSModel.TYPE_CARCON</code>	Cargas concentradas

<code>TQSModel.TYPE_CARLIN</code>	Cargas distribuídas lineares
<code>TQSModel.TYPE_CARARE</code>	Carga distribuída por área
<code>TQSModel.TYPE_BARICEN</code>	Tabela de baricentros de pilares
<code>TQSModel.TYPE_CARADI</code>	Carga adicional em laje
<code>TQSModel.TYPE_RAMPAS</code>	Laje inclinada
<code>TQSModel.TYPE_ORIEIX</code>	Origem do sistema de eixos
<code>TQSModel.TYPE_TABNP</code>	Tabela de seções não padrão
<code>TQSModel.TYPE_IDCLI</code>	Identificação do cliente
<code>TQSModel.TYPE_CONSBIB</code>	Biblioteca de consolos
<code>TQSModel.TYPE_FUROVIG</code>	Furo em viga
<code>TQSModel.TYPE_ELMJLP</code>	Elemento de laje pré-moldada
<code>TQSModel.TYPE_TABELEM</code>	Tabela com lista de elementos
<code>TQSModel.TYPE_COTDIA</code>	Cotagem de raio/diâmetro
<code>TQSModel.TYPE_COTANG</code>	Cotagem angular
<code>TQSModel.TYPE_COTNOT</code>	Cotagem de notas
<code>TQSModel.TYPE_VENDST</code>	Tabela de distribuição de vento
<code>TQSModel.TYPE_CENTOR</code>	Centro de torção - túnel de vento
<code>TQSModel.TYPE_CONSIND</code>	Consolos Independentes
<code>TQSModel.TYPE_CAREMP</code>	Cargas de empuxo
<code>TQSModel.TYPE_TABNIVPAV</code>	Tabela de níveis de pavimentos
<code>TQSModel.TYPE_LEGDESNIV</code>	Tabela de legenda de desníveis
<code>TQSModel.TYPE_SIMDESNIV</code>	Símbolo de desnível de pavimento
<code>TQSModel.TYPE_ELM3D</code>	Elemento 3D
<code>TQSModel.TYPE_INSRBLO</code>	Biblioteca de insertos
<code>TQSModel.TYPE_INSRPOS</code>	Inserto
<code>TQSModel.TYPE_FACHADA</code>	Fachada de painel pré-moldado
<code>TQSModel.TYPE_TUBOS</code>	Tubos
<code>TQSModel.TYPE_FUROPIL</code>	Furo em pilar
<code>TQSModel.TYPE_LAJESSOVOL</code>	Lajes somente volume
<code>TQSModel.TYPE_ESTACRAD</code>	Estaca sob Radier
<code>TQSModel.TYPE_VIGASINC</code>	Viga inclinada
<code>TQSModel.TYPE_PILARETES</code>	Pilaretes
<code>TQSModel.TYPE_ESCADAS</code>	Escada
<code>TQSModel.TYPE_PATAMARES</code>	Patamar
<code>TQSModel.TYPE_ORIBARI</code>	Origem de baricentros
<code>TQSModel.TYPE_EDIEDI</code>	Dados do edifício
<code>TQSModel.TYPE_PTBAS</code>	Ponto base do projeto
<code>TQSModel.TYPE_PTLVT</code>	Ponto de levantamento topográfico
<code>TQSModel.TYPE_CERCATOR</code>	Cerca de torre (separa torre e periferia)

11.2.3. Acesso aos pavimentos via Model.floors

Permite o acesso aos pavimentos do edifício, seja por enumeração, seja por nome. Os índices utilizados para acesso se iniciam em (1).

Error! Use the Home tab to apply Título 1 to the text that you want to appear here.131

`floors.GetFloor (nompla)`

Define a planta atual e retorna um objeto `Floor()` de planta

`nompla` (string) Nome da planta

Retorna:

`floor` Objeto `Floor()`, com dados de uma planta

`floors.GetNumFloors()`

Retorna o número de plantas do edifício

`floors.GetFloorName (indpla)`

Nome de uma planta

`indpla` Índice da planta 1..`GetNumFloors()`

Retorna:

`nompla` Nome da planta

`floors.GetFloorIndex (nompla)`

Índice da planta

`nompla` Nome da planta

Retorna:

`indpla` Índice da planta 1..`GetNumFloors()` ou zero se não achar

11.2.4. Dados globais `Model.current`

Esta variável retorna um objeto da classe `CurGlobalData()`, que contém subobjetos com dados atuais, usados na criação de diversos tipos de objetos.

Subobjetos	Conteúdo
<code>current.globalAxisData</code>	Eixos
<code>current.globalCutData</code>	Cortes
<code>current.globalSoilpress</code>	Carga de empuxo
<code>current.globalRefxData</code>	Crítérios de referências externas
<code>current.globalBeamData</code>	Vigas
<code>current.globalSlabData</code>	Lajes
<code>current.globalColumnData</code>	Pilares, pilaretes e fundações
<code>current.globalColumnOpening</code>	Furos em pilares
<code>current.globalSlabFPile</code>	Estaca de laje radier
<code>current.globalPrecast</code>	Prémoldados
<code>current.globalBimData</code>	BIM

O detalhamento da classe de cada um deles será feito neste capítulo, junto a cada objeto onde são aplicáveis. Nem todos os dados atuais são armazenados globalmente; alguns são definidos por pavimento.

11.2.5. Modos de visualização Model.visData

Modos de visualização são definidos para o modelo na tela do Modelador e diversos desenhos de formas. O objeto que define os modos depende então de onde devem ser aplicados:

```
current.visData.GetVisModes(imode)
```

Objeto de modos de visualização VisModes()

imode Aplicar modos em VISMODE_XXXX

Retorna:

Vismodes Objeto VisModes() para controle de visualização

Os modos possíveis são:

Constante	Uso
TQSModel.VISMODE_MATUAL	Modo de visualização atual
TQSModel.VISMODE_FORMAS	Desenho de formas FORnnnn.DWG
TQSModel.VISMODE_VERIFI	Modo de verificação
TQSModel.VISMODE_MODELO	Desenho MODELO.DWG
TQSModel.VISMODE_GRUPOA	Grupo A de visualização do usuário
TQSModel.VISMODE_GRUPOB	Grupo B de visualização do usuário

O objeto VisMode() retornado será detalhado adiante.

11.3. Objeto do pavimento: Floor()

O Floor() é responsável pela criação de todos os objetos do Modelador, assim como a iteração por eles. As tarefas são divididas nos seguintes subobjetos:

Objeto	Uso
current	Dados atuais por pavimento, CurrentFloorData()
create	Cria objetos na planta. Dependendo do objeto, ele pode ir para a lista espacial.
iterator	Itera pelos objetos da planta
util	Utilidades diversas

11.3.1. Dados atuais por pavimento Floor.current

Os dados atuais por pavimento (floor.current), junto com os dados atuais globais, são usados na criação de elementos novos. A lógica é semelhante à criação de objetos novos dentro do Modelador. São eles:

Objeto	Uso
floorDrawingData	Dados de desenho

floorLoadData	Cargas usadas na planta
floorBeamData	Dados de vigas
floorSlabData	Dados de lajes
floorColumnData	Dados de pilares
floorCutData	Dados de cortes
floorAxisData	Dados de eixos

Detalharemos cada um destes objetos junto dos respectivos elementos onde se aplicam.

11.3.2. Criação de objetos com Floor.create

Este subobjeto de Floor permite a criação de todos os elementos estruturais. O objeto Floor mantém a lista de todos os objetos planos do pavimento, mas repassa a criação de pilares e objetos inclinados para uma lista espacial. Veja os exemplos TSTMODEL1.PY e TSTMODEL2.PY que criam novos modelos estruturais. A seguir, mostraremos as funções para criação de objetos.

O objeto de pilares é usado para representar 3 tipos de objetos estruturais: Pilares, Pilaretes e Fundações. O objeto efetivamente definido depende dos dados de pilares definidos.

```
create.CreateColumn(xins, yins)
```

Criação de um pilar

xins Ponto de inserção X

yins Ponto de inserção Y

Retorna:

Objeto tipo Column()

Por exemplo, considerando o modelo em model, o pavimento atual em floor e as coordenadas de inserção xins, yins, podemos construir um pilar retangular 20x50 cm assim:

```
columnData          = model.current.globalColumnData.columnData
columnData.columnStarts = TQSModel.COLUMNSTART_NASCEPIFU
columnGeometry      = columnData.columnGeometry
columnGeometry.sectionType = TQSModel.COLUMNTYPE_R
columnGeometry.sectionBl = 20.
columnGeometry.sectionHl = 50.
column              = floor.create.CreateColumn (xins, yins)
```

A variável columnData contém dados atuais para a criação de pilares. Na criação de pilaretes e de fundações, usamos as variáveis shortColumnData e foundationData no lugar de columnData. Ainda no exemplo acima, a variável columnStarts recebeu uma constante que diz que o pilar nasce na fundação, e a variável sectionType, o formato do pilar retangular.

Nos pilares, assim como nos demais objetos do Modelador, detalharemos os dados atuais e os dados de cada objeto junto da definição de cada um deles.

A definição de pilaretes e fundações é semelhante, e gera um objeto armazenado na estrutura de dados de pilares:

```
create.CreateShortColumn(xins, yins)
```

Criação de um pilarete

xins Ponto de inserção X

yins Ponto de inserção Y

Retorna:

Objeto tipo `Column()`

```
create.CreateFoundation(xins, yins)
```

Criação de uma fundação

xins Ponto de inserção X

yins Ponto de inserção Y

Retorna:

Objeto tipo `Column()`

Os demais objetos que podem ser criados são:

```
create.CreateBeam(xy)
```

Cria viga no plano

xy ((x1,y1),(x2,y2)...) coordenadas da viga

Retorna:

Objeto tipo `Beam()`

```
create.CreateBeamOpening(beam, xins, yins)
```

Cria um furo horizontal em viga

beam Objeto da viga

xins,yins Centro do furo em planta no eixo da viga

Retorna:

Objeto tipo `BeamOpening()`

```
create.CreateSlabContour(x1, y1, x2, y2)
```

Cria contorno auxiliar de laje - bordo livre

x1,y1,x2,y2 Coordenadas da linha cm

Retorna:

Objeto tipo `SlabContour()`

Error! Use the Home tab to apply Título 1 to the text that you want to appear here.135

`create.CreateSlab(xins, yins, angpri)`

Cria laje dentro de contorno formado por vigas, pilares ou contorno auxiliar

xins Ponto X dentro da laje

yins Ponto Y dentro da laje

angpri Ângulo principal, graus

Retorna:

Objeto tipo `Slab()`

`create.CreateDropPanel(polig)`

Cria capitel

polig Poligonal `Polygon()` fechada que define o capitel

Retorna:

Objeto tipo `DropPanel()`

`create.CreateSlabMould(slab, xnerv, ynerv)`

Cria uma forma/cubeta de laje nervurada

slab Laje `Slab()` onde vai a forma

xnerv Posição do centro da forma

ynerv Posição do centro da forma

Retorna:

Objeto tipo `SlabMould()`

`create.CreateSlabOpening(polig, irecorte, isobrecapa)`

Cria capitel

polig Poligonal fechada que define o furo em planta

irecorte (0) Furo (respeita nervuras) (1) Recorte

isobrecapa (0) Normal (1) Só na capa de laje nervurada

Retorna:

Objeto tipo `SlabOpening()`

`create.CreateSoilPressureLoad(xini, yini, xfin, yfin)`

Cria uma carga de empuxo

xini Ponto inicial X

yini Ponto inicial Y

xfin Ponto final X

yfin Ponto final X

Retorna:

Objeto tipo `SoilPressureLoad()`

`create.CreateAdditionalLoad(xins, yins)`

Cria carga adicional em laje

xins Ponto X na laje com o símbolo da carga

yins Ponto Y

Retorna:

Objeto tipo `AdditionalLoad()`

`create.CreateAreaDistributedLoad(polig)`

Cria distribuída em laje dentro de uma poligonal fechada tf/m2

Polig Poligonal Polygon() onde a área é aplicada

Retorna:

Objeto tipo AreaDistributedLoad()

`create.CreateConcentratedLoad(xins, yins)`

Cria carga concentrada tf

xins Ponto X na laje com o símbolo da carga

yins Ponto Y

Retorna:

Objeto tipo ConcentratedLoad()

`create.CreateLinearyDistributedLoad(xins1, yins1, xins2, yins2)`

Cria linear tf/m sobre viga ou laje

xins1 Ponto inicial X na laje com o símbolo da carga

yins1 Ponto inicial Y

xins2 Ponto final X na laje com o símbolo da carga

yins2 Ponto final Y

Retorna:

Objeto tipo LinearyDistributedLoad()

11.3.3. Leitura do pavimento com Floor.iterator

O subobjetos iterator são do tipo SMOBJECT e permitem a leitura de todos os objetos de um pavimento. Você pode listar e editar os objetos lidos.

`iterator.GetNumObjects(itype)`

Retorna o número de objetos de uma lista de uma planta

itype TQSModel.TYPE_XXXX

Retorna:

Número de objetos deste tipo

`iterator.GetObject(itype, iobject)`

Retorna um objeto de uma lista

itype TQSModel.TYPE_XXXX

iobject 0..GetNumObjects()-1

Retorna:

smobject Objeto derivado de SMOBJECT ou None

`iterator.FindObject(identobj)`

Acha um objeto dado seu identificador único

identobj Identificador único de um objeto

Retorna:

objme Objeto ou None se não achar

Error! Use the Home tab to apply Título 1 to the text that you want to appear here.137

itype	Tipo do objeto TQSModel.TYPE_XXXX
index	Índice de acesso
istat	(0) Ok (1) Não achou

11.3.4. Utilitários do pavimento: Floor.util

São utilitários que agem sobre os objetos do pavimento.

util.DoIntersections()

Refaz as intersecções de vigas, pilares e lajes em uma planta. Necessário para reconhecimento de intersecções entre vigas e pilares, além do reconhecimento do contorno das lajes. É uma operação lenta, assim deve-se deixar para executar quando for necessário o conhecimento das intersecções de vigas e pilares, ou o contorno das vigas para a criação de lajes.

util.DistributeSlabMould(slab, slabMould)

Distribui formas de laje nervurada em uma laje, a partir de uma fornecida

slab Laje Slab() onde será feita a distribuição

slabMould Forma de nervura SlabMould() como semente

11.4. Identificação de elementos - SMOBJECTIDENT()

O objeto SMOBJECTIDENT() é usado em diversos elementos do Modelador, e contém dados de identificação e posição de um elemento.

objectNumber

Número do elemento

objectTitle

Título alfanumérico ou "" se não definido

textPosition

Retorna objeto TextPosition() com posição de texto editável

renumerable

Elemento renumeravel (0) Não (1) Sim

11.5. Identificação de pré-moldados PreCastGroup()

Uma das características principais das peças pré-moldadas é sua repetição, o que facilita a fabricação. Além do número dos elementos atribuídos pelo Modelador, os elementos pré-moldados são nomeados por grupos, que podem ser gerados pelo Modelador. O acesso aos nomes usados nos grupos é pelos objetos tipo PreCastGroup().

floorPlanGroup

Nome do grupo de formas

floorPlanNumber

Número único de elemento do grupo

reinforcementGroup

Nome do grupo de armação

beamLoad

Carga distribuída para vigas com grupo de armação renumerado (string)

11.6. Objeto Polygon

O objeto `Polygon()` é usado por alguns tipos de objeto, como pilares, capitéis, furos em lajes, cargas distribuídas por área, e outros. Trata-se de uma poligonal com pontos definida em 2D, aberta ou fechada, conforme o uso.

Clear()

Esvazia uma poligonal

NumPts()

Retorna o número de pontos

GetPt(ipt)

Lê um ponto

ipt Índice do ponto `0..NumPts()-1`

Retorna:

x, y Coordenadas do ponto

GetPts()

Lê a poligonal inteira como uma matriz

Retorna:

x[], y[]

Enter(x, y)

Entra um ponto no final da poligonal

x, y Coordenadas do ponto

Set(ipt, x, y)

Define as coordenadas de um ponto na posição ipt

ipt Índice do ponto `0..NumPts()-1`

x, y Coordenadas do ponto

Error! Use the Home tab to apply Título 1 to the text that you want to appear here.139

Insert(ipt, x, y)

Entra as coordenadas de um ponto na posição ipt e empurra os demais para frente

ipt Índice do ponto 0..NumPts()-1

x, y Coordenadas do ponto

11.7. Carga básica Load()

Este objeto representa uma força ou momento aplicado em um elemento estrutural. Ele é usado para compor cargas concentradas, lineares e distribuídas por área em objetos do Modelador.

SetLoad(itype)

Define o tipo de carga

itype Tipo LOADTYPE_XXXXX

Tipo	Uso
TQSModel.LOADTYPE_DISARE	Distribuída por área
TQSModel.LOADTYPE_DISLIN	Distribuída linear
TQSModel.LOADTYPE_CONCEN	Concentrada
TQSModel.LOADTYPE_MOMCON	Momento concentrado

SetAlpha(icase, ialpha)

Define modo de carga numérico ou alfanumérico

icase Caso de carga 1..número de casos de grelha do pavimento

ialfa Carga (0) Numérica (1) Alfanumérica da tabela de tipos de cargas

SetLoadName(icase, loadname)

Para cargas alfanuméricas, define o nome da carga na tabela de cargas

icase Caso de carga 1..número de casos de grelha do pavimento

loadname Nome da carga

SetMainLoad(icase, loadval)

Valor da carga permanente

icase Caso de carga 1..número de casos de grelha do pavimento

loadval Valor da carga em unidades coerentes

SetLiveLoad(icase, loadval)

Valor da carga variável

icase Caso de carga 1..número de casos de grelha do pavimento

loadval Valor da carga em unidades coerentes

SetWallHeight(icase, iwall, wallheight)

Define o uso da altura da parede. Paredes importadas ficam à disposição no modelo para visualização em 3D.

icase	Caso de carga 1..número de casos de grelha do pavimento
iwall	Usa parede para carga por área (0) Não (1) Sim
wallheight	Altura da parede (m) ou (0) Para PD da planta

11.8. Superclasse SMOobject dos objetos do Modelador

Os objetos do Modelador são todos derivados da classe SMOobject. Assim, tem os mesmos atributos abaixo.

type

Tipo do objeto (TQSMODEL.TYPE_XXXX)

identobj

Identificador único do objeto

number

Número do objeto

title

Título opcional do objeto

formattedTitle

Título considerando número e título opcional

A rotina que retorna o GUID de um objeto (identificador universal único em forma de string) exige o fornecimento de parâmetros adicionais. Isto porque a estrutura de dados usada no Modelador pode ser diferente do programa para onde os objetos serão exportados. Por exemplo, uma única viga com dois vãos no Modelador pode ser dois objetos diferentes em um programa BIM. Também um piso com repetição, pode ter que ser exportado como pisos separados, e os elementos em cada piso precisarão de um GUID diferente.

GetGuid(iadic, ipiso, itrecho)

iadic	Número adicional para diferenciar pisos e trechos
ipiso	Piso
itrecho	Trecho opcional

Retorna:

Identificador tipo GUID (string)

11.9. Pilar - Column(SMObject)

Pilares são armazenados na lista espacial do edifício. Apesar da lista especial, a criação do pilar é feita a partir do objeto do pavimento atual. Um pilar é composto por *seções de pilar*, independentes uma da outra. Um pilar pode ter mais de uma seção, de maneira que pode variar as dimensões de uma seção para outra, assim como variar a posição em planta. Em um pilar inclinado, também cada piso tem uma seção diferente, pois sua posição muda.

Todo pilar nasce em um determinado pavimento (não pode ser pavimento com repetição). A definição do pavimento final é dada na estrutura de dados de cada seção.

Pilaretes são um caso especial de pilar, com apenas uma seção que nasce e/ou morre em um piso auxiliar.

Fundações, por questões históricas, são armazenadas na mesma estrutura de dados de pilares.

11.9.1. Dados atuais de pilares

São os dados usados na criação de um pilar novo, que valem para todos os pavimentos. Você também pode criar um novo com quaisquer dados atuais, e alterá-los posteriormente diretamente na estrutura de dados do pilar. Nas descrições abaixo, `model` é o objeto do modelo estrutural obtido como em:

```
model = TQSModel.Model ()
```

A maior parte dos dados atuais é mantida globalmente. Entretanto alguns dados atuais são definidos por pavimento, e neste caso usa-se o pavimento atual em uma variável como `floor`:

```
floor = model.floors.GetFloor ("nome-da-planta")
```

Os dados atuais globais são estes:

```
model.current.globalBimData.userAttrib.GetUserAttrib(itypeattrib)
```

Retorna um objeto tipo `UserAttrib()`, com atributos definidos pelo usuário. Os tipos definidos em `itypeattrib` são do tipo `BIM_USRATRTP_XXXXX`. Esta chamada obtém dados BIM de pilares, fundações, vigas, lajes e elementos 3D:

Constante	Uso
<code>TQSModel.BIM_USRATRTP_PILAR</code>	Pilar
<code>TQSModel.BIM_USRATRTP_FUNDC</code>	Fundações
<code>TQSModel.BIM_USRATRTP_VIGAS</code>	Vigas
<code>TQSModel.BIM_USRATRTP_LAJES</code>	Lajes
<code>TQSModel.BIM_USRATRTP_ELM3D</code>	Elementos 3D

`model.current.globalBimData.globalAttrib`

Retorna um objeto tipo `GlobalAttrib()` com variáveis BIM definidas pelo usuário e que valem para todos os elementos exportados

`model.current.globalColumnData.foundationData`

Retorna dados que definem uma fundação - `FoundationData()`.

`model.current.globalColumnData.spreadFootingSideWalls`

Valor do colarinho de uma sapata, cm

`model.current.globalColumnData.foundationsTopFaceRecess`

Fundações: Valor do rebaixo cm

`model.current.globalColumnData.firstPileNumber`

Fundações: Número de 1ª estaca

`model.current.globalColumnData.lastPileNumber`

Fundações: Número da última estaca

`columnData = model.current.globalColumnData.columnData`

`columnData = model.current.globalColumnData.shortColumnData`

`columnData = model.current.globalColumnData.foundationColumnData`

Estas chamadas retornam um objeto da classe `GlobalColumnData()`, que contém dados do mesmo tipo respectivamente para pilares, pilaretes e fundações. Este objeto tem as seguintes variáveis a serem definidas:

`columnData.columnDetailing`

Dados de detalhamento de pilar – objeto `ColumnDetailing()`

`columnData.columnGeometry`

Dados de geometria de pilar - objeto `ColumnGeometry()`

`columnData.columnInsertion`

Dados de inserção de pilar - objeto `ColumnInsertion()`

`columnData.columnPolygon`

Contorno de pilar polygonal – objeto `Polygon()`

Error! Use the Home tab to apply Título 1 to the text that you want to appear here.143

`columnData.columnCoil`

Mola do pilar no pórtico especial – Objeto `ColumnCoil()`

`columnData.columnIdent`

Identificação do pilar – Objeto `SObjectIdent()`

`columnData.columnPrecastData`

Dados de pilar pré-moldados – Objeto `ColumnPrecastData()`

`columnData.columnStarts`

Pilar nasce em `COLUMNSTART_XXXX`

Constante	Uso
<code>TQSModel.COLUMNSTART_NASCEDIRT</code>	Nasce no solo (sem fundação definida)
<code>TQSModel.COLUMNSTART_NASCEVIGA</code>	Nasce em viga
<code>TQSModel.COLUMNSTART_NASCEPIFU</code>	Nasce em pilar ou fundação
<code>TQSModel.COLUMNSTART_NASCELAJE</code>	Nasce em laje

`columnData.columnModel`

Modelo de pilar: `COLUMNUSE_XXXXXX`

Constante	Uso
<code>TQSModel.COLUMNUSE_COMPRESSAO</code>	Padrão - forças de compressão
<code>TQSModel.COLUMNUSE_TRACAO</code>	Principalmente tração, pode compressão
<code>TQSModel.COLUMNUSE_COMPAT</code>	Pilar de compatibilização
<code>TQSModel.COLUMNUSE_ESCORA</code>	Escora (só compressão)
<code>TQSModel.COLUMNUSE_TIRANTE</code>	Tirante (só tração)

`columnData.columnWindSupport`

Suporte a vento: (0) não (1) sim

`columnData.columnSectionMode`

Dados de condições de contorno: (0) da seção (1) do pavimento

`columnData.columnInterference`

Verificação de interferências (0) não (1) sim

`columnData.columnSloped`

Pilar inclinado (0) não (1) sim

`columnData.columnIsAFoundation`

O objeto de pilar representa uma fundação (0) não (1) sim

`columnData.columnHinges`

Pilar articulado em (0)CONTPOR.DAT (1)base/topo (2)base (3)topo

`columnData.columnExport`

Pilar exportável para o 3D (0) não (1) sim

`columnData.columnNonLinearity`

Coefficientes de não linearidade (0)Pilar (1)Não fissurado (2)Fissurado

`columnData.columnBucklingX`

Coefficiente de flambagem X

`columnData.columnBucklingY`

Coefficiente de flambagem Y

`columnData.columnDoubleStoryX`

Pé-direito duplo X (0) Geometria (1) Não (2) Sim

`columnData.columnDoubleStoryY`

Pé-direito duplo Y (0) Geometria (1) Não (2) Sim

`columnData.columnConcreteFc`

Fck diferenciado de pilar (string)

`floor.current.column.columnFloorNames`

Retorna a lista de plantas de pilares `ColumnFloorNames()`

`columnData.column.columnBoundaryCond`

Retorna objeto de condições de contorno de grelha – `ColumnBoundaryCond()`

`columnData.column.columnCover`

Cobrimento do pilar na planta, cm

`columnData.column.columnExposure`

Pilar em contato com o solo (0) não (1) sim

`floor.current.column.columnDynHorLoad`

Carga dinâmica de veículo em pilar (0) não (1) sim

11.9.2. Dados de pilares

Parte dos dados é comum aos dados atuais.

hasFixedPoint

(1) Se o pilar tem um ponto fixo

fixedPointX

X do ponto fixo do pilar

fixedPointY

Y do ponto fixo do pilar

Error! Use the Home tab to apply Título 1 to the text that you want to appear here.145

columnDetailing

Dados de detalhamento de pilar – objeto ColumnDetailing ()

columnIdent

Identificação do pilar – objeto SMOBJECTIDENT ()

columnStarts

Pilar nasce em COLUMNSTART_XXXX

Constante	Uso
TQSModel.COLUMNSTART_NASCEDIRT	Nasce no solo (sem fundação definida)
TQSModel.COLUMNSTART_NASCEVIGA	Nasce em viga
TQSModel.COLUMNSTART_NASCEPIFU	Nasce em pilar ou fundação
TQSModel.COLUMNSTART_NASCELAJE	Nasce em laje

columnData.columnModel

Modelo de pilar: COLUMNUSE_XXXXXX

Constante	Uso
TQSModel.COLUMNUSE_COMPRESSAO	Normal - forças de compressão
TQSModel.COLUMNUSE_TRACAO	Principalmente tração, pode compressão
TQSModel.COLUMNUSE_COMPAT	Pilar de compatibilização
TQSModel.COLUMNUSE_ESCORA	Escora (só compressão)
TQSModel.COLUMNUSE_TIRANTE	Tirante (só tração)

columnWindSupport

Suporte a vento: (0) não (1) sim

columnInterference

Verificação de interferências (0) não (1) sim

columnSloped

Inclinação do pilar (0) não (1) sim

columnCoil

Mola do pilar no pórtico espacial - ColumnCoil ()

columnIsAFoundation

O objeto de pilar representa uma fundação (0) não (1) sim

foundationData

Dados de fundação - FoundationData ()

foundationDefined

Fundação definida (0) não (1) sim

isShortColumn

O pilar é (0) normal (1) pilarete

ShortColumnAuxiliaryFloor

Piso auxiliar de base do pilarete

columnPrecastData

Dados de pilar pré-moldados - `ColumnPrecastData()`

columnExport

Pilar exportável para o 3D (0) não (1) sim

GetFoudation()

Retorna pilar ou fundação que servem de apoio ao pilar atual ou None

columnNonLinearity

Coefficientes de não linearidade (0)Pilar (1)Não fissurado (2)Fissurado

userAttrib

Atributos de usuário - `UserAttrib()`

ColumnNumSections()

Retorna o número de seções `ColumnSection()` de pilar

ColumnGetSection(isec)

Retorna uma seção

`isec` `0..ColumnNumSections()-1`

Retorna:

objeto `ColumnSection()`

ColumnCreateSection(isec, lastfloor)

Insere nova seção de pilar. Empurra demais seções

A primeira seção é criada junto com o pilar

Cria com a geometria da seção anterior

`isec` `0..ColumnNumSections()-1`

`lastfloor` Última planta desta seção (string)

Retorna:

objeto `ColumnSection()`

ColumnSectionUpdateGeometry(columnSection)

Atualiza uma seção de pilar após alteração de geometria

`columnSection` Objeto `ColumnSection`

11.9.3. Seção de pilar – ColumnSection()

Um pilar pode ter uma ou mais *seções de pilares*. A seção contém dados de geometria. Com múltiplas seções, podemos definir um pilar que varia ao longo dos lances, assim como pilares inclinados.

column

Retorna pilar `Column()` correspondente à seção

insertionX

Ponto de inserção X cm

insertionY

Ponto de inserção Y cm

GetColumnFloorData(floor)

Retorna objeto com Informações por planta - `ColumnFloorData()`

Retorna um objeto com informações por planta

`floor` `Pavimento Floor()`

Retorna:

objeto `ColumnFloorData()`

GetColumnBoundaryCond(floor)

Retorna objeto de condições de contorno de grelha

`floor` `Pavimento Floor()`

Retorna:

objeto `ColumnBoundaryCond()`

SetColumnBoundaryMode(floor, idefcontpil)

Define se as condições de contorno são por seção de pilar ou planta

`floor` `Pavimento Floor()`

`idefcontpil` Condições de contorno por (0) seção (1) planta

GetShearWallStripNum()

Retorna o número de lâminas de discretização de pilares parede

GetShearWallStripNumPts(ilaam)

Retorna o número de pontos de uma lâmina

`ilaam` Índice da lâmina `0..GetShearWallStripNum()-1`

GetShearWallStripPoint(ilaam, ipt)

Retorna o número de pontos de uma lâmina

`ilaam` Índice da lâmina `0..GetShearWallStripNum()-1`

`ipt` Índice do ponto `0..GetShearWallStripNumPts()-1`

Retorna:

`x, y` Um ponto da lâmina

columnGeometry

Dados de geometria de pilar - `ColumnGeometry()`

sectionPolig

Seção transversal - `Polygon()`

sectionLastFloor()

Última planta da seção (string)

columnInsertion

Dados de inserção de pilar - `ColumnInsertion()`

11.9.4. Geometria de pilares – ColumnGeometry()

Contém dados de seções padrão e seções catalogadas. A poligonal é definida por `sectionPolig`.

sectionType()

Tipo de seção `COLUMNTYPE_XXX`

Constante	Uso
<code>TQSModel.COLUMNTYPE_R</code>	Seção Retangular
<code>TQSModel.COLUMNTYPE_L</code>	Seção L
<code>TQSModel.COLUMNTYPE_U</code>	Seção U
<code>TQSModel.COLUMNTYPE_C</code>	Seção Circular
<code>TQSModel.COLUMNTYPE_P</code>	Seção Poligonal
<code>TQSModel.COLUMNTYPE_N</code>	Outras não padrão

sectionB1

Dimensão B1 de seções R/L/U/C

sectionH1

Dimensão H1 de seções R/L/U

sectionB2

Dimensão B2 de seções L/U

sectionH2

Dimensão H2 de seções L/U

sectionAngle

Ângulo de inserção da seção em graus

sectionMaterial

Título do material diferente do concreto (não padrão)

sectionName

TQS Informática Ltda. - Rua dos Pinheiros 706 / casa 2 - 05422-001 - São Paulo/SP - Tel.: (011) 3883-2722 - Fax: (011) 3083-2798

Nome de seção não padrão – catalogada

11.9.5. Lista de planta de pilares - ColumnFloorNames()

Fora a planta onde um pilar nasce, cada planta desta lista representa um limite para uma seção de pilar.

floorNumber()

Número de plantas

Clear()

Limpa a lista de plantas e cria duas vazias: a primeira e a última do edifício

EnterFloor(iplanta, nompla)

Entra uma planta em posição fixa e empurra as demais plantas

iplanta Planta 0.. floorNumber()-1

nompla Nome da planta

GetFloor(iplanta)

Lê uma planta do pilar

iplanta Planta 0.. floorNumber()-1

Retorna:

nompla Nome da planta

11.9.6. Informações de pilar por seção ou pavimento - ColumnFloorData()

Certas informações de pilares podem ser definidas para todos os lances de mesma seção, ou para todos os lances na repetição de um pavimento.

infoByFloor

Informações de pilares por (0) Seção de pilar (1) Planta

columnBoundaryCond

Condições de contorno de pilares – objeto ColumnBoundaryCond()

columnBucklingX

Coeficiente de flambagem X

columnBucklingY

Coeficiente de flambagem Y

columnDoubleStoryX

Pé-direito duplo X (0) Geometria (1) Não (2) Sim

columnDoubleStoryY

Pé-direito duplo X (0) Geometria (1) Não (2) Sim

columnConcreteFc

Fck diferenciado de pilar (string)

columnCover

Cobrimento do pilar na planta, cm

columnExposure

Pilar em contato com o solo (0) não (1) sim

columnHinges

Pilar articulado em (0)CONTPOR.DAT (1)base/topo (2)base (3)topo

columnDynHorLoad

Carga dinâmica de veículo em pilar (0) não (1) sim

columnTransferBlock

Dados de Bloco de transição – objeto `ColumnTransferBlock()`

11.9.7. Inserção interativa de pilares - `ColumnInsertion()`

São dados usados na inserção interativa de um pilar.

insertionType

Inserção de pilar por (0) Centro (1) Canto

insertionCorner

Numero do canto (0..) para `insertionType == 1`

sectionCover

Revestimento cm

mediumPointInsertion

Inserção por ponto intermediário (0) Não (1) Sim

11.9.8. Condições de contorno `ColumnBoundaryCond()`

São condições de contorno para o Modelo IV na discretização em grelha de cada piso.

bearingModel

Tipo de apoio em grelha `COLUMNBEARING_XXXXX`

Constante	Uso
TQSModel.COLUMNBEARING_DEFAULT	Default
TQSModel.COLUMNBEARING_ARTCONT	Articulado contínuo
TQSModel.COLUMNBEARING_ARTINDP	Articulado independente
TQSModel.COLUMNBEARING_ELACONT	Elástico contínuo
TQSModel.COLUMNBEARING_ELAINDP	Elástico independente

GetSpringValue(icoef)

Retorna o valor da mola de rotação ou translação do apoio em pilar nas grelhas

icoef Coeficiente COLUMNSPRING_XXXX

Retorna:

coefmola Coeficiente de mola em tfm/rad (rotação) ou tf/m (translação)

Constante	Uso
TQSModel.COLUMNSPRING_IRX	Rx
TQSModel.COLUMNSPRING_IRY	Ry
TQSModel.COLUMNSPRING_ITZ	Tz
TQSModel.COLUMNSPRING_ITX	Tx
TQSModel.COLUMNSPRING_ITY	Ty

SetSpringValue(icoef, coefmola)

Define o valor da mola de rotação ou translação do apoio em pilar nas grelhas

icoef Coeficiente COLUMNSPRING_XXXX

coefmola Coeficiente de mola em tfm/rad (rotação) ou tf/m (translação)

mandatoryPunchingShear

Armadura de punção obrigatória (0) Não (1) Sim

shearWallDiscretization

Pilar parede discretizado (0) Não (1) Sim, estimado ou discretizado

GetGap(igap)

Valor do gap (m) de restrição de apoio

igap Gap TIPO COLUMNCOIL_IGAP_XXXX

Retorna:

gap Valor do gap (m)

Constante	Uso
TQSModel.COLUMNCOIL_IGAP_XPOS	Gap Tx+
TQSModel.COLUMNCOIL_IGAP_XNEG	Gap Tx-
TQSModel.COLUMNCOIL_IGAP_YPOS	Gap Ty+
TQSModel.COLUMNCOIL_IGAP_YNEG	Gap Ty-
TQSModel.COLUMNCOIL_IGAP_ZPOS	Gap Tz+
TQSModel.COLUMNCOIL_IGAP_ZNEG	Gap Tz-

SetGap(igap, gap)

Valor do gap (m) de restrição de apoio

igap Gap TIPO COLUMNCOIL_IGAP_XXXX

gap Valor do gap (m)

11.9.9. Detalhamento de pilar – ColumnDetailing()

São informações usadas para detalhamento de pilares.

foundationHeight

Altura da fundação, cm

baseRecess

Rebaixo da base, cm

topRecess

Rebaixo no topo, cm

isDetailable

Pilar detalhável (0) não (1) sim

isCurtain

Pilar simula cortina (0) não (1) sim

canStartOutOfFoundation

Pode nascer fora do piso fundação (0) não (1) sim

isFictitious

Pilar fictício (0) não (1) sim

isShearWall

Pilar parede para o BIM

11.9.10. Molas de pilar no portico especial - ColumnCoil()

São as molas sob o pilar ou fundação usados no portico especial.

GetSupportType(ires)

Tipo de apoio de pilar

ires Restrição a considerar COLUMNCOIL_IRES_XXXX

Retorna:

iapopor Tipo de apoio COLUMNCOIL_IAPOPOR_XXXX

Constante	Uso
TQSModel.COLUMNCOIL_IRES_IRX	Rotação X
TQSModel.COLUMNCOIL_IRES_IRY	Rotação Y
TQSModel.COLUMNCOIL_IRES_IRZ	Rotação Z
TQSModel.COLUMNCOIL_IRES_ITX	Translação X
TQSModel.COLUMNCOIL_IRES_ITY	Translação Y
TQSModel.COLUMNCOIL_IRES_ITZ	Translação Z

Constante	Uso
TQSModel.COLUMNCOIL_IAPOPOR_DEFAULT	Engaste ou padrão
TQSModel.COLUMNCOIL_IAPOPOR_ARTICULADO	Articulado
TQSModel.COLUMNCOIL_IAPOPOR_ELASTICO	Elástico
TQSModel.COLUMNCOIL_IAPOPOR_RECALQUE	Recalque

SetSupportType(ires, iapopor)

Tipo de apoio de pilar

ires Restrição a considerar COLUMNCOIL_IRES_XXXX

iapopor Tipo de apoio COLUMNCOIL_IAPOPOR_XXXX

GetCoil(ires)

Coefficiente de mola

ires Restrição a considerar COLUMNCOIL_IRES_XXXX

Retorna:

pormola Coeficiente de mola à Rotação tfm/rad Translação tf/m

SetCoil(ires, pormola)

Coefficiente de mola

ires Restrição a considerar COLUMNCOIL_IRES_XXXX

pormola Coeficiente de mola à Rotação tfm/rad Translação tf/m

GetGap(igap)

Valor do gap (m) de restrição de apoio

igap Gap TIPO COLUMNCOIL_IGAP_XXXX

Retorna:

gap Valor do gap (m)

Constante	Uso
TQSModel.COLUMNCOIL_IGAP_XPOS	Gap Tx+
TQSModel.COLUMNCOIL_IGAP_XNEG	Gap Tx-
TQSModel.COLUMNCOIL_IGAP_YPOS	Gap Ty+
TQSModel.COLUMNCOIL_IGAP_YNEG	Gap Ty-
TQSModel.COLUMNCOIL_IGAP_ZPOS	Gap Tz+
TQSModel.COLUMNCOIL_IGAP_ZNEG	Gap Tz-

SetGap(igap, gap)

Valor do gap (m) de restrição de apoio

igap Gap TIPO COLUMNCOIL_IGAP_XXXX

gap Valor do gap (m)

11.9.11. Dados de fundações - FoundationData()

Os objetos desta classe podem representar fundações em sapatas, blocos, tubulões e blocos sobre tubulões.

type

Tipo de fundação FOUNDATION_ITIPOFUNDAC_XXXX

Constante	Uso
TQSModel.FOUNDATION_ITIPOFUNDAC_SAPATA	Sapata
TQSModel.FOUNDATION_ITIPOFUNDAC_BLOCO	Bloco sobre estacas
TQSModel.FOUNDATION_ITIPOFUNDAC_TUBULAO	Tubulão

fictitiousColumn

Pilar fictício definido (0) não (1) sim

fictitiousColumnXDim

Dimensão X de pilar fictício cm

fictitiousColumnYDim

Dimensão Y de pilar fictício cm

beamSupport

Vigas apoiam na fundação (0) não (1) sim

footingXDim

Sapatas, dimensão X da base cm

footingYDim

Sapatas, dimensão Y da base cm

footingTopXDim

Sapatas, dimensão X do topo cm

footingTopYDim

Sapatas, dimensão Y do topo cm

footingTopXExc

Sapatas, excentricidade X do topo cm

footingTopYExc

Sapatas, excentricidade Y do topo cm

Error! Use the Home tab to apply Título 1 to the text that you want to appear here.155

footingTopExcLeft

Sapata de divisa à esquerda (0) não (1) sim

footingTopExcAbove

Sapata de divisa acima (0) não (1) sim

footingTopExcRight

Sapata de divisa à direita (0) não (1) sim

footingTopExcUnder

Sapata de divisa abaixo (0) não (1) sim

footingHeight

Altura total cm

footingXBaseBoard

Altura X do rodapé cm

footingYBaseBoard

Altura Y do rodapé cm

pileCapX

Base X do bloco, cm

pileCapY

Base Y do bloco, cm

pileCapHeight

Altura do bloco, cm

pileCapPiles

Número de estacas do bloco

pileCapFormat

Formato (0..) Varia com o número de estacas

pileCapDiameter

Diâmetro da estaca cm

pileCapHeightW

Altura da estaca dentro do bloco cm

pileCapPileXDist

Distância X entre eixos de estacas cm

pileCapPileYDist

Distância Y entre eiyos de estacas cm

pileCapPileFaceDist

Distância do eixo da estaca à face do bloco cm

pileCapDistMode

Cálculo da dimensão do bloco (0) dimensões fornecidas (1) distância entre estacas

pileCapPileHeight

Altura das estacas cm

pileCapOverPiers

Bloco sobre tubulões (0) não (1) sim

pierShaftDiameter

Tubulão - diâmetro do fuste cm

pierShaftHeight

Tubulão - altura do fuste cm

pierBaseboardDiameter

Tubulão - diâmetro do rodapé cm

pierBaseboardHeight

Tubulão - altura do rodapé cm

pierBaseboardCone

Tubulão - altura do cone cm

pierFalseEllipse

Tubulão com base alargada (0) não (1) sim

pierFalseEllipseWidth

Tubulão - alargamento da base cm

pierFalseEllipseAngle

Tubulão - Rotação da base, sistema global, graus

pocketFoundation

Fundação em cálice (0) não (1) sim

pocketFoundationFormat

Cálice: Formato FOUNDATION_FORM_XXXXX

Error! Use the Home tab to apply Título 1 to the text that you want to appear here.157

Constante	Uso
TQSModel.FOUNDATION_FORM_RETANGULAR	Cálculo: Formato retangular
TQSModel.FOUNDATION_FORM_CIRCULAR	Cálculo: Formato circular

pocketFoundationRoughness

Cálculo: Rugosidade FOUNDATION_RUGO_XXXX

Constante	Uso
TQSModel.FOUNDATION_RUGO_PADRAO	Cálculo: Rugosidade conforme critérios
TQSModel.FOUNDATION_RUGO_SULISA	Cálculo: Superfície lisa
TQSModel.FOUNDATION_RUGO_RUGOSA	Cálculo: Superfície rugosa, pilar também
TQSModel.FOUNDATION_RUGO_CHAVECIS	Cálculo: Chave de cisalhamento

pocketFoundationXDim

Cálculo: dimensão X em planta cm (D circular)

pocketFoundationYDim

Cálculo: dimensão Y em planta cm (D circular)

pocketFoundationXWall

Cálculo: dimensão parede X cm

pocketFoundationYWall

Cálculo: dimensão parede Y cm

pocketFoundationXExc

Cálculo: Excentricidade X em relação ao centro da fundação cm

pocketFoundationYExc

Cálculo: Excentricidade Y em relação ao centro da fundação cm

pocketFoundationXInc

Cálculo: Caimento X planta parede interna cm

pocketFoundationYInc

Cálculo: Caimento Y planta parede interna cm

pocketFoundationHeight

Cálculo: Altura cm

pocketFoundationRecess

Cálculo: Rebaixo em relação ao nível da fundação cm

11.9.12. Dados de pré-moldados - ColumnPrecastData()

Estes são dados específicos para pilares pré-moldados.

precastRegion

Pilar pré-moldado: região construtiva

isPrecast

Pilar pré-moldado: (0) Não (1) Sim

rebarBundling

Pilar pré-moldado: Alojamento de armadura (0)Critérios (1)TQS Pilar (2)Feixe

rainWaterPipe

Tubo de água pluvial - objeto RainWaterPipe()

liftingAnchors

Alça de içamento de pilares pré-moldados – objeto LiftingAnchors()

liftOpenning

Furos de levantamento de pilares pré-moldados – objeto LiftOpenning()

preCastGroup

Grupo de pré-moldados – objeto PreCastGroup()

11.9.13. Pré-moldados: tubo de água pluvial RainWaterPipe()

É um tubo de água pluvial que pode ser definido dentro de um pilar pré-moldado.

pipeDiameter

Diâmetro do tubo cm

minFunnelWidth

Diâmetro do funil menor cm

maxFunnelWidth

Diâmetro do funil maior cm

rotation

Ângulo adicional em relação ao pilar em graus

baseDistance

Distância de saída da base cm (0.) Adota default

tubeIdentification

Identificador do tubo

11.9.14. Pré-moldados: alças de içamento LiftingAnchors()

São as alças pelas quais as peças são suspensas e transportadas na fábrica.

GetNumLiftingAnchors()

Retorna o número de alças/rebaixos definidos

GetLiftingRecess(ialca)

Retorna um rebaixo de alça\n

ialca 0..GetNumLifting()-1

Retorna:

dfs Rebaixo de alça cm

SetLiftingRecess(ialca, dfs)

Define um rebaixo de alça

ialca 0..GetNumLifting()-1

dfs Rebaixo de alça cm

EnterLiftingRecess(dfs)

Entra um rebaixo de alça no final da lista

dfs Rebaixo de alça cm

EraseLiftingAnchor(ialca)

Apaga um rebaixo de alça

ialca 0..GetNumLifting()-1

SortLiftingAnchors()

Classifica as alças por ordem de rebaixo

liftingAnchorId

Identificador de alças (string)

liftingPosition

Alças em posição padrão (0) Não (1) Sim

liftingAnchorsDefined

Alças definidas (0) Não (1) Sim

topLiftingAngle

Ângulo da alça de topo em graus

11.9.15. Pré-moldados: furos de levantamento LiftOpening()

São os furos por onde passam cabos para levantar as peças pré-moldadas na montagem.

GetNumLiftingOpennings()

Retorna o número de furos de içamento definidos

GetLiftingRecess(ifur)

Retorna um rebaixo de furo

```
ifur          0..GetNumLiftingOpennings()-1
```

Retorna:

```
dfs          Rebaixo de furo cm
```

SetLiftingRecess(ifur, dfs)

Define um rebaixo de furo

```
ifur          0..GetNumLiftingOpennings()-1
```

```
dfs          Rebaixo de furo cm
```

EnterLiftingRecess(dfs)

Entra um rebaixo de furo no final da lista

```
dfs          Rebaixo de furo cm
```

EraseLiftingOpening(ifur)

Apaga um rebaixo de furo

```
ifur          0..GetNumLifting()-1
```

SortLiftingOpennings()

Classifica as furos por ordem de rebaixo

liftingDiameter

Diâmetro do furo, cm

liftingEccentricity

Excentricidade do furo (cm) em relação ao CG do pilar

liftingPosition

Furos em posição padrão (0) Não (1) Sim

liftOpeningDefined

Furos de içamento definidos (0) Não (1) Sim

11.10. Viga - Beam(SMObject)

Vigas são objetos compostos por trechos, definidos por *nós de vigas*. Um nó de viga ao mesmo tempo define as coordenadas do nó, e as propriedades do trecho até o próximo nó (o último nó não tem trecho definido).

Você não define necessariamente todos os nós de uma viga. Você define os *nós originais*, em geral o primeiro e último nó, e intermediários se houver mudança de direção. Os demais nós são gerados automaticamente após atualização das intersecções com outras vigas e pilares do pavimento.

Alguns dados de geometria e inércia, podem ser definidos especificamente para um trecho. Assim, temos dados que valem para toda a viga e dados que são específicos por trecho.

11.10.1. Dados atuais de vigas

São dados usados na criação de uma viga, antes dela existir. Você também pode criar uma viga sem considerar os dados atuais, e modifica-la depois de criada. Considerando o modelo e pavimento:

```
model = TQSModel.Model ()  
floor = model.floors.GetFloor ("nome-da-planta")
```

temos os seguintes dados atuais:

```
model.current.globalBeamData.beamRestraint  
Objeto de articulação de vigas BeamRestraint()
```

```
model.current.globalBeamData.firstSlopedBeamNumber  
Número da primeira viga inclinada
```

```
model.current.globalBeamData.slopedBeamIdent  
Identificação da viga inclinada
```

```
model.current.globalBimData.userAttrib  
Dados BIM de usuário – Objeto UserAttrib()
```

```
floor.current.floorBeamData.beamGeometry  
Geometria da viga (todos os trechos) – Objeto BeamGeometry()
```

```
floor.current.floorBeamData.beamInertia  
Inércia de viga (todos os trechos) – Objeto BeamInertia()
```

```
floor.current.floorBeamData.beamBond  
Vinculações e outros dados de viga – objeto BeamBond()
```

```
floor.current.floorBeamData.beamInsertion  
Dados de inserção de vigas – objeto BeamInsertion()
```

```
floor.current.floorBeamData.beamTemperShrink  
Temperatura / retração de vigas – objeto TemperatureShrink()
```

```
floor.current.floorBeamData.beamDetailing  
Detalhamento de vigas – objeto BeamDetailing()
```

```
floor.current.floorBeamData.beamExport  
(1) Se viga exportável para o 3D/BIM
```

```
floor.current.floorLoadData.load  
Objeto Load() com carga distribuída em toda a viga
```

11.10.2. Dados de vigas

São dados de uma viga existente, e podem ser editados.

beamIdent

Identificação da viga – Objeto `SObjectIdent()`

NumNodes()

Número de nós de uma viga (objetos `BeamNode()`)

GetBeamNode(ino)

Retorna um nó da viga na forma de um objeto `BeamNode()`

GetBeamGeometry()

Geometria de viga - válida nos trechos sem definição específica

(`SetSegmentBeamGeometryDefined`) – Objeto `BeamGeometry()`

GetSegmentBeamGeometry(ino)

Geometria de viga - do trecho ou geral (conforme

`SetSegmentBeamGeometryDefined`) – Objeto `BeamGeometry()`

GetSegmentBeamGeometryDefined(ino)

Retorna (1) se a geometria do trecho `ino` é específica ou (0) se é geral

SetSegmentBeamGeometryDefined(ino, ndefgeovig)

Define (1) se a geometria do trecho `ino` é específica ou (0) se é geral

GetBeamInertia()

Inércia de viga - válida nos trechos sem definição específica

(`SetSegmentBeamInertiaDefined`) – Objeto `BeamInertia()`

GetSegmentBeamInertia(ino)

Inércia de viga - Do trecho ou geral, conforme `SetSegmentBeamInertiaDefined` –

Objeto `BeamInertia()`

SetSegmentBeamInertiaDefined(ino, ndefinervig)

Define (1) se a inércia do trecho `ino` é específica ou (0) se é geral

beamBond

Vinculações e outros dados de viga – objeto `BeamBond()`

beamInsertion

Dados de inserção de vigas – objeto `BeamInsertion()`

GetLoad(floor)

Retorna objeto `Load()` com carga distribuída em toda a viga

`floor` Objeto `Floor()`, com dados do pavimento atual

temperShrink

Temperatura / retração de vigas – objeto TemperatureShrink()

beamDetailing

Detalhamento de vigas – objeto BeamDetailing()

auxiliaryFloor

Piso auxiliar

userAttrib

Atributos de usuário – objeto UserAttrib()

beamExport

(1) Se viga exportável para o 3D/BIM

11.10.3. Nó de viga - BeamNode()

Um nó de viga representa o nó, original ou gerado por intersecções, e as propriedades do trecho em seguida. O último nó não representa o trecho, que não existe.

nodeX

X do nó da viga

nodeY

Y do nó da viga

crossingType

Tipo de vinculação do nó da viga BEAMCROSSING_XXXX

Constante	Uso
TQSModel.BEAMCROSSING_INDEFINIDO	Indefinido
TQSModel.BEAMCROSSING_RECEBE	Recebe carga
TQSModel.BEAMCROSSING_CRUZAMENTO	Cruzamento sem apoio identificado
TQSModel.BEAMCROSSING_APOIAVIGA	Apoia em viga
TQSModel.BEAMCROSSING_APOIAPILAR	Apoia em pilar
TQSModel.BEAMCROSSING_N	Neutro

GetSlabConnection(idireito)

Retorna engastamento de um lado da viga com a laje

idireito (0) Esquerdo (1) Direito

Retorna:

iengast Tipo de engaste BEAMCONNECTION_XXX

fatengast Fator de engaste 0..1 para

iengast==BEAMCONNECTION_FATEMGs

Constante	Uso
<code>TQSModel.BEAMCONNECTION_DEFAULT</code>	Default
<code>TQSModel.BEAMCONNECTION_ENLIVRE</code>	Livre
<code>TQSModel.BEAMCONNECTION_ARTCMXY</code>	Articulado mx,my
<code>TQSModel.BEAMCONNECTION_ARTMXYF</code>	Articulado mx,my,fx
<code>TQSModel.BEAMCONNECTION_FATEMGS</code>	Engastado com fatengast

SetSlabConnection(idireito, iengast, fatengast)

Define engastamento de um lado da viga com a laje

idireito (0) Esquerdo (1) Direito

iengast Tipo de engaste BEAMCONNECTION_XXX

fatengast Fator de engaste 0..1 para
iengast==BEAMCONNECTION_FATEMGS

GetRestraint(ifinal)

Retorna objeto de engastamento no início ou fim de um trecho

ifinal (0) Inicial (1) Final

Retorna:

Objeto `BeamRestraint()`

arcSegment()

Retorna objeto `ArcSegment()` com dados de arco no trecho atual

11.10.4. Geometria de viga - **BeamGeometry()**

Contém dados de geometria da viga, que podem valer para toda a viga ou somente para um trecho.

width

Largura de viga retangular cm

depth

Altura de viga retangular cm

eccentricity

Excentricidade lateral cm

recess

Rebaixo em cm, positivo para baixo.

Error! Use the Home tab to apply Título 1 to the text that you want to appear here.165

GetFloorRecess (floor, beam)

Rebaixo em cm, considerando piso auxiliar

floor Objeto Floor(), com dados de um pavimento

beam Objeto Beam() que contém esta geometria

Retorna:

dfspiso Rebaixo da viga, cm, considerando piso auxiliar

sectionName

Nome de seção não padrão – catalogada

sectionRotation

Ângulo de rotação da seção, graus

sectionMaterial

Título do material diferente do concreto (não padrão)

variableSection

Retorna objeto VariableSection() para a definição de mísula

precastRegion

Pré-moldados: Região construtiva

beamPrecastData

Retorna objeto BeamPrecastData() para a definição de viga pré-moldadas

beamMetalSection

Retorna objeto BeamMetalSection() para a definição de viga com seção metálica

11.10.5. Inércia de viga - BeamInertia()

São dados que afetam o cálculo de inércia da viga no modelo estrutural.

effectiveFlange

Mesa colaborante (0) Não (1) Sim

maxEffectiveFlange

Mesa colaborante máxima cm

fixedEffectiveFlange

Mesa colaborante fixa cm

torsionalMomentMode

Divisor de inércia à torção definido em (0) Critérios (1) torsionalMomentDivider ()

torsionalMomentDivider

Divisor de inércia à flexão, conforme torsionalMomentMode

inertiaMomentDivider

Divisor de inércia à flexão, se (!=0)

notIntersectLeftSlab

Viga em relação à laje à esquerda (0) intercepta (1) não intercepta

notIntersectRightSlab

Viga em relação à laje à direita (0) intercepta (1) não intercepta

plasticAdaptability

Capacidade de adaptação plástica à torção (0) Não (1) Sim

11.10.6. Vinculações de viga - BeamBond()

Definição de vinculações e outros dados de vigas.

wideBeam

Viga faixa (0) Não (1) Sim

fixAtTheBegin

Engaste inicial (0) Não (1) Sim

fixAtTheEnd

Engaste final (0) Não (1) Sim

disableSelfWeight

Desabilitar peso próprio (0) Não (1) Sim

bearColumnByTheFaces

Interseção de pilar com faces (0) Não (1) Sim

adjustEdges

Ajuste automático das pontas (0) Não (1) Sim

intersectOthersBeams

Interseção com outras vigas (0) Não (1) Sim

startStrapBeam

Alavanca inicial (0) Não (1) Sim

endStrapBeam

Alavanca final (0) Não (1) Sim

transferGirder

Transição (0) pela geometria (1) Sim (2) Não

Error! Use the Home tab to apply Título 1 to the text that you want to appear here.167

workAs

Trabalha como (0) Viga (1) Tirante (2) Escora

11.10.7. Inserção interativa de viga - BeamInsertion()

Usados quando uma viga é inserida ou editada.

insertBy

Alinhamento (0) Esquerda (1) Direita (2) Eixo

covering

Revestimento das faces, cm

11.10.8. Detalhamento de viga - BeamDetailing()

Dados para detalhamento de vigas.

detailable

Detalhamento da viga (0) Normal (1) Não (2) De compatibilização

cover

Cobrimento diferenciado em cm

exposure

Em contato com o solo (0) Não (1) Sim (2) Exposta ao ambiente

restrainColumns

A viga trava pilares (0) Não (1) Sim

simulatesCurtain

Simula uma cortina (0) Não (1) Sim

prestressed

Protendida (0) Não (1) Sim

vproInterface

Interface com o VPRO (0) Não (1) Sim

11.10.9. Articulação de vigas – BeamRestraint()

Articulação nas extremidades da viga.

GetRestraintCoef(irdir)

Coeficientes de articulação na extremidade da viga

Irdir Direção (0) RX (1) RY (2) RZ (3) FZ

Retorna:

iartic Articulação definida (0) Não (1) Sim

coef Coeficiente de rotação ou translação

SetRestraint(irdir, iartic, coef)

Coeficientes de articulação na extremidade da viga

irdir Direção (0) RX (1) RY (2) RZ (3) FZ

iartic Articulação definida (0) Não (1) Sim

coef Coeficiente de rotação ou translação

GetPlasticMoments(itipopl, imompos)

Coeficientes de plastificação impostos

Valor máximo de momento fletor em uma ligação

itipopl Definição (0) My (1) Mz

imompos Definição (0) Máximo negativo (1) Positivo

Retorna:

isfmax (0) Conforme critérios (1) Não definido (2) em esfmax

esfmax Momento máximo tfm

SetPlasticMoments(itipopl, imompos, isfmax, esfmax)

Coeficientes de plastificação impostos

Valor máximo de momento fletor em uma ligação

itipopl Definição (0) My (1) Mz

imompos Definição (0) Máximo negativo (1) Positivo

isfmax Conforme critérios (1) Não definido (2) em esfmax

esfmax Momento máximo tfm

11.10.10. Segmento de arco de viga - ArcSegment()

Descreve o próximo trecho da viga como um segmento de arco.

startNode

Nó inicial da viga 0..NumNodes()-1

endNode

Nó final da viga 0..NumNodes()-1

signal

Sinal do arco (1) anti-horário (-1) horário

centerX

X do centro do arco cm

centerY

Y do centro do arco cm

11.10.11. Mísula e seção variável em viga - VariableSection()

Mísulas e seções variáveis não prismáticas em vigas não são calculadas no TQS. A sua definição permite construir um modelo 3D mais realista.

corbelDefined

Mísula definida (0) Não (1) Sim

startBottomHeight

Mísula: Altura inferior inicial cm

endBottomHeight

Mísula: Altura inferior final cm

startTopHeight

Mísula: Altura superior inicial cm

endTopHeight

Mísula: Altura superior final cm

startLeftWidth

Mísula: Largura esquerda inicial cm

endLeftWidth

Mísula: Largura esquerda final cm

startRightWidth

Mísula: Largura direita inicial cm

endRightWidth

Mísula: Largura direita final cm

variableSectionDefined

Seção variável definida (0) Não (1) Sim

startWidth

Seção variável: Largura inicial cm

startHeight

Seção variável: Altura inicial cm

startRecess

Seção variável: Rebaixo inicial cm

endWidth

Seção variável: Largura final cm

endHeight

Seção variável: Altura final cm

endRecess

Seção variável: Rebaixo final cm

11.10.12. Dados de viga pré-moldada - BeamPrestData()

Em desenvolvimento.

11.10.13. Seção metálica - BeamMetalSection()

Em desenvolvimento.

11.11. Laje - Slab(SMObject)

Lajes planas no pavimento são definidas pelas coordenadas do título da laje. O contorno é obtido automaticamente a partir de uma malha de vigas, pilares e fechamentos de bordo. O contorno de uma laje é aquele onde está seu título. Laje sem contorno definido é uma condição de erro.

Em lajes sem vigas, é comum a condição de contornos externos com contornos internos (como por exemplo no núcleo do edifício). Para as laje externas, os contornos internos se comportam como furos.

11.11.1. Dados atuais de laje

São dados usados na criação de uma laje – existem antes da laje. Você pode também criar uma laje sem considerar estes dados, e modifica-la posteriormente.

Considerando o modelo e pavimento:

```
model = TQSModel.Model ()  
floor = model.floors.GetFloor ("nome-da-planta")
```

temos os seguintes dados atuais:

```
floor.current.floorSlabData.slabIdent
```

Objeto SMObjectIdent() com identificação da laje

```
floor.current.floorSlabData.slabFirstNumber
```

Número da primeira laje

Error! Use the Home tab to apply Título 1 to the text that you want to appear here.171

`floor.current.floorLoadData.GetLoad(TQSModel.TPLOAD_CARLAJ)`

Objeto Load() com carga distribuída em toda laje

`floor.current.floorSlabData.slabGeometry`

Objeto SlabGeometry() com dados de geometria de laje

`floor.current.floorSlabData.slabGrid`

Objeto SlabGrid() com dados para discretização de laje por grelha

`floor.current.floorSlabData.stairTitle`

Título de escada

`floor.current.floorSlabData.slabExport`

Exportar a laje para visualização 3D (0) não (1) sim

`floor.current.floorSlabData.slabDetailing`

Objeto SlabDetailing() com dados de detalhamento de lajes

`model.current.globalSlabData.stairCase`

Objeto StairCase() com dados de escadas

`model.current.globalSlabData.firstRampNumber`

Número da primeira rampa

11.11.2. Dados de laje

São dados de uma laje existente, podem ser editados.

slabIdent

Objeto SMOBJECTIDENT() de identificação da laje

GetLoad(floor)

Objeto Load() com carga distribuída em toda a laje tf/m2

slabGeometry

Objeto SlabGeometry() com dados de geometria de laje

slabGrid

Objeto SlabGrid() com dados para discretização de laje por grelha

mainAngle

Ângulo principal em graus

slabTemperShrink

Objeto de temperatura / retração de lajes - TemperatureShrink()

slabDetailing

Objeto de detalhamento de lajes - `SlabDetailing()`

auxiliaryFloor

Piso auxiliar

isASTair

Lance de escada (0) Não (1) Sim

stairCase

Objeto `StairCase()` com dados de escada

isALanding

Patamar de escada (0) Não (1) Sim

stairIdent

Título de escadas

isVolumeOnly

Laje somente de volume (0) Não (1) Sim

volumeOnlySlab

Objeto de laje somente de volume - `VolumeOnlySlab()`

userAttrib

Atributos de usuário - `UserAttrib()`

slabExport

Laje exportável para 3D/BIM (0) Não (1) Sim

11.11.3. Dados de geometria da laje - **SlabGeometry()**

type

Tipo de laje `SLABTYPE_XXXXX`

Constante	Uso
<code>TQSModel.SLABTYPE_MACICA</code>	Maciça
<code>TQSModel.SLABTYPE_NERVUR</code>	Nervurada retangular
<code>TQSModel.SLABTYPE_NERVUT</code>	Nervurada trapezoidal
<code>TQSModel.SLABTYPE_VIGOTA</code>	Vigotas treliçadas
<code>TQSModel.SLABTYPE_TRELIC</code>	Treliçada
<code>TQSModel.SLABTYPE_PREFAB</code>	Pré-fabricada
<code>TQSModel.SLABTYPE_MISNER</code>	Mista nervurada (ex: TUPER)

noSelfWeight

Desconsiderar peso proprio (0) Não (1) Sim

GetSelfWeightLoad(floor)

Retorna carga fixa de peso próprio para lajes preo

floor Objeto Floor(), com dados de uma planta

Retorna:

Load Objeto Load(), com carga de peso próprio

thickness

Altura da laje maciça em cm

recess

Rebaixo em cm

GetFloorRecess(floor, slab)

Rebaixo em cm, considerando piso auxiliar

floor Objeto Floor(), com dados de uma planta

slab Objeto Slab() que contém esta geometria

Retorna:

dfspiso Rebaixo da laje, cm, considerando piso auxiliar

additionalTopRecess

Rebaixo adicional superior em cm

additionalBottomRecess

Rebaixo adicional inferior em cm

ribbedTopping

Laje nervurada: capa, cm

ribbedBottomTopping

Laje nervurada: capa inferior, cm

ribHeight

Laje nervurada: altura de nervura cm

ribbedFill

Laje nervurada: peso de enchimento tf/m³

averageSizeX

Laje nervurada: distância média horizontal entre nervuras cm

averageSizeY

Laje nervurada: distância média vertical entre nervuras cm

topSpacingX

Laje nervurada: largura superior de nervura horizontal cm

topSpacingY

Laje nervurada: largura superior de nervura vertical cm

bottomSpacingX

Laje nervurada: largura inferior de nervura horizontal cm

bottomSpacingY

Laje nervurada: largura inferior de nervura vertical cm

inertiaX

Laje nervurada: inércia opcional horizontal cm⁴

inertiaY

Laje nervurada: inércia opcional vertical cm⁴

formworkVolume

Laje nervurada: volume cubeta trapezoidal cm³

mouldManufacturer

Laje nervurada: nome do fabricante de enchimentos

mouldID

Laje nervurada: nome do enchimento

latticeTraverseRibEvery

Laje treliçada: nervura transversal a cada N blocos

latticeTraverseWidth

Laje treliçada: largura da nervura secundária cm

latticeJoistWidth

Laje treliçada: largura da vigota cm

latticeJoistHeight

Laje treliçada: altura da vigota cm

latticeMiniPanel

Laje treliçada: minipainél (0) Não (1) Sim

compositeHeight

Laje mista: altura do perfil, cm

compositeWidth

Laje mista: largura do perfil, cm

compositeFlange

Laje mista: largura da aba, cm

Error! Use the Home tab to apply Título 1 to the text that you want to appear here.175

precastRegion

Pré-moldados: Região construtiva

sectionName

Nome de seção não padrão - catalogada

prestressedReinforcement

Nome da configuração de armaduras protendidas

LoadManufacturer (nomfab, nomenc)

Carrega os dados de moldes de lajes nervuradas. Retorna (0) Se carregou

nomfab Nome do fabricante

nomenc Nome do enchimento

11.11.4. Discretização de laje por grelha - SlabGrid()

Dados para discretização de laje por grelha.

asGrid

Discretizar esta laje em grelha (0) Não (1) Sim

forceAsGrid

Forçar a discretização em laje de escadas (0) Não (1) Sim

plastSupports

Plastificação (0) conforme critérios (1) Sim (2) Não

elstBase

Laje sobre base elástica (0) Não (1) Sim

elstBaseTxSpring

Mola de translação Tx por nó tf/m

elstBaseTySpring

Mola de translação Ty por nó tf/m

elstBaseTzSpring

Mola de translação Tz por nó tf/m

elstBaseGapZPlus

Gap de translação Z+ (m)

elstBaseGapZMinus

Gap de translação Z- (m)

11.11.5. Detalhamento de lajes - SlabDetailing()

detailable

Detalhamento da laje (0) Não (1) Sim

rigidDiafragm

Diafragma rígido (0) Não (1) Sim

prestressed

Protendida (0) Não (1) Sim

cover

Cobrimento diferenciado em cm

exposure

Em contato com o solo (0) Não (1) Sim (2) Exposta ao ambiente

cantileverFactor

Majorador para laje em balanço ou (0.)

cantilever

Verificação de dimensões: Balanço (0) Estimado, geometria (1) Não (2) Sim

estimatedSpan

Vão estimado ou (0.) cm

roofSlab

Verificação de dimensões: Laje de cobertura (0) Não (1) Sim

flatSlab

Verificação de dimensões: Laje plana (0) Geometria (1) Não (2) Sim (3) C/capitéis

bottomReinforcement

Direção de armação: (0) Padrão (1) uma direção (2) duas direções

11.11.6. Dados de escada - StairCase()

Para lajes inclinadas que são lances de escada.

tread

Passo cm

riser

Espelho cm

adjustment

Ajuste inicial ou final cm conforme adjustmentMode

Error! Use the Home tab to apply Título 1 to the text that you want to appear here.177

adjustmentMode

Ajuste (0) Inicial (1) Final

fixedSteps

Número de degraus fixos

winderStair

Representar como escada plissada (0) Não (1) Sim

11.11.7. Laje somente de volume - VolumeOnlySlab()

Lajes somente de volume são elementos que não tem função no modelo estrutural, mas são representados e exportados em 3D. São uma maneira simples de exportar lajes inclinadas e escadas sem a complexidade do modelo estrutural.

insertionX

Ponto de inserção X de laje retangular

insertionY

Ponto de inserção Y de laje retangular

fixedSpace

Fixar comprimento da laje (0) Não (1) Sim

space

Comprimento de laje retangular cm

fixedWidth

Fixar largura da laje (0) Não (1) Sim

width

Largura de laje retangular cm

fixedAngle

Fixar ângulo da laje (0) Não (1) Sim

angle

Direção da laje retangular em graus

recess

Rebaixo inicial em cm

zHeight

Altura Z total em cm

11.12. Fechamento de bordo SlabContour(SMObject)

Linhas de fechamento ou contorno auxiliar, são contornos de laje em bordo livre.

11.12.1. Dados de fechamento de bordo

startX

X inicial cm

startY

Y inicial cm

endX

X final cm

endY

Y final cm

auxiliaryFloor

Piso auxiliar

11.13. Capitel - DropPanel(SMObject)

Um capitel é uma região dentro da laje delimitada por um polígono, e com altura independente da laje.

11.13.1. Dados atuais de um capitel

Usados na criação de um capitel.

`floor.current.floorSlabData.dropPanelThickness`

Altura de capitel cm

`floor.current.floorSlabData.dropPanelDiv`

Divisor de flexão de capitel

11.13.2. Dados de um capitel

dropPanelPolygon

Poligonal do capitel - Polygon()

dropPanelThickness

Altura do capitel, cm

dropPanelDiv

Divisor de inércia à flexão do capitel

auxiliaryFloor

Piso auxiliar

11.14. Forma de laje nervurada SlabMould(SMObject)

São moldes usados na construção de uma laje nervurada.

11.14.1. Dados atuais de forma de laje nervurada

São usados na criação de uma nova forma.

`floor.current.floorSlabData.slabMouldXSize`

Tamanho X da forma de nervura cm

`floor.current.floorSlabData.slabMouldYSize`

Tamanho Y da forma de nervura cm

`floor.current.floorSlabData.slabMouldXSpace`

Espaçamento X de forma de nervura cm

`floor.current.floorSlabData.slabMouldYSpace`

Espaçamento Y de forma de nervura cm

`floor.current.floorSlabData.slabMouldBasePoint`

Ponto base de inserção de nervura (0..5)

11.14.2. Dados de forma de laje nervurada

São dados de uma forma existente, podem ser editados.

insX

X de inserção do molde

insY

Y de inserção do molde

averageSizeX

Tamanho X médio cm

averageSizeY

Tamanho Y médio cm

rotationAngle

Ângulo de rotação em graus

sizeXDelta

Variação de espessura trapezoidal X cm

sizeYDelta

Variação de espessura trapezoidal Y cm

auxiliaryFloor

Piso auxiliar

11.15. Furo em laje SlabOpening(SMObject)

Furo em laje são aberturas definidas por poligonais.

11.15.1. Dados de furo em laje

slabOpeningPolygon

Poligonal do furo em laje - Polygon()

isCut

Tipo de furo (0) Shaft (respeita nervuras) (1) Corte

No tipo *Shaft*, na geração de lajes nervuradas, o programa considera o contorno do furo como um alinhamento, e tenta passar barras de grelha no contorno.

inConcretTopping

Somente na capa da laje nervurada (0) Não (1) Sim

auxiliaryFloor

Piso auxiliar

11.16. Furo em pilar - ColumnOpening(SMObject)

Em desenvolvimento.

11.17. Estaca para laje radier SlabFoundationPile (SMObject)

Em desenvolvimento

11.18. Carga adicional em laje AdditionalLoad(SMObject)

É uma carga que atua sobre toda a laje, além da declarada nos dados da laje.

11.18.1. Dados atuais de carga adicional

Usados na criação de uma carga nova.

Error! Use the Home tab to apply Título 1 to the text that you want to appear here.181

```
model = TQSModel.Model ()  
floor = model.floors.GetFloor ("nome-da-planta")  
  
floor.current.floorLoadData.GetLoad (TPLOAD_CARADI)  
Objeto Load() com a carga distribuída.
```

11.18.2. Dados de carga adicional

load

Valor da carga - Objeto Load ()

insertionX

Ponto de inserção X

insertionY

Ponto de inserção Y

auxiliaryFloor

Piso auxiliar

11.19. Carga distribuída por área AreaDistributedLoad(SMObject)

É uma carga distribuída definida por uma poligonal fechada. Se a carga se sobrepor a vários elementos estruturais, o programa quebra a carga e faz o lançamento em partes.

11.19.1. Dados atuais de carga adicional

Usados na criação de uma carga nova.

```
model = TQSModel.Model ()  
floor = model.floors.GetFloor ("nome-da-planta")  
  
floor.current.floorLoadData.GetLoad (TPLOAD_CARADI)  
Objeto Load() com a carga distribuída.
```

11.19.2. Dados de carga adicional

load

Valor da carga - Objeto Load()

polygon

Poligonal fechada onde a polig é distribuída - Polygon()

auxiliaryFloor

Piso auxiliar

11.20. Carga concentrada ConcentratedLoad(SMObject)

Carga pontual sobre laje, viga ou pilar.

11.20.1. Dados atuais de carga concentrada

São usados na criação de uma nova carga

```
model = TQSModel.Model ()  
floor = model.floors.GetFloor ("nome-da-planta")
```

```
floor.current.floorLoadData.GetLoad (tipo)
```

Objeto Load() com a carga concentrada. Tipo é uma das direções de força/momento nas constantes TQSModel.TPLOAD_FORxxx.

11.20.2. Dados de carga concentrada

loadFx

Força Fx - tf - Somente pilar - Objeto Load()

loadFy

Força Fy - tf - Somente pilar - Objeto Load()

loadFz

Força Fz - tf - Pilar, viga ou laje - Objeto Load()

loadMx

Força Mx - tfm - Somente pilar - Objeto Load()

loadMy

Força My - tfm - Somente pilar - Objeto Load()

insertionX

Ponto de inserção X

insertionY

Ponto de inserção Y

auxiliaryFloor

Piso auxiliar

11.21. Carga distribuída linearmente LinearyDistributedLoad(SMObject)

É uma carga distribuída por metro, entre dois pontos. Se a carga estiver sobre mais de uma laje e/ou viga, ela será quebrada para cada elemento.

11.21.1. Dados atuais de carga distribuída linearmente

```
model = TQSModel.Model ()  
floor = model.floors.GetFloor ("nome-da-planta")  
  
floor.current.floorLoadData.GetLoad (TPLOAD_CARLIN)  
Objeto Load() com a carga distribuída linearmente.
```

11.21.2. Dados de carga distribuída linearmente

load

Valor da carga - Objeto Load()

startPointX

Ponto inicial X

startPointY

Ponto inicial Y

endPointX

Ponto final X

endPointY

Ponto final Y

auxiliaryFloor

Piso auxiliar

wall

Parede 3D importada do BIM - Objeto Wall() ou None

11.22. Carga de empuxo SoilPressureLoad(SMObject)

Define a carga de empuxo entre dois pontos, seguindo a convenção interativa do Modelador. Uma carga de empuxo é um objeto espacial, com definição de pavimento inicial e final de atuação.

11.22.1. Dados atuais de carga de empuxo

São definidos por um objeto tipo `SoilPressureLoad()` atual.

```
model = TQSModel.Model ()
```

```
model.current.globalSoilpress.soilPressureLoad
```

Objeto do tipo `SoilPressureLoad()` com carga de empuxo

11.22.2. Dados de carga de empuxo

topLoad

Carga de empuxo no topo - Objeto `Load()`

baseLoad

Carga de empuxo na base - Objeto `Load()`

topFloorName

Nome da planta do topo da carga de empuxo

baseFloorName

Nome da planta da base da carga de empuxo

startPointX

Ponto inicial X

startPointY

Ponto inicial Y

endPointX

Ponto final X

endPointY

Ponto final Y

11.23. Temperatura e retração de vigas e lajes - `TemperatureShrink()`

Dados de temperatura e retração de vigas, se habilitados nos dados do edifício.

temperatureDefined

Definição de variação de temperatura ao longo da viga (0) Não (1) Sim

uniformVariation

Variação longitudinal de temperatura em graus Celcius

uniformVariation2

TQS Informática Ltda. - Rua dos Pinheiros 706 / casa 2 - 05422-001 - São Paulo/SP - Tel.: (011) 3883-2722 - Fax: (011) 3083-2798

Error! Use the Home tab to apply Título 1 to the text that you want to appear here.185

Variação longitudinal de temperatura em graus Celcius - caso 2

transverseVariation

Variação transversal de temperatura em graus Celcius

transverseVariation2

Variação transversal de temperatura em graus Celcius - caso 2

shrinkageDefined

Definição de retração ao longo da viga (0) Não (1) Sim

uniformShrinkage

Variação longitudinal de temperatura equivalente à retração em graus Celcius

11.24. Modos de visualização - VisModes()

Os modos de visualização se aplicam a diversos tipos de desenho. O objeto que define os modos é obtido a partir do modelo estrutural:

```
model = TQSModel.Model ()
```

```
model.current.visData.GetVisModes (imode)
```

Objeto VisModes () com m odos de visualização

Onde imode pode ser:

Constante	Uso
TQSModel.VISMODE_MATUAL	Modo de visualização atual
TQSModel.VISMODE_FORMAS	Desenho de formas FORnnnn.DWG
TQSModel.VISMODE_VERIFI	Modo de verificação
TQSModel.VISMODE_MODELO	Desenho MODELO.DWG
TQSModel.VISMODE_GRUPOA	Grupo A de visualização do usuário
TQSModel.VISMODE_GRUPOB	Grupo B de visualização do usuário

Os modos de visualização são separados nos seguintes subobjetos:

Subobjeto	Tipo	Uso
columns	VisModeColumns ()	Pilares
beams	VisModeBeams ()	Vigas
slabs	VisModeSlabs ()	Lajes
bim	VisModeBim ()	BIM
loads	VisModeLoads ()	Cargas
foundations	VisModeFoundations ()	Fundações
precast	VisModePrecast ()	Pré-moldados
details	VisModeDetails ()	Outros

11.24.1. Modos de visualização de pilares

`columns.columns`

Pilares (0) Não (1) Sim

`columns.name`

Título de pilar (0) Não (1) Sim

`columns.dimensions`

Dimensões de pilares (0) Não (1) Sim

`columns.overlapUpper`

Sobreposição de pilar atual e superior (0) Não (1) Sim

`columns.overlapBottom`

Sobreposição de pilar atual e inferior (0) Não (1) Sim

`columns.gridSupport`

Condições de apoio em grelha (0) Não (1) Sim

`columns.axes`

Eixos de pilares (0) Não (1) Sim

`columns.details`

Outros dados de detalhamento de pilares (0) Não (1) Sim

`columns.circle`

Pilares circulares como círculos (0) Não (1) Sim

`columns.shearStrip`

Mostra lâmina de pilares parede (0) Não (1) Sim

`columns.shearWallDiscret`

Discretização de pilar parede (0) Não (1) Sim

`columns.fixedPoint`

Ponto fixo do pilar (0) Não (1) Sim

`columns.imposedRestraint`

Travamento imposto (0) Não (1) Sim

`columns.fictitiousSupport`

Apoios fictícios (0) Não (1) Sim

Error! Use the Home tab to apply Título 1 to the text that you want to appear here.187

columns.horizontalOpenings

Furos em pilares (0) Não (1) Sim

columns.metalCheck

Resultados de pilares no MetalCheck (0) Não (1) Sim

11.24.2. Modos de visualização de vigas

beams.beams

Vigas (0) Não (1) Sim

beams.name

Título de vigas (0) Não (1) Sim

beams.dimensions

Título de vigas (0) Não (1) Sim

beams.axes

Eixos de vigas (0) Não (1) Sim

beams.connections

Vinculações de vigas (0) Não (1) Sim

beams.nodes

Nós de vigas (0) Não (1) Sim

beams.details

Outros dados de detalhamento de vigas (0) Não (1) Sim

beams.sectionT

Seção T e B colaborante (0) Não (1) Sim

beams.cantilever

Pintar vigas em balanço (0) Não (1) Sim

beams.openings

Furos horizontais (0) Não (1) Sim

beams.transferBeam

Viga de transição (0) Não (1) Sim

beams.metalCheck

Vigas no MetalCheck (0) Não (1) Sim

beams.precast

Pintar vigas protendidas (0) Não (1) Sim

11.24.3. Modos de visualização de lajes

slabs.slabs

Lajes (0) Não (1) Sim

slabs.name

Título de lajes (0) Não (1) Sim

slabs.dimensions

Dimensões de lajes (0) Não (1) Sim

slabs.mainDirection

Símbolo de ângulo principal da laje (0) Não (1) Sim

slabs.contour

Contorno da laje (0) Não (1) Sim

slabs.details

Outros dados de detalhamento de lajes (0) Não (1) Sim

slabs.ribs

Nervuras em lajes (0) Não (1) Sim

slabs.openings

Furos em lajes (0) Não (1) Sim

slabs.caps

Capitéis (0) Não (1) Sim

slabs.trapezoidalRibs

Bordas de nervuras trapezoidais (0) Não (1) Sim

slabs.ficticiusRibs

Nervuras em maciços (0) Não (1) Sim

slabs.prefabDir

Direção de nervuras (0) Não (1) Sim

slabs.cantilever

Laje em balanço (0) Não (1) Sim

11.24.4. Modos de visualização de BIM

bim.importStatus

Status de importação do BIM (0) Não (1) Sim

bim.notExported

Elementos não exportados para o BIM (0) Não (1) Sim

Error! Use the Home tab to apply Título 1 to the text that you want to appear here.189

bim.importedPipes

Tubos importados (0) Não (1) Sim

bim.pipeView

Visualizar tubos (0) Só na vista 3D (1) Na vista 3D e 2D

bim.importedWalls

Paredes importadas (0) Não (1) Sim

bim.objects3D

Objetos 3D (0) Não (1) Sim

bim.plan2D

Planta 2D como referência para o 3D (0) Não (1) Sim

bim.reference3D

Referência externa 3D (0) Abaixo (1) Acima (2) Ambos

bim.basePoint

Ponto base de projeto (0) Não (1) Sim

bim.surveyPoint

Ponto de levantamento topográfico (0) Não (1) Sim

11.24.5. Modos de visualização de cargas

loads.beamDistributedLoad

Cargas distribuídas nas vigas (0) Não (1) Sim

loads.slabDistributedLoad

Cargas distribuídas nas lajes (0) Não (1) Sim

loads.concentratedLoad

Cargas concentradas (0) Não (1) Sim

loads.linearLoad

Cargas concentradas (0) Não (1) Sim

loads.areaDistributedLoad

Carga distribuída por área (0) Não (1) Sim

loads.soilPressureLoad

Cargas de empuxo (0) Não (1) Sim

loads.showCaseOnlyLoad

Restringir ao caso (0)=todos

`loads.manualWindDistribution`

Distribuição manual de vento (0) Não (1) Sim

`loads.manualWindPrefix`

Prefixo do caso de vento para distribuição manual (""=Todos)

`loads.loadViewMode`

Mostrar cargas em (0) 2D (1) 3D (2) 2D e 3D

11.24.6. Modos de visualização de fundações

`foundations.foundations`

Fundações (0) Não (1) Sim

`foundations.name`

Títulos de fundações (0) Não (1) Sim

`foundations.dimensions`

Dimensões de fundações (0) Não (1) Sim

`foundations.details`

Outros dados de detalhamento de fundações (0) Não (1) Sim

`foundations.piles`

Estacas de blocos (0) Não (1) Sim

`foundations.cutOffLevel`

Cotas de arrasamento (0) Não (1) Sim

`foundations.pierDimension`

Dimensão da base do tubulão (0) Não (1) Sim

`foundations.pileQuantity`

Quantidade e bitola de estacas (0) Não (1) Sim

`foundations.pileIdent`

Número da estaca (0) Não (1) Sim

`foundations.pileDimensions`

Diâmetro da estaca (0) Não (1) Sim

11.24.7. Modos de visualização de pré-moldados

`precast.constructionRegions`

Regiões pré-moldadas (0) Não (1) Sim

Error! Use the Home tab to apply Título 1 to the text that you want to appear here.191

precast.floorPlanGroups

Título dos grupos de formas (0) Não (1) Sim

precast.reinforcementGroups

Título dos grupos de armação (0) Não (1) Sim

precast.corbelsName

Título de consolos (0) Não (1) Sim

precast.corbels

Consolos (0) Não (1) Sim

precast.corbelsData

Dados de consolos (0) Não (1) Sim

precast.accessories

Acessórios (0) Não (1) Sim

precast.slabsReport

Resultados do cálculo de lajes pré-moldadas, se disponível (0) Não (1) Sim

precast.showSlabsReport

Mostrar o relatório após o cálculo de lajes pré-moldadas (0) Não (1) Sim

precast.beamsSectionTitle

Título de seções catalogadas vigas (0) Não (1) Sim

precast.sectionsLines

Linhas de contorno das seções não padrão (0) Não (1) Sim

precast.inserts

Insertos (0) Não (1) Sim

precast.facades

Fachadas (0) Não (1) Sim

11.24.8. Outros modos de visualização

details.crossSections

Cortes da planta de formas (0) Não (1) Sim

details.dimensions

Cotas (0) Não (1) Sim

details.controlPoints

Pontos de controle de cotagem (0) Não (1) Sim

`details.formworkArea`

Áreas de vigas, pilares, lajes (0) Não (1) Sim

`details.axes`

Eixos (0) Não (1) Sim

`details.centroidsTable`

Tabela de baricentros (0) Não (1) Sim

`details.overlapAuxFloors`

Sobrepõe pisos auxiliares (0) Não (1) Sim

`details.elementsTable`

Tabela de elementos (0) Não (1) Sim

`details.tunnelReference`

Centro de torção - referência de túnel de vento (0) Não (1) Sim

`details.linesOutOfXY`

Linhas fora da direção X/Y global (0) Não (1) Sim

`details.disablePlottingHatches`

Inibe display de hachuras plotadas (0) Não (1) Sim

`details.hatchedUnevenness`

Legenda e hachura de desníveis (0) Não (1) Sim

`details.floorLevels`

Cota pavimentos em desnível (0) Não (1) Sim

`details.floorLevelsTable`

Tabela de nível dos pavimentos (0) Não (1) Sim

`details.towerFence`

Cerca da torre (0) Não (1) Sim

11.25. Atributos BIM de objeto - UserAttrib()

É um grupo de atributos na forma chave=valor, que podem ser associados a elementos do modelo estrutural.

`usrattribClear()`

Limpa atributos do usuário

`usrattribNum()`

Retorna o número de atributos

Error! Use the Home tab to apply Título 1 to the text that you want to appear here.193

usratribInsert(chave, sval)

Insere novo atributo

chave Chave

sval Valor

usratribErase(chave)

Apaga atributo existente

chave Chave do atributo a apagar

usratribBegin()

Prepara para ler todos os atributos

usratribReadNext()

Lê próximo atributo

Retorna:

chave Chave

sval Valor

istat (0) Ok (!=0) Acabou

usratribFind(chave)

Lê próximo atributo

chave Chave a pesquisar

Retorna:

sval Valor

istat (0) Ok (!=0) Acabou

11.26. Atributos BIM globais - GlobalAttrib()

Consistem em grupos de atributos comuns gravados para todos os elementos exportados para o BIM. Cada grupo de atributos contém um ou mais objetos tipo `UserAttrib()`.

Clear()

Limpa atributos globais

SetGroup(nomgrupo, iabrang, ielementos, ipiso, nompla)

Define novo grupo. Se já existir, acessa grupo existente

nomgrupo Nome do grupo

iabrang Abrangência: BIM_GLBA_XXXX

ielementos Elementos definidos TYPE_XXXX (TYPE_INDEF=todos)

ipiso Piso para iabrang==BIM_GLBA_UMPISO

nompla Pavimento para iabrang==BIM_GLBA_PLANTA

Retorna:

objeto `UserAttrib()` de atributos do grupo

Constante	Uso
<code>TQSModel.BIM_GLBA_GLOBAL</code>	Todos do edifício
<code>TQSModel.BIM_GLBA_PLANTA</code>	Todos de uma planta
<code>TQSModel.BIM_GLBA_UMPISO</code>	Todos de um piso

NumGroups ()

Retorna o número de grupos de atributos

EraseGroup (nomgrupo)

Apaga atributo existente

ReadBegin ()

Prepara para ler todos os atributos

ReadNext ()

Lê próximo atributo

Retorna:

```

nomgrupo      Nome do grupo
iabrang       Abrangência: TQSModel.BIM_GLBA_xxxx
ielementos   Elementos definidos TQSModel.TYPE_xxxx (TYPE_INDEF=todos)
ipiso        Piso para iabrang== TQSModel.BIM_GLBA_UMPISO
nompla       Pavimento para iabrang== TQSModel.BIM_GLBA_PLANTA
UserAttrib() Objeto de atributos do grupo
istat        (!=0) quando acabar atributos

```

ReadGroup (nomgrupo)

Lê atributos dado nome do grupo

nomgrupo Nome do grupo

Retorna:

```

iabrang       Abrangência: TQSModel.BIM_GLBA_xxxx
ielementos   Elementos definidos TQSModel.TYPE_xxxx (TYPE_INDEF=todos)
ipiso        Piso para iabrang== TQSModel.BIM_GLBA_UMPISO
nompla       Pavimento para iabrang== TQSModel.BIM_GLBA_PLANTA
UserAttrib() Objeto de atributos do grupo
istat        (!=0) o grupo não existe

```

Sort ()

Classifica grupos por nome

11.27. Parede 3D importada do BIM – Wall()

Em desenvolvimento

11.28. Testes do TQSModel.PY

Os seguintes programas mostram o funcionamento do TQSModel.

Programa	Finalidade
TSTMODEL1.PY	Mostra a criação de um edifício simples e dos principais objetos do modelo estrutural: pilares, vigas, cruzamento de vigas com definição de quem apoia, furos em vigas, lajes, capiteis, formas de nervuras, fechamentos de bordo, furos em lajes, cargas concentradas, lineares, distribuídas, adicionais e empuxo, fundações em blocos e sapatas.
TSTMODEL2.PY	Mostra a criação de um edifício com pilares helicoidais, trabalhosos para criar manualmente, mas simples por programa. Demonstra a distribuição automática de formas de nervuras em batch para qualquer formato de laje.
TSTMODEL3.PY	Lista elementos estruturais do edifício "Mod-Padrão" distribuído para teste de instalação. Leitura de atributos de vigas, pilares, lajes e fundações.

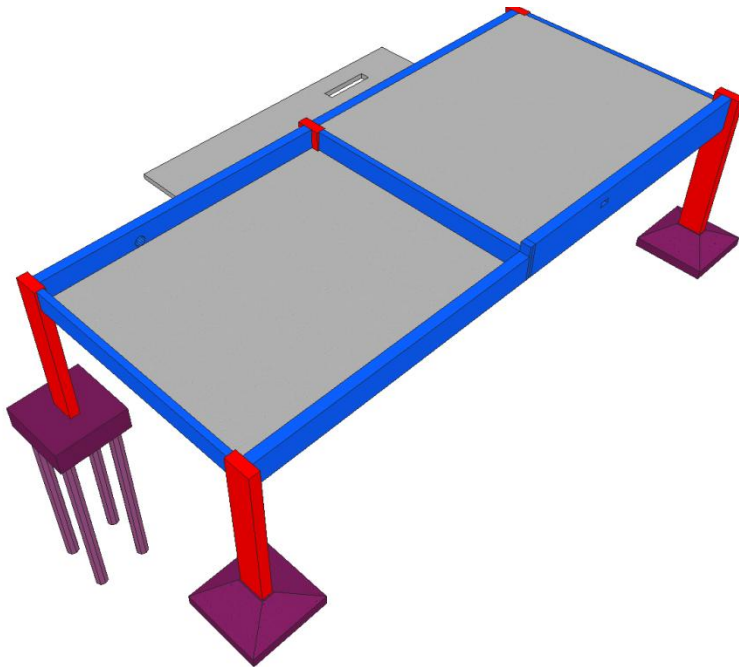
11.28.1. Programa TSTMODEL1.PY

Este programa gera um edifício novo de nome TSTMODEL1. O modelo estrutural é aberto sempre no contexto do edifício. Então este programa primeiro usa a classe TQSBuild para criar um edifício novo e depois torna atual a pasta raiz, daonde o modelo estrutural pode ser aberto para gravação:

```
build                = CriarDadosDoEdificio ()
build.RootFolder    ("TSTMODEL1")
model                = TQSModel.Model ()
istat                = model.file.OpenNewModel ()
```

Os elementos estruturais são criados sempre usando um pavimento como base:

```
floorTerreo          = model.floors.GetFloor ("Terreo ")
floorFundacao        = model.floors.GetFloor ("Fundacao")
CriarPilares         (model, floorTerreo)
CriarVigasTerreo     (model, floorTerreo)
CriarLajesTerreo     (model, floorTerreo)
CriarCargasTerreo    (model, floorTerreo)
CriarFundacoes       (model, floorFundacao)
```



11.28.2. Programa TSTMODEL2.PY

O programa para este modelo TSTMODEL2 é mais simples que o anterior porque gera menos tipos de elementos, mas a estrutura final é mais complexa, com 20 pisos com pilares em formato helicoidal, laje plana nervurada e vigas no contorno que acompanham os pilares. Os pilares circulares helicoidais são marcados como inclinados, e um loop gera todos os pilares de um piso e gira as coordenadas de piso a piso:

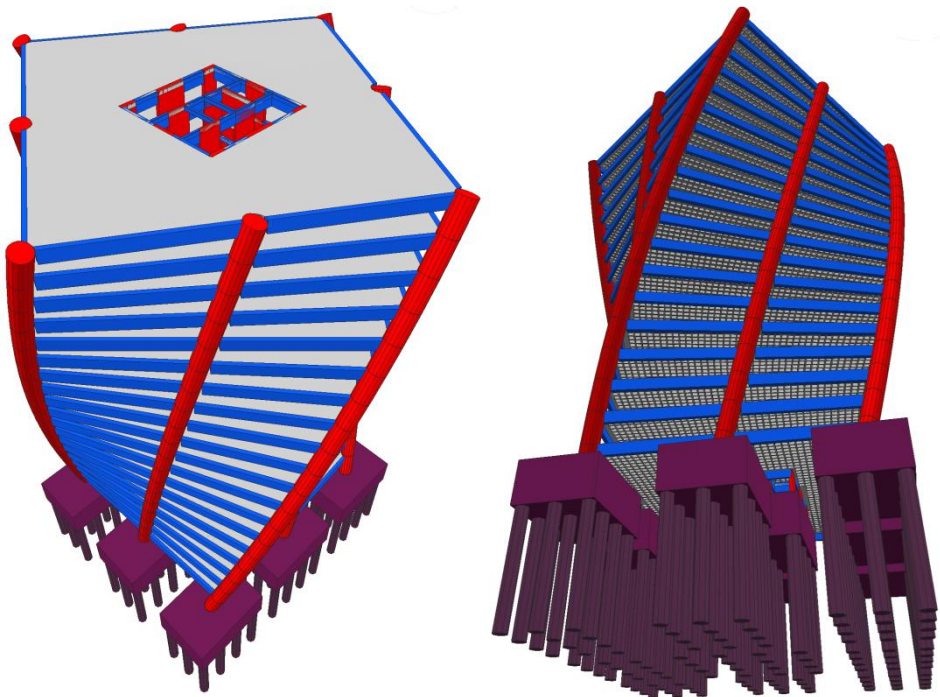
```
xy          = ((-1316., 1274.),....)
for         coords in xy:
    column   = floor.create.CreateColumn (coords [0], coords [1])
    for      ipla in range (1, 20):
        nompla      = PegarNomePlanta (ipla)
        isec        = ipla
        columnSection = column.ColumnCreateSection (isec, nompla)
        xins        = columnSection.insertionX
        yins        = columnSection.insertionY
        xins, yins  = TransformarCoords (1., xins, yins)
        columnSection.insertionX = xins
        columnSection.insertionY = yins
        column.ColumnSectionUpdateGeometry (columnSection)
```

As lajes em cada piso recebem o mesmo ângulo principal do giro dos pilares, assim como as formas de nervuras distribuídas. Para especificar a geometria da laje nervurada, carregamos os dados de fabricante e cubetas Atex®:

```
nompla = PegarNomePlanta (ipla)
floor  = model.floors.GetFloor (nompla)
slabGeometry= floor.current.floorSlabData.slabGeometry
slabGeometry.type= TQSModel.SLABTYPE_NERVUT
istat=slabGeometry.LoadManufacturer("Atex","Atex 650 Capa 7.5 H 35")
```

Depois criamos a laje e uma cubeta, que mandamos distribuir:

```
angpri      = 3.0*float (ipla)
xins        = -609.0
yins        = 101.7
slab         = floor.create.CreateSlab (xins, yins, angpri)
slabMould    = floor.create.CreateSlabMould (slab, xins, yins)
floor.util.DistributeSlabMould (slab, slabMould)
```



O núcleo do edifício tem coordenadas fixas, e é formado por um pilar genérico e outros retangulares, ligados por vigas. A distribuição de formas de nervuras reconhece o contorno do núcleo.

11.28.3. Programa TSTMODEL3.PY

Este programa demonstra que é possível ler e listar um modelo. Esta lógica também serve para modificar modelos – basta iterar, carregar elementos do modelo e modificá-los.

O programa começa abrindo um modelo existente – "Mod-Padrão", que vem na pasta de testes \TQSW\USUARIO\TESTE com o nome de MODPLA.TQS e deve estar descompactado.

```
GLB_NOMEDIF = "Mod-Padrão"
build = TQSBuild.Building ()
if (build.RootFolder (GLB_NOMEDIF) != 0):
    TQSUtil.writef ("O edifício [%s] não existe" % GLB_NOMEDIF)
    return
model = TQSModel.Model ()
if (model.file.OpenModel () != 0):
    TQSUtil.writef ("Não abri o edifício [%s]" % GLB_NOMEDIF)
```

Um loop pelas plantas do edifício faz a extração de cada uma delas:

```
floorslist      = model.floors
numpla          = floorslist.GetNumFloors ()
for indpla in range (1, numpla+1):
    nompla       = floorslist.GetFloorName (indpla)
    floor        = floorslist.GetFloor (nompla)
    ExtrairPlanta (model, nompla, floor)
```

Em cada planta, existe uma lista separada de objetos por tipo de elemento estrutural. O loop pelos tipos de elemento é este:

```
numtypes        = model.type.GetNumTypes ()
for itype in range (0, numtypes):
    ExtrairTipo  (model, floor, itype)
```

Nem todos os tipos de elementos são tratados atualmente pela interface, somente os listados neste manual. Listamos como exemplo as vigas, pilares e lajes:

Error! Use the Home tab to apply Título 1 to the text that you want to appear here.199

```
numobjetos= floor.iterator.GetNumObjects (itype)
for iobjeto in range (0, numobjetos):
    smobject = floor.iterator.GetObject (itype, iobjeto)
    if (smobject is None):
        continue
    if (itype == TQSModel.TYPE_VIGAS):
        ListarViga (floor, smobject)
    elif (itype == TQSModel.TYPE_SECPIL):
        ListarPilar (model, floor, smobject)
    elif (itype == TQSModel.TYPE_LAJES):
        ListarLaje (model, floor, smobject)
```

O objeto retornado por GetObject é derivado do tipo SMOBJECT, é um dos objetos do Modelador. A rotina ListarViga por exemplo, recebe este objeto como tipo Beam():

```
def ListarViga (floor, beam):
```

A listagem emitida é um exemplo do tipo de informação que pode ser obtida na leitura de um modelo.

11.28.4. Programa TSTMODEL4.PY

O TSTMODEL4.PY demonstra o uso da interface Python para testar múltiplas soluções estruturais, variando elementos de um modelo. Após duplicar e modificar um edifício, o módulo TQSExec é usado para fazer um processamento global. No final, os edifícios resultantes poderão ser comparados.

Assim como em TSTMODEL3.PY, abrimos o modelo "Mod-Padrão", que deve estar descompactado. Este edifício é copiado três vezes com sufixos diferentes ("A", "B" e "C") e em cada cópia são alterados quatro pilares, mais duas vigas do pavimento "Tipo". A lista edifícios contém as alterações, como abaixo:

```
edificios=
(
    ("A",
    (
        ("Fundacao",
        (
            (GLB_PILAR, 5, 3, -295.5, 544.5, 40., 19.),
            (GLB_PILAR, 8, 2, -295.5, 315.5, 40., 19.),
            (GLB_PILAR, 6, 3, -000.5, 544.5, 35., 14.),
            (GLB_PILAR, 9, 2, -000.5, 315.5, 35., 14.),
        )
        ),
    ("Tipo",
    (
        (GLB_VIGA, 2, 0, 000.0, 000.0, 14., 35.),
        (GLB_VIGA, 3, 0, 000.0, 000.0, 14., 35.),
```

```

        ),
    ),
),
("B",
.....

```

Temos uma lista de e 3 edifícios "A", "B" e "C", cada edifício com alteração em pavimentos "Fundacao" e "Tipo", e cada pavimento com elementos a alterar – vigas ou pilares. Os parâmetros de cada lista se referem no caso de pilares ao número do pilar a ser alterado, o ponto de inserção, e a nova dimensão de pilar retangular. No caso das vigas, o seu número e sua nova dimensão.

Cada versão do edifício primeiro é copiada:

```

GLB_NOMEDIF = "Mod-Padrão"
for edificio in edificios:
    nomedi = GLB_NOMEDIF + "-" + edificio [0]
    building.file.SuppressMessages (1)
    building.file.SaveAs (nomedi)

```

Depois, para cada elemento a modificar, a rotina `ModificarModelo` é chamada para iterar por todos os elementos do mesmo tipo até achar o elemento com o número correto. A modificação do pilar redefine o seu ponto de inserção, para garantir que com o tamanho diferente continue na posição correta:

```

columnSection = pilar.ColumnGetSection (0)
columnSection.insertionX = xins
columnSection.insertionY = yins
columnSection.columnInsertion.insertionType = 1
columnSection.columnInsertion.insertionCorner = icanto
columnSection.columnGeometry.sectionB1 = dimb
columnSection.columnGeometry.sectionH1 = dimh
pilar.ColumnSectionUpdateGeometry (columnSection)

```

Mas como a alteração na geometria provoca uma excentricidade na fundação, vamos à fundação sob o pilar e definimos um pilar fictício para não causar erro de extração:

```

fundacao = pilar.GetFoudation ()
if (fundacao != None):
    if (fundacao.columnIsAFoundation != 0 and \
        fundacao.foundationDefined != 0):
        fundacao.foundationData.fictitiousColumn = 1
        fundacao.foundationData.fictitiousColumnXDim = dimb
        fundacao.foundationData.fictitiousColumnYDim = dimh

```

Error! Use the Home tab to apply Título 1 to the text that you want to appear here.201

No caso das vigas alteradas, tínhamos no modelo original uma mudança de seção. Alteramos a geometria para que todas as seções sejam iguais:

```
numnodes      = viga.NumNodes ()
for           inode in range (0, numnodes-1):
    viga.SetSegmentBeamGeometryDefined (inode, 0)
beamGeometry = viga.GetBeamGeometry ()
beamGeometry.width = dimb
beamGeometry.depth = dimh
```

As alterações de contorno de lajes causada pela alteração na dimensão dos pilares faz com que o contorno das lajes fique desatualizado no final. No final, varremos todas as plantas e mandamos refazer as intersecções:

```
numfloors      = model.floors.GetNumFloors ()
for            ifloor in range (1, numfloors+1):
    nompav      = model.floors.GetFloorName (ifloor)
    floor       = model.floors.GetFloor (nompav)
    floor.util.DoIntersections ()
```

O processamento global de cada edificio é feito o TQSExec, onde a partir da pasta Espacial:

```
job            = TQSExec.Job ()
job.EnterTask (TQSExec.TaskFolder (nomedi, \
    TQSExec.TaskFolder.FOLDER_FRAMES))
job.EnterTask (TQSExec.TaskGlobalProc ( \
    floorDraw = 1, slabs = 2, beams = 3, \
    columns = 2, foundations = 1, stairs = 1))
job.Execute    ()
```


This image shows a single sheet of white paper with horizontal ruling lines. The lines are evenly spaced and run across the width of the page. There are no margins, text, or other markings on the paper.



TQS Informática Ltda.

Rua dos Pinheiros 706 / casa 2 - São Paulo / SP - 05422-001

Tel. (011) 3883-2722 tqs@tqs.com.br - www.tqs.com.br

TQS